

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

درس یادگیری ماشین

استاد: دکتر امیرحسین نیکوفرد

تمرین سری سوم

گروه جاویدنام سپهر ایران

نام و نام خانوادگی	محمدامین محمدیون شبستری
شماره دانشجویی	۴۰۱۲۲۵۰۳
نام و نام خانوادگی	محمدسبحان سخایی
شماره دانشجویی	۴۰۱۱۹۲۵۳
نام و نام خانوادگی	آرین خوشنویس
شماره دانشجویی	۴۰۱۱۷۹۸۳
تاریخ	اردیبهشت ۱۴۰۵



فهرست مطالب

۵	سوال ۱: تحلیل ریاضیاتی محوشدگی گرادیان و نقش Cell State در LSTM	۵
۵	۱.۱ الف) تحلیل محوشدگی گرادیان در RNN استاندارد	۵
۷	۲.۱ ب) حل مشکل محوشدگی گرادیان در LSTM	۷
۱۰	سوال ۲: استخراج معادلات پس انتشار Backward Pass برای گیت Reset در GRU	۱۰
۱۰	۱.۲ ۱-۲- قاعده Chain Rule بر اساس معادلات حاکم	۱۰
۱۱	۲.۲ ۲-۲- گرادیان خطا نسبت به حالت پنهان	۱۱
۱۱	۳.۲ ۳-۲- گرادیان خطا نسبت به خروجی پنهان کاندید	۱۱
۱۱	۴.۲ ۴-۲- گرادیان خطا نسبت به گیت Reset	۱۱
۱۲	۱.۴.۲ ۱-۴-۲- گرادیان خروجی پنهان کاندید نسبت به a_t	۱۲
۱۲	۲.۴.۲ ۲-۴-۲- گرادیان a_t نسبت به گیت Reset	۱۲
۱۳	۵.۲ ۵-۲- آخرین عبارت Chain Rule: گرادیان خطا نسبت به وزن های گیت Reset	۱۳
۱۴	۱.۵.۲ ۱-۵-۲- گرادیان گیت Reset نسبت به c_t (مشتق تابع سیگموئید)	۱۴
۱۴	۲.۵.۲ ۲-۵-۲- گام آخر: گرادیان c_t نسبت به W_r	۱۴
۱۵	۶.۲ ۶-۲- جمع بندی روابط و فرمول نهایی	۱۵
۱۶	سوال ۳: پیاده سازی Coupled Input-Forget Gate LSTM از پایه با Numpy	۱۶
۱۶	۱.۳ Optimized Coupled Input-Forget Gate LSTM Implementation	۱۶
۱۶	۱.۱.۳ معادلات مدل	۱۶
۱۷	۲.۱.۳ کد	۱۷
۲۶	۳.۱.۳ پیاده سازی مبتنی بر محاسبات ماتریسی مجزا	۲۶
۲۸	۴.۱.۳ پیاده سازی بهینه با ترکیب ماتریس ها	۲۸
۲۹	۵.۱.۳ نتایج تحلیل کاهش زمان اجرا	۲۹
۳۱	سوال ۴: پیاده سازی برداری پردازش دسته ای (Batched Processing) و پس انتشار در زمان (BPTT) با ترفندهای بهینه سازی	۳۱
۳۱	۱.۴ پیاده سازی برداری شبکه Vanilla RNN با قابلیت پردازش دسته ای (Batched Processing)	۳۱
۳۱	۱.۱.۴ مروری بر سازوکار شبکه های RNN و مدل Vanilla RNN	۳۱
۳۱	۲.۱.۴ طراحی کلاس شبکه با پشتیبانی از پردازش دسته ای	۳۱
۳۲	۳.۱.۴ بهینه سازی با ترفند الحاق ماتریسی (Matrix Concatenation)	۳۲
۳۲	۲.۴ پیاده سازی پس انتشار در زمان (BPTT) و کنترل انفجار گرادیان	۳۲
۳۲	۱.۲.۴ مروری بر الگوریتم Backpropagation Through Time	۳۲
۳۳	۲.۲.۴ پیاده سازی متد backward	۳۳
۳۳	۳.۲.۴ مشکل انفجار گرادیان و استفاده از Gradient Clipping	۳۳
۳۴	۳.۴ ایجاد داده مصنوعی Many-to-One و آموزش مدل با SGD	۳۴



۳۴	تولید مجموعه داده مصنوعی	۱.۳.۴
۳۴	رویکرد اول: دنباله کوتاه ($S_L = 3$) و نرخ یادگیری بالا	۲.۳.۴
۳۵	رویکرد دوم: دنباله بلند ($S_L = 10$) و نرخ یادگیری 0.9	۳.۳.۴
۳۶	تحلیل نوسانات تابع هزینه و دقت	۴.۳.۴
۳۸	رویکرد سوم: دنباله بلند ($S_L = 10$) و نرخ یادگیری پایین (0.1)	۵.۳.۴
۳۹	کد	۶.۳.۴

۵۰	سوال ۵: پیاده سازی حرفه ای و ترفند های آموزش RNN ها در Pytorch	۵
۵۰	۱.۵ داده های ترتیبی (Sequential Data)	
۵۱	۲.۵ چالش توالی های طولانی (Long Sequences)	
۵۲	۳.۵ پدیده انفجار گرادیان (Exploding Gradient)	
۵۳	۴.۵ محاسبات هدررفته ناشی از Padding	
۵۴	۵.۵ ساختار و نحوه عملکرد BiLSTM	
۵۴	۱.۵.۵ مرور ساختار LSTM	
۵۵	۲.۵.۵ ایده اصلی Bidirectional Processing	
۵۶	۳.۵.۵ تحلیل شهودی عملکرد	
۵۶	۴.۵.۵ کاربرد در دسته بندی دنباله (Sequence Classification)	
۵۶	۵.۵.۵ مزایای BiLSTM	
۵۶	۶.۵.۵ هزینه محاسباتی	
۵۷	۷.۵.۵ جمع بندی مفهومی	
۵۷	۶.۵ بررسی pack_padded_sequence و pad_packed_sequence	
۵۸	۱.۶.۵ مشکل مهم تر از اتلاف محاسبات	
۵۸	۲.۶.۵ راه حل: Packed Sequence	
۵۹	۳.۶.۵ چرا بعداً از pad_packed_sequence استفاده می کنیم؟	
۵۹	۷.۵ لایه Embedding	
۵۹	۱.۷.۵ تعریف مفهومی	
۶۰	۲.۷.۵ راه حل: نگاشت به فضای برداری	
۶۱	۳.۷.۵ مقایسه با One-Hot	
۶۱	۴.۷.۵ نقش embedding در BiLSTM	
۶۲	۸.۵ تحلیل دقیق معماری RobustBiLSTM	
۶۲	۱.۸.۵ هدف کلی مدل	
۶۲	۲.۸.۵ پارامترهای ورودی سازنده کلاس	
۶۳	۳.۸.۵ تحلیل اجزای داخلی مدل	
۶۴	۴.۸.۵ تحلیل دقیق متد forward	
۶۵	۵.۸.۵ تحلیل تابع pack_padded_sequence	
۶۶	۶.۸.۵ تحلیل خروجی LSTM: تفاوت packed_output و output h_n	



۶۷	چرا در این مسئله فقط از <code>h_n</code> استفاده شده است؟	۷.۸.۵
۶۸	چرا خروجی نهایی <code>concatenate</code> شده است؟	۸.۸.۵
۶۹	تحلیل دقیق <code>pad_packed_sequence</code>	۹.۸.۵
۶۹	تحلیل حالت‌های پنهان و سلولی	۱۰.۸.۵
۷۰	ساختار بلوک دیاگرامی مدل	۱۱.۸.۵
۷۱	تحلیل دیتاست: تولید، ساختار و توزیع نمونه‌ها	۹.۵
۷۱	نحوه تولید دیتاست	۱.۹.۵
۷۱	نمونه‌هایی از دیتاست	۲.۹.۵
۷۲	چرا این دیتاست برای <code>BiLSTM</code> مناسب است؟	۳.۹.۵
۷۲	تحلیل توزیع دیتاست	۴.۹.۵
۷۳	جمع‌بندی تحلیل دیتاست	۵.۹.۵
۷۴	آماده‌سازی داده و پایداری گرادیان‌ها در <code>LSTM</code>	۱۰.۵
۷۴	مدیریت داده‌های با طول متغیر: <code>collate_fn</code> و <code>SequenceDataset</code>	۱.۱۰.۵
۷۶	تحلیل عمیق مقداردهی اولیه متعامد در <code>LSTM</code>	۲.۱۰.۵
۷۷	مدیریت حافظه و جلوگیری از ناپایداری گرادیان‌ها	۱۱.۵
۷۷	تابع <code>detach_hidden</code> و مدیریت حافظه	۱.۱۱.۵
۷۹	برش گرادیان (<code>Gradient Clipping</code>)	۲.۱۱.۵
۸۰	تحلیل نتایج آموزش و اعتبارسنجی	۱۲.۵
۸۱	کد کامل	۱۳.۵



فهرست تصاویر

۳۵	 نمودارهای دقت و خطا در طول آموزش (رویکرد اول)	۱
۳۶	 نمودارهای دقت و خطا در طول آموزش (رویکرد دوم)	۲
۳۹	 نمودارهای دقت و خطا در طول آموزش (رویکرد سوم)	۳
۷۰	 ساختار کلی مدل RobustBiLSTM	۴
۷۳	 توزیع طول دنباله‌ها در دیتاست	۵
۷۴	 توزیع کلاس‌ها در کل دیتاست	۶
۷۵	 مقایسه توزیع کلاس‌ها بین مجموعه‌های آموزش و اعتبارسنجی	۷
۸۱	 نمودار خطای آموزش و اعتبارسنجی در طول دوره‌ها.	۸



۱ سوال ۱: تحلیل ریاضیاتی محوشدگی گرادیان و نقش Cell State در LSTM

۱.۱ الف) تحلیل محوشدگی گرادیان در RNN استاندارد

پدیده vanishing gradient یا محوشدگی گرادیان یکی از مشکلات اصلی آموزش شبکه‌های عصبی بازگشتی (RNN) است. این پدیده باعث می‌شود که گرادیان‌ها هنگام backpropagation through time (BPTT) در لایه‌های عمیق زمانی، به سرعت به صفر میل کنند و یادگیری وابستگی‌های بلندمدت در توالی‌های طولانی سخت شود.

ابتدا معادله حالت یک RNN استاندارد را به صورت زیر می‌نویسیم:

$$s_t = \tanh(Ux_t + Ws_{t-1}) \quad (۱)$$

که در آن:

• $s_t \in \mathbb{R}^n$ حالت مخفی در زمان t

• $x_t \in \mathbb{R}^m$ ورودی در زمان t

• $U \in \mathbb{R}^{n \times m}$ ماتریس وزن ورودی

• $W \in \mathbb{R}^{n \times n}$ ماتریس وزن بازگشتی (از حالت قبلی)

• $\tanh(\cdot)$ تابع فعال‌سازی تانژانت هیپربولیک که المان به المان اعمال می‌شود

هدف تحلیل: محاسبه مشتق جزئی حالت فعلی نسبت به حالت d گام زمانی قبل، یعنی $\frac{\partial s_t}{\partial s_{t-d}}$ ، و بررسی رفتار آن با افزایش d .

اعمال قاعده زنجیره‌ای (Chain Rule) با توجه به این که حالت در زمان t با حالت قبلی s_{t-1} مرتبط است، مشتق $\frac{\partial s_t}{\partial s_{t-d}}$ برابر است با ضرب زنجیره‌ای مشتقات میان گام‌ها:

$$\frac{\partial s_t}{\partial s_{t-d}} = \prod_{k=t-d+1}^t \frac{\partial s_k}{\partial s_{k-1}} = \frac{\partial s_t}{\partial s_{t-1}} \times \frac{\partial s_{t-1}}{\partial s_{t-2}} \times \dots \times \frac{\partial s_{t-d+1}}{\partial s_{t-d}} \quad (۲)$$

هر عامل $\frac{\partial s_k}{\partial s_{k-1}}$ در این ضرب، یک ماتریس Jacobian است که مشتق بردار s_k نسبت به بردار s_{k-1} را نشان می‌دهد.

محاسبه ماتریس ژاکوبین $\frac{\partial s_k}{\partial s_{k-1}}$ فرض کنید:

$$a_k = Ux_k + Ws_{k-1} \quad (۳)$$

و

$$s_k = \tanh(a_k) \quad (۴)$$

طبق قاعده زنجیره‌ای داریم:

$$\frac{\partial s_k}{\partial s_{k-1}} = \frac{\partial s_k}{\partial a_k} \cdot \frac{\partial a_k}{\partial s_{k-1}} \quad (۵)$$



تابع $\tanh(x)$ یک تابع المان به المان است که مشتق آن به شکل زیر است:

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) \quad (۶)$$

پس:

$$\frac{\partial s_k}{\partial a_k} = \text{diag}(1 - s_k^2) \quad (۷)$$

که $\text{diag}(\cdot)$ ماتریس قطری با ورودی‌های قطری است. مشتق $\frac{\partial a_k}{\partial s_{k-1}}$ برابر ماتریس وزن بازگشتی است:

$$\frac{\partial a_k}{\partial s_{k-1}} = W \quad (۸)$$

بنابراین:

$$\frac{\partial s_k}{\partial s_{k-1}} = \text{diag}(1 - s_k^2) W \quad (۹)$$

رابطه کلی مشتق نسبت به s_{t-d} طبق (۲) و (۹) داریم:

$$\frac{\partial s_t}{\partial s_{t-d}} = \prod_{k=t-d+1}^t \text{diag}(1 - s_k^2) W \quad (۱۰)$$

تحلیل رفتاری و محوشدگی گرادیان برای بررسی این که گرادیان با افزایش d چه رفتاری دارد، نرم ماتریسی آن را می‌سنجیم. با استفاده از خاصیت نرم‌های ماتریسی داریم:

$$\left\| \frac{\partial s_t}{\partial s_{t-d}} \right\| \leq \prod_{k=t-d+1}^t \left\| \text{diag}(1 - s_k^2) W \right\| \quad (۱۱)$$

اگر مقدار ثابت

$$y = \sup_k \left\| \text{diag}(1 - s_k^2) W \right\| \quad (۱۲)$$

باشد، آنگاه:

$$\left\| \frac{\partial s_t}{\partial s_{t-d}} \right\| \leq y^d \quad (۱۳)$$



بررسی نرم ماتریس قطری $\text{diag}(1 - s_k^2)$ مقادیر هر مولفه از s_k در بازه $(-1, 1)$ قرار دارد چون تابع $\tanh$ روی $\mathbb{R}$ تصویرش این محدوده است.

بنابراین:

$$0 < 1 - s_k^2 \leq 1 \quad (۱۴)$$

بنابراین:

$$\|\text{diag}(1 - s_k^2)\| \leq 1 \quad (۱۵)$$

بررسی نرم ماتریس وزن بازگشتی W نرم ماتریس W به مقدار ویژه بزرگترین مقدار قدر مطلق $(\lambda_{\max}(W))$ وابسته است. اگر مقادیر ویژه W کوچک باشند، یعنی:

$$\|W\| = \lambda_{\max}(W) < 1 \quad (۱۶)$$

که این حالت معمولاً هنگام مقداردهی وزن‌ها به صورت تصادفی کوچک رخ می‌دهد.

نتیجه نهایی اگر $y < 1$ باشد، رابطه زیر برقرار است:

$$\lim_{d \rightarrow \infty} y^d = 0 \quad (۱۷)$$

که به معنی آن است که $\frac{\partial s_t}{\partial s_{t-d}}$ به صورت نمایی به صفر میل می‌کند. این همان مسئله معروف vanishing gradient یا محوشدگی گرادیان در RNN است.

۲.۱ (ب) حل مشکل محوشدگی گرادیان در LSTM

شبکه‌های Long Short-Term Memory (LSTM) با معرفی ساختار حافظه سلولی (Cell State) و گیت‌های کنترلی، مشکل محوشدگی گرادیان در شبکه‌های عصبی بازگشتی استاندارد را به‌طور موثری حل کرده‌اند.

ساختار حافظه سلولی و معادله به‌روزرسانی حافظه سلولی در LSTM به صورت زیر به‌روزرسانی می‌شود:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (۱۸)$$

که در آن:

- C_t : بردار وضعیت حافظه در زمان t .
- $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \in (0, 1)^n$: گیت فراموشی (forget gate)، تصمیم می‌گیرد چه مقدار از حافظه قدیمی حفظ شود.



- گیت ورودی (input gate)، مقدار به روزرسانی حافظه. $i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \in (0, 1)^n$
- وضعیت کاننید جدید حافظه. $\tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$
- ضرب المان به المان. $\odot$

محاسبه مشتق $\frac{\partial C_t}{\partial C_{t-1}}$ انتقال گرادیان از C_t به C_{t-1} کلید فهم نحوه جلوگیری از محوشدگی گرادیان در LSTM است. مشتق کامل عبارت (۱۸) نسبت به C_{t-1} را با قاعده مشتق گیری موسوم به product rule و sum rule محاسبه می کنیم:

$$\frac{\partial C_t}{\partial C_{t-1}} = \frac{\partial}{\partial C_{t-1}} (f_t \odot C_{t-1} + i_t \odot \tilde{C}_t) \quad (۱۹)$$

$$= \underbrace{\frac{\partial f_t}{\partial C_{t-1}} \odot C_{t-1}}_I + \underbrace{f_t \odot \frac{\partial C_{t-1}}{\partial C_{t-1}}}_{II} + \underbrace{\frac{\partial i_t}{\partial C_{t-1}} \odot \tilde{C}_t}_{III} + \underbrace{i_t \odot \frac{\partial \tilde{C}_t}{\partial C_{t-1}}}_{IV} \quad (۲۰)$$

که در آن:

- عبارت $II = f_t \odot I$ زیرا $\frac{\partial C_{t-1}}{\partial C_{t-1}} = I$ ماتریس همانی است.
- عبارات I, III, IV معرف وابستگی غیر مستقیم گیت ها و کاننید $\tilde{C}_t, f_t, i_t$ به C_{t-1} از طریق حالت مخفی h_{t-1} هستند.

تقریب حذف مشتقات غیر مستقیم توجه داشته باشید که معمولاً، مقادیر مشتقات $\frac{\partial f_t}{\partial C_{t-1}}, \frac{\partial i_t}{\partial C_{t-1}}, \frac{\partial \tilde{C}_t}{\partial C_{t-1}}$ کوچک هستند یا در برخی تحلیل ها قابل چشم پوشی. به خصوص در مسیر انتقال اصلی حافظه (مسیر خطی)، این مشتقات به عنوان نویز تلقی می شوند. پس تقریب می زنیم:

$$\frac{\partial C_t}{\partial C_{t-1}} \approx f_t \quad (۲۱)$$

به عبارت ساده تر، نرخ تغییر حافظه جدید نسبت به حافظه قبلی توسط گیت فراموشی کنترل می شود.

انتقال گرادیان در مسیر حافظه سلولی براساس قاعده زنجیره ای در انتقال گرادیان در زمان، مشتق حافظه در زمان t نسبت به $t-d$ برابر است با ضرب زنجیره ای مشتقات میانی:

$$\frac{\partial C_t}{\partial C_{t-d}} = \prod_{k=t-d+1}^t \frac{\partial C_k}{\partial C_{k-1}} \approx \prod_{k=t-d+1}^t f_k \quad (۲۲)$$

بررسی رفتار طولانی مدت گرادیان مقایسه با RNN استاندارد:

- در RNN معمولی، این مسیر مشتق به شکل ضرب متوالی ماتریس وزن بازگشتی W است، که چون $\|W\| < 1$ ، مقدار گرادیان به سرعت $0 \rightarrow$ می رود و باعث محوشدگی می شود.

- در LSTM، مسیر خطی حافظه در قالب ضرب های المان به المان گیت فراموشی f_k وجود دارد که بین ۰ و ۱ است.

ایده کلیدی: حفظ گرادیان با تنظیم گیت فراموشی تابع $\sigma(\cdot)$ که سیگموید است، مقدار گیت فراموشی را در بازه $(0, 1)$ نگه می‌دارد. - در صورتی که اطلاعات گذشته اهمیت بالایی داشته باشد، شبکه گیت f_k را یاد می‌گیرد که مقدار آن نزدیک ۱ باشد:

$$f_k \approx 1 \Rightarrow \prod_{k=t-d+1}^t f_k \approx 1 \quad (23)$$

در نتیجه گرادیان به صورت پویا و بدون کاهش شدید حفظ می‌شود. - اگر اطلاعات قدیمی اهمیت نداشته باشند، مقدار f_k به عددی نزدیک صفر می‌رسد و حافظه قبلی فراموش می‌شود.

نکته ریاضی: علت کاهش سریع محوشدگی ساختار ضرب المان به المان بر خلاف ضرب ماتریسی، اجازه می‌دهد گرادیان مستقیماً از گام‌های زمانی دور بدون تبدیل پیچیده عبور کند. در واقع، ماتریس وزن W در RNN نقش فیلتر قوی روی گرادیان دارد (که اغلب منجر به کاهش شدت گرادیان می‌شود)، ولی در LSTM، مسیر خطی حافظه باعث می‌شود که کاهی‌ها کمتر شود.

تشریح مشتقات غیرمستقیم (جزئی‌تر) اگر دقیق‌تر بررسی کنیم:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad h_{t-1} = o_{t-1} \odot \tanh(C_{t-1}) \quad (24)$$

بنابراین:

$$\frac{\partial f_t}{\partial C_{t-1}} = \frac{\partial f_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial C_{t-1}} = \sigma'(a_{f_t}) \cdot U_f \cdot \text{diag}(o_{t-1} \odot (1 - \tanh^2(C_{t-1}))) \quad (25)$$

که این مقدار معمولاً کوچک است زیرا:
- $\sigma'(\cdot)$ مشتق سیگموید است که حداکثر ۰.۲۵ دارد.
- $\tanh^2(\cdot)$ که باعث کاهش شیب می‌شود.
- همچنین U_f وزن‌هایی است که معمولاً مقدار متوسط دارند.
برای همین تأثیر کل این مشتق نسبت به f_t اصلی کوچک است.

نتیجه‌گیری کلی - مسیر خطی و ضرب المان به المان حافظه سلولی همراه با گیت فراموشی، امکان انتقال گرادیان با دامنه مناسب در گام‌های زمانی طولانی را فراهم می‌کند.

- این ویژگی، LSTM را قادر می‌سازد وابستگی‌های بلندمدت را یاد بگیرد و همزمان بتواند اطلاعات نا مربوط را فراموش کند.
- این مزیت اصلی LSTM نسبت به RNN‌های معمولی است و توضیح می‌دهد چرا LSTM در مسائل کاربردی با توالی‌های طولانی عملکرد بهتری دارد.



۲.۲ -۲-۲ گرادیان خطا نسبت به حالت پنهان

همان طور که در صورت سوال بیان شده است، گرادیان تابع خطا نسبت به حالت پنهان در زمان t در اختیار ما قرار دارد. بنابراین داریم:

$$\delta h_t = \frac{\partial E}{\partial h_t} \quad (31)$$

این کمیت نقطه شروع پس انتشار برای این بخش از شبکه است.

۳.۲ -۳-۲ گرادیان خطا نسبت به خروجی پنهان کاندید

برای محاسبه گرادیان نسبت به $\tilde{h}_t$ ، از معادله خروجی پنهان نهایی استفاده می‌کنیم:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (32)$$

در این رابطه، تنها جمله‌ای که به $\tilde{h}_t$ وابسته است، جمله دوم یعنی $z_t \odot \tilde{h}_t$ است. بنابراین مشتق h_t نسبت به $\tilde{h}_t$ برابر خواهد بود با:

$$\frac{\partial h_t}{\partial \tilde{h}_t} = z_t \quad (33)$$

از آنجا که این وابستگی به صورت درایه به درایه است، گرادیان خطا نسبت به $\tilde{h}_t$ از ضرب درایه به درایه δh_t در z_t به دست می‌آید:

$$\delta \tilde{h}_t = \frac{\partial E}{\partial \tilde{h}_t} = \frac{\partial E}{\partial h_t} \odot \frac{\partial h_t}{\partial \tilde{h}_t} = \delta h_t \odot z_t \quad (34)$$

این رابطه نشان می‌دهد که سهم خطا که از طریق $\tilde{h}_t$ عبور می‌کند، به وسیله گیت Update یعنی z_t وزن دهی می‌شود.

۴.۲ -۴-۲ گرادیان خطا نسبت به گیت Reset

اکنون باید گرادیان را از $\tilde{h}_t$ به r_t منتقل کنیم. برای این منظور، معادله حالت پنهان کاندید را در نظر می‌گیریم:

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \quad (35)$$

برای ساده‌تر شدن محاسبات، ابتدا ورودی تابع $\tanh$ را با یک متغیر میانی نمایش می‌دهیم:

$$a_t = W \cdot [r_t \odot h_{t-1}, x_t] \quad (36)$$

در این صورت:

$$\tilde{h}_t = \tanh(a_t) \quad (37)$$

پس برای محاسبه گرادیان نسبت به r_t ، ابتدا گرادیان را نسبت به a_t به دست می‌آوریم و سپس از a_t به r_t منتقل می‌کنیم. بنابراین:

$$\frac{\partial \tilde{h}_t}{\partial r_t} = \frac{\partial \tilde{h}_t}{\partial a_t} \cdot \frac{\partial a_t}{\partial r_t} \quad (38)$$



۱.۴.۲ ۱-۴-۲- گرادیان خروجی پنهان کاندید نسبت به a_t

از آنجا که:

$$\tilde{h}_t = \tanh(a_t) \quad (39)$$

و می دانیم مشتق تابع تانژانت هیپربولیک برابر است با:

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) \quad (40)$$

در نتیجه خواهیم داشت:

$$\frac{\partial \tilde{h}_t}{\partial a_t} = 1 - \tilde{h}_t^2 \quad (41)$$

بنابراین گرادیان خطا نسبت به a_t برابر می شود با:

$$\delta a_t = \frac{\partial E}{\partial a_t} = \frac{\partial E}{\partial \tilde{h}_t} \odot \frac{\partial \tilde{h}_t}{\partial a_t} = \delta \tilde{h}_t \odot (1 - \tilde{h}_t^2) \quad (42)$$

حال با جایگذاری رابطه $\delta \tilde{h}_t$ از مرحله قبل، خواهیم داشت:

$$\delta a_t = \delta h_t \odot z_t \odot (1 - \tilde{h}_t^2) \quad (43)$$

این کمیت، گرادیان خطا نسبت به ورودی غیرخطی تولیدکننده خروجی کاندید است.

۲.۴.۲ ۲-۴-۲- گرادیان a_t نسبت به گیت **Reset**

اکنون باید تاثیر r_t بر a_t را بررسی کنیم. برای این منظور، ماتریس وزن W را به دو بخش متناظر با ورودی های الحاق شده تقسیم می کنیم:

$$W = [W_h \quad W_x] \quad (44)$$

در این صورت رابطه a_t را می توان به شکل زیر بازنویسی کرد:

$$a_t = W_h \cdot (r_t \odot h_{t-1}) + W_x \cdot x_t \quad (45)$$

برای ساده تر شدن مشتق گیری، عبارت زیر را تعریف می کنیم:

$$b_t = r_t \odot h_{t-1} \quad (46)$$

بنابراین:

$$a_t = W_h \cdot b_t + W_x \cdot x_t \quad (47)$$

از آنجا که جمله $W_x \cdot x_t$ به r_t وابسته نیست، در مشتق گیری نسبت به r_t حذف می شود. پس برای مشتق گیری از a_t نسبت به r_t داریم:

$$\frac{\partial a_t}{\partial r_t} = \frac{\partial a_t}{\partial b_t} \cdot \frac{\partial b_t}{\partial r_t} \quad (48)$$

حال هر کدام از این مشتق‌ها را جداگانه محاسبه می‌کنیم.
از رابطه

$$a_t = W_h \cdot b_t + W_x \cdot x_t \quad (49)$$

نتیجه می‌شود:

$$\frac{\partial a_t}{\partial b_t} = W_h \quad (50)$$

اما در پس‌انتشار، وقتی گرادیان از a_t به b_t منتقل می‌شود، باید ترانزاده ماتریس وزن ظاهر شود. بنابراین:

$$\delta b_t = W_h^T \cdot \delta a_t \quad (51)$$

از طرف دیگر، چون

$$b_t = r_t \odot h_{t-1} \quad (52)$$

و h_{t-1} نسبت به r_t ثابت در نظر گرفته می‌شود، مشتق b_t نسبت به r_t برابر است با:

$$\frac{\partial b_t}{\partial r_t} = h_{t-1} \quad (53)$$

بنابراین گرادیان خطا نسبت به r_t از رابطه زیر به دست می‌آید:

$$\delta r_t = \frac{\partial E}{\partial r_t} = \delta b_t \odot h_{t-1} \quad (54)$$

اکنون با جایگذاری δb_t خواهیم داشت:

$$\delta r_t = (W_h^T \cdot \delta a_t) \odot h_{t-1} \quad (55)$$

و با جایگذاری δa_t از مرحله قبل:

$$\delta r_t = \left(W_h^T \cdot \left(\delta h_t \odot z_t \odot (1 - \tilde{h}_t^2) \right) \right) \odot h_{t-1} \quad (56)$$

در اینجا باید توجه کرد که:

- عبارت $W_h^T \cdot \delta a_t$ یک ضرب ماتریسی است،
- سپس حاصل آن به صورت درایه‌به‌درایه در h_{t-1} ضرب می‌شود.

۵.۲-۵-۲ آخرین عبارت Chain Rule: گرادیان خطا نسبت به وزن‌های گیت Reset

اکنون به آخرین بخش زنجیره می‌رسیم. رابطه گیت Reset برابر است با:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (57)$$

برای ساده‌سازی، ورودی تابع سیگموئید را با یک متغیر میانی نمایش می‌دهیم:

$$c_t = W_r \cdot [h_{t-1}, x_t] \quad (58)$$



در نتیجه:

$$r_t = \sigma(c_t) \quad (59)$$

بنابراین برای رسیدن از r_t به W_r ، ابتدا گرادیان نسبت به c_t را محاسبه کرده و سپس آن را نسبت به W_r گسترش می‌دهیم:

$$\frac{\partial r_t}{\partial W_r} = \frac{\partial r_t}{\partial c_t} \cdot \frac{\partial c_t}{\partial W_r} \quad (60)$$

۱.۵.۲ ۲-۵-۱-۲ گرادیان گیت **Reset** نسبت به c_t (مشتق تابع سیگموید)

از آنجا که:

$$r_t = \sigma(c_t) \quad (61)$$

و مشتق تابع سیگموید برابر است با:

$$\frac{d}{dx} \sigma(x) = \sigma(x) (1 - \sigma(x)) \quad (62)$$

پس داریم:

$$\frac{\partial r_t}{\partial c_t} = r_t \odot (1 - r_t) \quad (63)$$

بنابراین گرادیان خطا نسبت به c_t از رابطه زیر به دست می‌آید:

$$\delta c_t = \frac{\partial E}{\partial c_t} = \delta r_t \odot \frac{\partial r_t}{\partial c_t} = \delta r_t \odot r_t \odot (1 - r_t) \quad (64)$$

اکنون اگر رابطه δr_t را جایگذاری کنیم، خواهیم داشت:

$$\delta c_t = \left(\left(W_h^T \cdot \left(\delta h_t \odot z_t \odot (1 - \tilde{h}_t^2) \right) \right) \odot h_{t-1} \right) \odot r_t \odot (1 - r_t) \quad (65)$$

۲.۵.۲ ۲-۵-۲ گام آخر: گرادیان c_t نسبت به W_r

از تعریف c_t داریم:

$$c_t = W_r \cdot [h_{t-1}, x_t] \quad (66)$$

اگر بردار ورودی الحاق شده را به صورت زیر تعریف کنیم:

$$u_t = [h_{t-1}, x_t] \quad (67)$$

آنگاه خواهیم داشت:

$$c_t = W_r \cdot u_t \quad (68)$$

در این حالت، گرادیان نسبت به ماتریس وزن از قاعده استاندارد مشتق‌گیری در یک لایه خطی به دست می‌آید و برابر است با حاصل ضرب خارجی (outer product) گرادیان ورودی لایه در بردار ورودی آن:

$$\frac{\partial E}{\partial W_r} = \delta c_t \cdot u_t^T \quad (۶۹)$$

با جایگذاری $u_t = [h_{t-1}, x_t]$ ، نتیجه نهایی این مرحله به صورت زیر خواهد بود:

$$\delta W_r = \frac{\partial E}{\partial W_r} = \delta c_t \cdot [h_{t-1}, x_t]^T \quad (۷۰)$$

این رابطه نشان می‌دهد که چون c_t از یک تبدیل خطی نسبت به W_r به دست آمده است، گرادیان نسبت به W_r باید به صورت ضرب ماتریسی نوشته شود، نه ضرب درایه‌به‌درایه.

۶.۲ -۶-۲ جمع‌بندی روابط و فرمول نهایی

اکنون تمامی گرادیان‌های میانی لازم برای زنجیره مشتق‌گیری به دست آمده‌اند. برای شفافیت بیشتر، ابتدا روابط میانی را یک‌بار دیگر به صورت خلاصه می‌نویسیم:

$$\tilde{h}_t = \delta h_t \odot z_t \quad (۷۱)$$

$$\delta a_t = \tilde{h}_t \odot (1 - \tilde{h}_t^2) = \delta h_t \odot z_t \odot (1 - \tilde{h}_t^2) \quad (۷۲)$$

$$\delta r_t = (W_h^T \cdot \delta a_t) \odot h_{t-1} \quad (۷۳)$$

$$\delta c_t = \delta r_t \odot r_t \odot (1 - r_t) \quad (۷۴)$$

$$\delta W_r = \delta c_t \cdot [h_{t-1}, x_t]^T \quad (۷۵)$$

در نهایت، با جایگذاری همه روابط بالا در یکدیگر، فرمول نهایی گرادیان نسبت به وزن‌های گیت Reset به دست می‌آید:

$$\delta W_r = \frac{\partial E}{\partial W_r} = \left(\left(W_h^T \cdot \left(\delta h_t \odot z_t \odot (1 - \tilde{h}_t^2) \right) \right) \odot h_{t-1} \odot r_t \odot (1 - r_t) \right) \cdot [h_{t-1}, x_t]^T \quad (۷۶)$$

بنابراین، رابطه نهایی نشان می‌دهد که گرادیان وزن‌های W_r از ترکیب چهار بخش اصلی تشکیل شده است:

۱. گرادیان خطا نسبت به خروجی پنهان یعنی δh_t

۲. اثر گیت Update از طریق z_t

۳. مشتق غیرخطی تابع $\tanh$ از طریق جمله $(1 - \tilde{h}_t^2)$,

۴. مشتق تابع سیگموئید گیت Reset از طریق جمله $r_t \odot (1 - r_t)$.

همچنین نقش h_{t-1} در دو بخش ظاهر می‌شود:

- یک‌بار در مسیر $r_t \odot h_{t-1}$ که نشان می‌دهد گیت Reset چگونه بر اطلاعات گذشته اثر می‌گذارد،

- و یک‌بار در بردار ورودی نهایی $[h_{t-1}, x_t]^T$ که در محاسبه گرادینان نسبت به W_r ظاهر می‌شود.

در نتیجه، این فرمول بیان دقیق پس‌انتشار خطا برای ماتریس وزن‌های گیت Reset در واحد GRU است.

۳ سوال ۳: پیاده سازی Coupled Input-Forget Gate LSTM از پایه با Numpy

۱.۳ Optimized Coupled Input-Forget Gate LSTM Implementation

در نسخه‌ای از شبکه LSTM، گیت‌های ورودی و فراموشی به یکدیگر متصل (Coupled) هستند. در LSTM استاندارد این دو گیت به صورت مستقل محاسبه می‌شوند اما در این معماری فرض می‌شود که ورود اطلاعات جدید و فراموشی اطلاعات قبلی به یکدیگر وابسته هستند. به همین دلیل رابطه زیر برقرار است:

$$f_t = 1 - i_t$$

در نتیجه معادله به‌روزرسانی حافظه نیز به صورت زیر نوشته می‌شود:

$$C_t = (1 - i_t) \odot C_{t-1} + i_t \odot \tilde{C}_t$$

که در آن $\odot$ نشان‌دهنده ضرب درایه‌به‌درایه (Hadamard Product) است. با حذف گیت فراموشی مستقل، پارامترهای مربوط به آن نیز حذف می‌شوند و در نتیجه تعداد پارامترهای مدل کاهش می‌یابد. این موضوع می‌تواند باعث ساده‌تر شدن مدل، افزایش سرعت آموزش و کاهش احتمال بیش‌برازش شود.

۱.۱.۳ معادلات مدل

در این معماری، محاسبات در هر گام زمانی t به صورت زیر انجام می‌شود.

گیت ورودی این گیت مشخص می‌کند چه مقدار از اطلاعات جدید وارد حافظه شود.

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

حافظه کاندید در این مرحله، برداری از اطلاعات جدید که قابلیت اضافه شدن به حافظه را دارند تولید می‌شود.

$$\tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

به روزرسانی وضعیت سلول حافظه قبلی با اطلاعات جدید ترکیب شده و حافظه جدید ساخته می شود.

$$C_t = (1 - i_t) \odot C_{t-1} + i_t \odot \tilde{C}_t$$

گیت خروجی این گیت تعیین می کند چه مقدار از حافظه فعلی به عنوان خروجی شبکه استفاده شود.

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

حالت پنهان در نهایت حالت پنهان شبکه که خروجی این گام زمانی است محاسبه می شود.

$$h_t = o_t \odot \tanh(C_t)$$

در روابط فوق، x_t ورودی در گام زمانی t ، h_{t-1} حالت پنهان مرحله قبل و C_{t-1} وضعیت حافظه قبلی شبکه هستند. ماتریس های W و U وزن های قابل یادگیری و b بردار بایاس هستند. تابع σ (تابع سیگموئید) با خروجی بین 0 و 1 و $\tanh$ (تابع تانژانت هیپربولیک) با خروجی بین -1 و 1 است.

در این بخش دو رویکرد برای پیاده سازی کلاس پایتونی اتخاذ شده است. در رویکرد نخست، تمامی محاسبات ریاضی به صورت گام به گام و مجزا پیاده سازی شدند که در آن، سادگی و وضوح روند محاسباتی بر بهینه سازی عملکرد اولویت داشت. در رویکرد دوم، با استفاده از تکنیک الحاق ماتریس ها (Concatenation)، سعی بر کاهش بار محاسباتی مدل شد؛ به طوری که به جای اجرای چندین عمل ضرب ماتریسی در خطوط مجزا، محاسبات مربوط به گیت های مختلف در قالب یک ضرب ماتریسی واحد ادغام گردید تا بازدهی محاسباتی افزایش یابد.

در فایل کد، بخش مربوط به پیاده سازی با محاسبات ماتریسی مجزا با هدر Coupled Input-Forget Gate LSTM Implementation مشخص شده است. در این قسمت، عملیات مربوط به هر گیت به صورت جداگانه و در چند مرحله انجام می شود. همچنین بخش دارای هدر Optimized Coupled Input-Forget Gate LSTM Implementation به پیاده سازی بهینه کلاس اختصاص دارد که در آن با ترکیب محاسبات و استفاده از ساختارهای ماتریسی کارآمدتر، تلاش شده است تا هزینه محاسباتی مدل کاهش یابد. در زیر کدهای زده شده به صورت کامل آورده شده است:

۲.۱.۳ کد

```

1  #!/usr/bin/env python
2  # coding: utf-8
3
4  # # Coupled Input-Forget Gate LSTM Implementation
5
6  # In[1]:
7
8
9  import numpy as np
10
    
```



```
11 class CIFG_LSTM:
12     def __init__(self, input_size, hidden_size):
13         """
14         Initialize network parameters
15         input_size: dimension of the input vector at each time step
16         hidden_size: dimension of the hidden state (memory)
17         """
18         self.input_size = input_size
19         self.hidden_size = hidden_size
20
21         # Initialize weights with a normal distribution and biases with zeros
22         # W: weights connected to input X
23         # U: weights connected to previous hidden state h
24         # b: biases
25
26         # 1. Input Gate parameters
27         self.Wi = np.random.randn(hidden_size, input_size) * 1.0
28         self.Ui = np.random.randn(hidden_size, hidden_size) * 1.0
29         self.bi = np.zeros((hidden_size, 1))
30
31         # 2. Candidate Cell State parameters
32         self.Wc = np.random.randn(hidden_size, input_size) * 1.0
33         self.Uc = np.random.randn(hidden_size, hidden_size) * 1.0
34         self.bc = np.zeros((hidden_size, 1))
35
36         # 3. Output Gate parameters
37         self.Wo = np.random.randn(hidden_size, input_size) * 1.0
38         self.Uo = np.random.randn(hidden_size, hidden_size) * 1.0
39         self.bo = np.zeros((hidden_size, 1))
40
41     def _sigmoid(self, x):
42         """Sigmoid activation function producing output between 0 and 1"""
43         # Prevent numerical overflow
44         x = np.clip(x, -500, 500)
45         return 0.1 / (0.1 + np.exp(-x))
46
47     def _tanh(self, x):
48         """Hyperbolic tangent activation function producing output between -1 and 1"""
49         return np.tanh(x)
50
```



```
51 def forward_step(self, x_t, h_prev, c_prev):
52     """
53     Perform the forward pass for a single time step
54     x_t: input at time t with shape (input_size, 1)
55     h_prev: previous hidden state with shape (hidden_size, 1)
56     c_prev: previous cell state with shape (hidden_size, 1)
57     """
58     # 1. Compute input gate (i_t)
59     i_t = self._sigmoid(np.dot(self.Wi, x_t) + np.dot(self.Ui, h_prev) + self.bi)
60
61     # Forget gate is implicitly defined as (1 - i_t)
62     f_t = 0.1 - i_t
63
64     # 2. Compute candidate cell state (c_tilde_t)
65     c_tilde_t = self._tanh(np.dot(self.Wc, x_t) + np.dot(self.Uc, h_prev) + self.bc)
66
67     # 3. Update the cell state (C_t) using the given formula
68     #  $C_t = (1 - i_t) C_{t-1} + i_t \tilde{C}_t$ 
69     c_t = f_t * c_prev + i_t * c_tilde_t # In NumPy, * means element-wise multiplication
70
71     # 4. Compute output gate (o_t)
72     o_t = self._sigmoid(np.dot(self.Wo, x_t) + np.dot(self.Uo, h_prev) + self.bo)
73
74     # 5. Compute the new hidden state (h_t)
75     h_t = o_t * self._tanh(c_t)
76
77     return h_t, c_t
78
79 def forward(self, X):
80     """
81     Perform the forward pass for a full input sequence
82     X: input sequence with shape (sequence_length, input_size)
83     """
84     seq_length = X.shape[0]
85
86     # Initialize memory states with zeros
87     h_t = np.zeros((self.hidden_size, 1))
88     c_t = np.zeros((self.hidden_size, 1))
89
90     # List to store all hidden states over time
```

```
91     hidden_states = []
92
93     # Loop through time steps
94     for t in range(seq_length):
95         # Reshape the t-th input into a column vector (input_size, 1)
96         x_t = X[t].reshape(-1, 1)
97
98         # Run one LSTM step
99         h_t, c_t = self.forward_step(x_t, h_t, c_t)
100
101         # Store output
102         hidden_states.append(h_t.flatten())
103
104     return np.array(hidden_states)
105
106
107 # In[3]:
108
109
110 import time
111 # =====
112 # Testing the code on a Synthetic Time Series
113 # =====
114 if __name__ == "__main__":
115     # Dimension settings based on problem requirements
116     INPUT_SIZE = 10    # Input vector dimension at each time step
117     HIDDEN_SIZE = 16   # Hidden state (memory) dimension
118     SEQ_LENGTH = 50    # Synthetic time series length (50 time steps)
119
120     # 1. Generate synthetic time series randomly
121     np.random.seed(42) # For reproducible results
122     X_synthetic = np.random.randn(SEQ_LENGTH, INPUT_SIZE)
123
124     # 2. Create model instance
125     # (Random weight initialization is done automatically in __init__)
126     model = CIFG_LSTM(input_size=INPUT_SIZE, hidden_size=HIDDEN_SIZE)
127
128     # 3. Run the model and get outputs
129     all_hidden_states = model.forward(X_synthetic)
130
```



```
131 # --- Verify matrix dimensions ---
132 print("=== Matrix Dimensions Verification ===")
133 print(f"Input time series shape: {X_synthetic.shape} -> (Seq_len=50, Input_size=10)")
134 print(f"Total output matrix shape: {all_hidden_states.shape} -> (Seq_len=50,
    ↳ Hidden_size=16)")
135 assert all_hidden_states.shape == (SEQ_LENGTH, HIDDEN_SIZE), "Output dimension mismatch
    ↳ error!"
136 print("Matrix dimensions verified successfully.\n")
137
138 # --- Extract h_t at each time step ---
139 print("=== Extracting h_t at each time step ===")
140
141 # Print the initial state (h_0)
142 # Assuming initial state is zeros as per standard LSTM initialization
143 h_0 = np.zeros(HIDDEN_SIZE)
144 print(f"Time Step 00 (Initial h_0) | Shape: {h_0.shape}")
145 print(f"Values: {np.round(h_0, 3)}\n")
146
147 for t in range(SEQ_LENGTH):
148     h_t = all_hidden_states[t] # Extract hidden state vector at time t
149
150     # To avoid excessively long console output, print only the first few and the last step
151     if t < 3:
152         print(f"Time Step {t+1:02d} | Shape: {h_t.shape}")
153         print(f"Values: {np.round(h_t, 3)}\n")
154     elif t == 3:
155         print("...\n[Outputs for steps 4 to 49 calculated and extracted, but hidden for
            ↳ brevity]\n...")
156     elif t == SEQ_LENGTH - 1:
157         print(f"Time Step {t+1:02d} (Last Step) | Shape: {h_t.shape}")
158         print(f"Values: {np.round(h_t, 3)}\n")
159
160     # =====
161     # Performance Benchmarking
162     # =====
163
164     print("=== Performance Benchmarking ===")
165     num_runs = 1000 #
166
167     # (Warm-up)
168     for _ in range(10):
169         _ = model.forward(X_synthetic)
```



```
۱۶۸
۱۶۹ #
۱۷۰ start_time = time.perf_counter()
۱۷۱
۱۷۲ for _ in range(num_runs):
۱۷۳     _ = model.forward(X_synthetic)
۱۷۴
۱۷۵ end_time = time.perf_counter()
۱۷۶
۱۷۷ total_time = end_time - start_time
۱۷۸ avg_time_per_run = (total_time / num_runs) * 1000 #
۱۷۹
۱۸۰ print(f"Total time for {num_runs} forward passes: {total_time:4f.} seconds")
۱۸۱ print(f"Average time per forward pass: {avg_time_per_run:4f.} milliseconds")
۱۸۲ print("=====\n")
۱۸۳
۱۸۴
۱۸۵ # # Optimized Coupled Input-Forget Gate LSTM Implementation
۱۸۶
۱۸۷ # In[4]:
۱۸۸
۱۸۹
۱۹۰ import numpy as np
۱۹۱
۱۹۲ class CIFG_LSTM:
۱۹۳     def __init__(self, input_size, hidden_size):
۱۹۴         """
۱۹۵         Initialize parameters (same interface as the original non-optimized version)
۱۹۶         input_size: dimension of input vector
۱۹۷         hidden_size: dimension of hidden state
۱۹۸         """
۱۹۹         self.input_size = input_size
۲۰۰         self.hidden_size = hidden_size
۲۰۱
۲۰۲         # Original-style initialization (same distributions as the first code)
۲۰۳         Wi = np.random.randn(hidden_size, input_size) * 1.0
۲۰۴         Ui = np.random.randn(hidden_size, hidden_size) * 1.0
۲۰۵         bi = np.zeros((hidden_size, 1))
۲۰۶
۲۰۷         Wc = np.random.randn(hidden_size, input_size) * 1.0
```



```

۲۰۸     Uc = np.random.randn(hidden_size, hidden_size) * 1.0
۲۰۹     bc = np.zeros((hidden_size, 1))
۲۱۰
۲۱۱     Wo = np.random.randn(hidden_size, input_size) * 1.0
۲۱۲     Uo = np.random.randn(hidden_size, hidden_size) * 1.0
۲۱۳     bo = np.zeros((hidden_size, 1))
۲۱۴
۲۱۵     # ----- Optimized combined matrices -----
۲۱۶     # Shape: (input_size + hidden_size , 3 * hidden_size)
۲۱۷     self.W = np.zeros((input_size + hidden_size, 3 * hidden_size))
۲۱۸
۲۱۹     # Input part
۲۲۰     self.W[:input_size, 0:hidden_size] = Wi.T
۲۲۱     self.W[:input_size, hidden_size:2*hidden_size] = Wc.T
۲۲۲     self.W[:input_size, 2*hidden_size:3*hidden_size] = Wo.T
۲۲۳
۲۲۴     # Hidden part
۲۲۵     self.W[input_size:, 0:hidden_size] = Ui.T
۲۲۶     self.W[input_size:, hidden_size:2*hidden_size] = Uc.T
۲۲۷     self.W[input_size:, 2*hidden_size:3*hidden_size] = Uo.T
۲۲۸
۲۲۹     # Bias
۲۳۰     self.b = np.concatenate((bi.T, bc.T, bo.T), axis=1)
۲۳۱
۲۳۲     def _sigmoid(self, x):
۲۳۳         x = np.clip(x, -500, 500)
۲۳۴         return 0.1 / (0.1 + np.exp(-x))
۲۳۵
۲۳۶     def forward_step(self, x_t, h_prev, c_prev):
۲۳۷         """
۲۳۸         Forward computation for a single time step
۲۳۹         Shapes are identical to the original implementation
۲۴۰         """
۲۴۱         x_t = x_t.reshape(1, -1)
۲۴۲         h_prev = h_prev.reshape(1, -1)
۲۴۳         c_prev = c_prev.reshape(1, -1)
۲۴۴
۲۴۵         combined = np.concatenate((x_t, h_prev), axis=1)
۲۴۶
۲۴۷         gates = np.dot(combined, self.W) + self.b

```




```
۲۴۸
۲۴۹     i_raw = gates[:, 0:self.hidden_size]
۲۵۰     c_raw = gates[:, self.hidden_size:2*self.hidden_size]
۲۵۱     o_raw = gates[:, 2*self.hidden_size:3*self.hidden_size]
۲۵۲
۲۵۳     i_t = self._sigmoid(i_raw)
۲۵۴     c_tilde = np.tanh(c_raw)
۲۵۵     o_t = self._sigmoid(o_raw)
۲۵۶
۲۵۷     c_t = (0.1 - i_t) * c_prev + i_t * c_tilde
۲۵۸     h_t = o_t * np.tanh(c_t)
۲۵۹
۲۶۰     return h_t.T, c_t.T
۲۶۱
۲۶۲ def forward(self, X):
۲۶۳     """
۲۶۴     Forward pass for a full sequence
۲۶۵     Same input/output format as the original implementation
۲۶۶     """
۲۶۷     seq_length = X.shape[0]
۲۶۸
۲۶۹     h_t = np.zeros((self.hidden_size, 1))
۲۷۰     c_t = np.zeros((self.hidden_size, 1))
۲۷۱
۲۷۲
۲۷۳     hidden_states = []
۲۷۴
۲۷۵     for t in range(seq_length):
۲۷۶         x_t = X[t].reshape(-1, 1)
۲۷۷         h_t, c_t = self.forward_step(x_t, h_t, c_t)
۲۷۸         hidden_states.append(h_t.flatten())
۲۷۹
۲۸۰     return np.array(hidden_states)
۲۸۱
۲۸۲
۲۸۳ # In[5]:
۲۸۴
۲۸۵
۲۸۶ import time
۲۸۷ # =====
```



```
۲۸۸ # Testing the Optimized CIFG-LSTM Model
۲۸۹ # =====
۲۹۰ if __name__ == "__main__":
۲۹۱     INPUT_SIZE = 10
۲۹۲     HIDDEN_SIZE = 16
۲۹۳     SEQ_LENGTH = 50
۲۹۴
۲۹۵     np.random.seed(42)
۲۹۶     X_synthetic = np.random.randn(SEQ_LENGTH, INPUT_SIZE)
۲۹۷
۲۹۸     model = CIFG_LSTM(input_size=INPUT_SIZE, hidden_size=HIDDEN_SIZE)
۲۹۹
۳۰۰     all_hidden_states = model.forward(X_synthetic)
۳۰۱
۳۰۲     print("=== Matrix Dimensions Verification ===")
۳۰۳     print(f"Input Time Series Shape: {X_synthetic.shape} -> (Seq_len=50, Input_size=10)")
۳۰۴     print(f"Optimized Weight Matrix Shape: {model.W.shape} -> (Input+Hidden, 3*Hidden)")
۳۰۵     print(f"Output Hidden States Shape: {all_hidden_states.shape} -> (Seq_len=50,
        ↪ Hidden_size=16)")
۳۰۶     assert all_hidden_states.shape == (SEQ_LENGTH, HIDDEN_SIZE), "Output dimension mismatch!"
۳۰۷     print("Dimensions verified successfully.\n")
۳۰۸
۳۰۹     print("=== Extracting h_t at each time step ===")
۳۱۰     h_0 = np.zeros(HIDDEN_SIZE)
۳۱۱     print(f"Time Step 00 (Initial h_0) | Shape: {h_0.shape}")
۳۱۲     print(f"Values: {np.round(h_0, 3)}\n")
۳۱۳
۳۱۴     for t in range(SEQ_LENGTH):
۳۱۵         h_t = all_hidden_states[t]
۳۱۶
۳۱۷         if t < 3:
۳۱۸             print(f"Time Step {t+1:02d} | Shape: {h_t.shape}")
۳۱۹             print(f"Values: {np.round(h_t, 3)}\n")
۳۲۰         elif t == 3:
۳۲۱             print("...\n[Steps 4 to 49 calculated and extracted, hidden for brevity]\n...")
۳۲۲         elif t == SEQ_LENGTH - 1:
۳۲۳             print(f"Time Step {t+1:02d} (Last Step) | Shape: {h_t.shape}")
۳۲۴             print(f"Values: {np.round(h_t, 3)}\n")
۳۲۵     # =====
۳۲۶     # Performance Benchmarking
```

```

۳۲۷ # =====
۳۲۸ print("=== Performance Benchmarking ===")
۳۲۹ num_runs = 1000 #
۳۳۰
۳۳۱ # (warm-up)
۳۳۲ for _ in range(10):
۳۳۳     _ = model.forward(X_synthetic)
۳۳۴
۳۳۵ #
۳۳۶ start_time = time.perf_counter()
۳۳۷
۳۳۸ for _ in range(num_runs):
۳۳۹     _ = model.forward(X_synthetic)
۳۴۰
۳۴۱ end_time = time.perf_counter()
۳۴۲
۳۴۳ total_time = end_time - start_time
۳۴۴ avg_time_per_run = (total_time / num_runs) * 1000 #
۳۴۵
۳۴۶ print(f"Total time for {num_runs} forward passes: {total_time:4f.} seconds")
۳۴۷ print(f"Average time per forward pass: {avg_time_per_run:4f.} milliseconds")
۳۴۸ print("=====\n")
۳۴۹
    
```

۳.۱.۳ پیاده‌سازی مبتنی بر محاسبات ماتریسی مجزا

ایده اصلی پیاده‌سازی در رویکرد اول، پیاده‌سازی مدل CIFG-LSTM به صورت مستقیم و مطابق با معادلات ریاضی انجام شد. در این روش، برای هر گیت شبکه (گیت ورودی، حافظه کاندید و گیت خروجی) محاسبات ماتریسی به صورت مستقل انجام می‌شود. به عبارت دیگر، برای هر گیت دو ضرب ماتریسی جداگانه در نظر گرفته می‌شود: یکی برای بردار ورودی x_t و دیگری برای حالت پنهان قبلی h_{t-1} ، سپس بردار بایاس به آن اضافه می‌شود.

پس از محاسبه گیت ورودی i_t ، گیت فراموشی به صورت ضمنی و با استفاده از رابطه زیر به دست می‌آید:

$$f_t = 1 - i_t$$

سپس حافظه کاندید $\tilde{C}_t$ تولید شده و وضعیت حافظه شبکه مطابق رابطه زیر به‌روزرسانی می‌شود:

$$C_t = (1 - i_t) \odot C_{t-1} + i_t \odot \tilde{C}_t$$

در ادامه گیت خروجی o_t محاسبه شده و حالت پنهان جدید h_t به شکل زیر به دست می‌آید:

$$h_t = o_t \odot \tanh(C_t)$$

این فرآیند برای تمامی گام‌های زمانی دنباله ورودی تکرار شده و در هر مرحله در بردار حالت پنهان ذخیره می‌شود.

ساختار داده و بررسی ابعاد ماتریس‌ها برای ارزیابی عملکرد مدل، یک سری زمانی مصنوعی با طول ۵۰ گام زمانی و بعد ورودی ۱۰ تولید شد. اندازه حالت پنهان شبکه نیز برابر با ۱۶ در نظر گرفته شد. در نتیجه انتظار می‌رود خروجی مدل ماتریسی با ابعاد زیر باشد:

(50, 16)

یعنی برای هر یک از ۵۰ گام زمانی، یک بردار حالت پنهان با بعد ۱۶ تولید می‌شود. خروجی برنامه نیز این موضوع را به طور کامل تأیید می‌کند:

```
=== Matrix Dimensions Verification ===
Input time series shape: (50, 10) -> (Seq_len=50, Input_size=10)
Total output matrix shape: (50, 16) -> (Seq_len=50, Hidden_size=16)
Matrix dimensions verified successfully.
```

این نتیجه نشان می‌دهد که ابعاد ماتریس‌های ورودی و خروجی کاملاً با طراحی مدل مطابقت دارند و هیچ ناسازگاری در ابعاد داده‌ها وجود ندارد.

تحلیل خروجی‌های تولید شده پس از اجرای مدل، بردار حالت پنهان h_t در هر گام زمانی استخراج شد. مطابق انتظار، حالت اولیه شبکه به صورت بردار صفر مقداردهی شده است:

```
Time Step 00 (Initial h_0) | Shape: (16,)
Values: [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

در گام‌های بعدی، شبکه با دریافت ورودی‌های جدید به تدریج وضعیت حافظه و حالت پنهان خود را به‌روزرسانی می‌کند. برای مثال در گام زمانی اول خروجی به صورت زیر است:

```
Time Step 01 | Shape: (16,)
Values: [-0.031 -0.074 -0.025 -0.012 0.069 -0.064 -0.053 0.009 0.008 0.037 0.037 0.039 0.081 0.092 0.002 -0.059]
```

و در گام زمانی بعد:

```
Time Step 02 | Shape: (16,)
Values: [-0.053 -0.008 0.071 0.024 -0.122 0.055 -0.007 0.031 -0.081 -0.094 -0.024 0.048 -0.046 0.042 0.034 -0.065]
```

این مقادیر نشان می‌دهند که خروجی شبکه در بازه‌ای نزدیک به $[-1, 1]$ قرار دارد که ناشی از استفاده از تابع فعال‌سازی $\tanh$ در معماری LSTM است. در طول پردازش دنباله زمانی، شبکه اطلاعات را در حافظه داخلی خود ذخیره کرده و به تدریج وضعیت پنهان را تغییر می‌دهد.



در آخرین گام زمانی نیز خروجی به شکل زیر گزارش شده است:

Time Step 50 (Last Step) | Shape: (16,)

Values: [0.126 0.04 -0.021 -0.006 -0.064 0.064 0.023 0.061 0.058 0.003 0. -0.073 -0.013 -0.101 0.09 0.034]

تمامی بردارهای حالت پنهان دارای بعد ۱۶ هستند که نشان‌دهنده تطابق کامل با اندازه حالت پنهان تعریف شده در مدل است.

۴.۱.۳ پیاده‌سازی بهینه با ترکیب ماتریس‌ها

ایده بهینه‌سازی در رویکرد دوم، هدف کاهش هزینه محاسباتی مدل بود. در پیاده‌سازی اولیه، برای هر گیت شبکه چندین ضرب ماتریسی مجزا انجام می‌شد. با توجه به اینکه این عملیات در هر گام زمانی تکرار می‌شود، تعداد ضرب‌های ماتریسی افزایش پیدا کرده و بار محاسباتی مدل بالا می‌رود.

برای رفع این مسئله، بردار ورودی x_t و حالت پنهان قبلی h_{t-1} با یکدیگر الحاق (Concatenate) شدند و یک ماتریس وزن بزرگ ساخته شد که شامل وزن‌های مربوط به هر سه گیت شبکه است. در نتیجه، به جای انجام چند ضرب ماتریسی جداگانه، تنها یک ضرب ماتریسی انجام می‌شود و سپس خروجی حاصل به سه بخش تقسیم می‌شود تا مقادیر مربوط به گیت‌های مختلف استخراج شوند.

ساختار ماتریس وزن ترکیبی در این پیاده‌سازی یک ماتریس وزن بزرگ با ابعاد زیر ساخته شد:

$$(Input_size + Hidden_size) \times (3 \times Hidden_size)$$

با توجه به مقادیر مسئله داریم:

$$(10 + 16) \times (3 \times 16) = 26 \times 48$$

این ماتریس شامل وزن‌های مربوط به سه بخش اصلی شبکه یعنی گیت ورودی، حافظه کاندید و گیت خروجی است. به این ترتیب تمام محاسبات گیت‌ها در قالب یک عملیات ضرب ماتریسی واحد انجام می‌شود.

بررسی ابعاد و خروجی مدل پس از اجرای نسخه بهینه مدل، بررسی ابعاد ماتریس‌ها نشان داد که تمامی ابعاد با طراحی شبکه مطابقت دارند:

=== Matrix Dimensions Verification ===

Input Time Series Shape: (50, 10) -> (Seq_len=50, Input_size=10)

Optimized Weight Matrix Shape: (26, 48) -> (Input+Hidden, 3*Hidden)

Output Hidden States Shape: (50, 16) -> (Seq_len=50, Hidden_size=16)

Dimensions verified successfully.

همچنین خروجی بردارهای حالت پنهان در هر گام زمانی دقیقاً با خروجی نسخه غیر بهینه یکسان بود. برای مثال:

Time Step 01 | Shape: (16,)

Values: [-0.031 -0.074 -0.025 -0.012 0.069 -0.064 -0.053 0.009 0.008 0.037 0.037 0.039 0.081 0.092 0.002 -0.059]



و در گام نهایی:

Time Step 50 (Last Step) | Shape: (16,)

Values: [0.126 0.04 -0.021 -0.006 -0.064 0.064 0.023 0.061 0.058 0.003 0. -0.073 -0.013 -0.101 0.09 0.034]

این تطابق کامل نشان می‌دهد که نسخه بهینه دقیقاً همان محاسبات مدل اصلی را انجام می‌دهد اما ساختار محاسباتی آن کارآمدتر است.

تحلیل کاهش بار محاسباتی در پیاده‌سازی اولیه، برای هر گیت شبکه چندین ضرب ماتریسی مستقل انجام می‌شد. در مقابل، در پیاده‌سازی بهینه تمامی این عملیات در قالب یک ضرب ماتریسی واحد انجام می‌شود و سپس خروجی آن به بخش‌های مختلف تقسیم می‌شود.

این تغییر باعث می‌شود:

- تعداد عملیات ماتریسی کاهش یابد،
- استفاده از حافظه و کش پردازنده بهینه‌تر شود،
- سرعت اجرای مدل افزایش پیدا کند.

بنابراین پیاده‌سازی دوم از نظر محاسباتی کارآمدتر است، در حالی که از نظر خروجی دقیقاً معادل پیاده‌سازی اولیه عمل می‌کند. این موضوع نشان می‌دهد که با طراحی مناسب ساختار ماتریس‌ها می‌توان بدون تغییر در رفتار مدل، کارایی محاسباتی آن را به شکل قابل توجهی بهبود داد.

۵.۱.۳ نتایج تحلیل کاهش زمان اجرا

به منظور ارزیابی میزان کاهش بار محاسباتی، زمان اجرای ۱۰۰۰ بار forward pass برای هر دو پیاده‌سازی اندازه‌گیری شد. نتایج مربوط به پیاده‌سازی غیر بهینه (بدون ماتریس الحاقی) به صورت زیر است:

```
=== Performance Benchmarking ===
Total time for 1000 forward passes: 6622.3 seconds
Average time per forward pass: 6622.3 milliseconds
=====
```

نتایج مربوط به پیاده‌سازی بهینه (با استفاده از ماتریس وزن الحاقی) به صورت زیر به دست آمد:

```
=== Performance Benchmarking ===
Total time for 1000 forward passes: 4251.3 seconds
Average time per forward pass: 4251.3 milliseconds
=====
```



با مقایسه دو مقدار زمان کل اجرا داریم:

$$\Delta T = 3.6622 - 3.4251 = 0.2371 \text{ seconds}$$

که تقریباً برابر با:

$$0.2371 \approx 0.2 \text{ seconds} = 200 \text{ milliseconds}$$

کاهش زمان برای هر ۱۰۰۰ بار اجرای forward pass است.

همچنین در مقیاس هر اجرای منفرد داریم:

$$3.6622 - 3.4251 = 0.2371 \text{ milliseconds}$$

کاهش زمان در هر بار اجرا مشاهده می‌شود.

این نتایج نشان می‌دهد که استفاده از ماتریس وزن الحاقی و انجام یک ضرب ماتریسی واحد به جای چند ضرب ماتریسی مجزا، منجر به کاهش حدود ۲۰۰ میلی‌ثانیه در اجرای ۱۰۰۰ بار محاسبه شده است. این موضوع بیانگر کاهش بار محاسباتی روی CPU و بهبود کارایی پردازشی مدل است، بدون آنکه تغییری در خروجی یا رفتار شبکه ایجاد شود.



۴ سوال ۴: پیاده سازی برداری پردازش دسته ای (Batched Processing) و پس انتشار در زمان (BPTT) با ترفندهای بهینه سازی

۱.۴ پیاده سازی برداری شبکه Vanilla RNN با قابلیت پردازش دسته ای (Batched Processing)

۱.۱.۴ مروری بر سازوکار شبکه های RNN و مدل Vanilla RNN

شبکه های عصبی بازگشتی (Recurrent Neural Networks - RNN) برای مدل سازی داده های ترتیبی طراحی شده اند. در این شبکه ها برخلاف شبکه های پیشخور، وابستگی زمانی بین نمونه ها حفظ می شود. این کار از طریق یک متغیر حالت پنهان (Hidden State) انجام می شود که در هر گام زمانی به روزرسانی شده و به گام بعد منتقل می گردد. در ساده ترین نوع این شبکه ها یعنی Vanilla RNN، در هر گام زمانی t ، بردار ورودی x_t و حالت پنهان قبلی h_{t-1} ترکیب شده و حالت پنهان جدید تولید می شود:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$$

سپس خروجی شبکه به صورت زیر محاسبه می شود:

$$y_t = W_y h_t + b_y$$

در این ساختار، تابع $\tanh$ نقش غیرخطی سازی را بر عهده دارد و باعث می شود شبکه قادر به مدل سازی روابط غیرخطی در داده های ترتیبی باشد.

۲.۱.۴ طراحی کلاس شبکه با پشتیبانی از پردازش دسته ای

در این پیاده سازی، یک کلاس با نام BatchedVanillaRNN طراحی شده است که برخلاف حالت ساده، قابلیت پردازش دسته ای (Mini-Batch Processing) را دارد.

ورودی شبکه به صورت یک تانسور سه بعدی با ابعاد زیر در نظر گرفته شده است:

(Size Batch, Length Sequence, Features)

که در آن:

- Batch Size: تعداد نمونه های پردازش شونده به صورت همزمان
- Sequence Length: طول دنباله زمانی
- Features: تعداد ویژگی های هر گام زمانی

در متد forward ابتدا حالت پنهان اولیه برای تمام اعضای دسته برابر صفر در نظر گرفته شده است. سپس برای هر گام زمانی، محاسبات مربوط به تولید حالت پنهان و خروجی انجام می شود.



۳.۱.۴ بهینه‌سازی با ترفند الحاق ماتریسی (Matrix Concatenation)

به جای انجام دو ضرب ماتریسی مجزا برای:

$$W_x x_t \quad \text{و} \quad W_h h_{t-1}$$

در این پیاده‌سازی از ترفند الحاق ماتریسی استفاده شده است. بدین صورت که ابتدا بردار ورودی و حالت پنهان قبلی با یکدیگر ترکیب می‌شوند:

$$[x_t; h_{t-1}]$$

سپس یک ماتریس وزن ترکیبی به صورت زیر تعریف می‌شود:

$$W_{\text{combined}} \in \mathbb{R}^{(\text{Features} + \text{Hidden}) \times \text{Hidden}}$$

و کل عملیات خطی تنها با یک ضرب ماتریسی انجام می‌شود:

$$h_t = \tanh([x_t, h_{t-1}] \cdot W_{\text{combined}} + b_h)$$

این کار باعث می‌شود:

- تعداد عملیات ضرب ماتریسی کاهش یابد،
- سربار محاسباتی کم شود،
- بهره‌وری کش پردازنده افزایش یابد،
- سرعت اجرای مدل به شکل محسوسی بیشتر شود.

در کد پیاده‌سازی شده، این کار با استفاده از تابع np.concatenate برای اتصال x_t و h_{t-1} و سپس یک np.dot واحد انجام شده است.

ذخیره‌سازی مقادیر برای پس‌انتشار برای اجرای صحیح الگوریتم BPTT در مرحله آموزش، مقادیر موردنیاز هر گام زمانی (شامل ورودی الحاق شده و حالت پنهان تولید شده) در یک ساختار cache ذخیره شده‌اند تا در مرحله پس‌انتشار مورد استفاده قرار گیرند.

۲.۴ پیاده‌سازی پس‌انتشار در زمان (BPTT) و کنترل انفجار گرادیان

۱.۲.۴ Backpropagation Through Time

در شبکه‌های بازگشتی، به دلیل وجود وابستگی زمانی، گرادیان خطا باید در طول تمامی گام‌های زمانی منتشر شود. این فرآیند با نام Backpropagation Through Time (BPTT) شناخته می‌شود.

در این روش، شبکه در طول زمان باز می‌شود (Unroll) و سپس گرادیان خطا از آخرین گام زمانی به سمت گام‌های ابتدایی منتشر می‌شود. بنابراین گرادیان هر گام نه تنها به خروجی همان گام بلکه به تأثیر آن بر گام‌های بعدی نیز وابسته است.



۲.۲.۴ پیاده‌سازی متد backward

در متد backward ابتدا گرادیان تمامی پارامترها مقداردهی اولیه شده‌اند. سپس حلقه زمانی به صورت معکوس اجرا می‌شود:

$$t = T - 1 \rightarrow 0$$

در هر گام:

- گرادیان خروجی نسبت به وزن‌های خروجی محاسبه می‌شود.
- گرادیان نسبت به حالت پنهان محاسبه می‌شود.
- مشتق تابع $\tanh$ به صورت $(1 - h_t^2)$ اعمال می‌شود.
- گرادیان نسبت به ماتریس وزن ترکیبی W_{combined} محاسبه می‌شود.
- بخش مربوط به حالت پنهان قبلی استخراج شده و به گام قبل منتقل می‌شود.
- به این ترتیب وابستگی زمانی گرادیان‌ها به درستی در کل دنباله لحاظ می‌شود.

۳.۲.۴ مشکل انفجار گرادیان و استفاده از Gradient Clipping

یکی از مشکلات اساسی در شبکه‌های بازگشتی، پدیده انفجار گرادیان (Exploding Gradients) است. در این حالت، هنگام انتشار گرادیان در طول زمان، مقدار گرادیان‌ها ممکن است به صورت نمایی افزایش یابد و باعث ناپایداری آموزش شود. برای جلوگیری از این مشکل، از تکنیک Gradient Clipping استفاده می‌شود. در این روش، قبل از به‌روزرسانی وزن‌ها، اندازه گرادیان‌ها محدود می‌شود:

$$g \leftarrow \text{clip}(g, -th, th)$$

که در آن th یک آستانه از پیش تعیین شده است. این کار باعث می‌شود:

- آموزش پایدارتر شود،
- از نوسانات شدید جلوگیری شود،
- همگرایی مدل بهبود یابد.

در پیاده‌سازی انجام شده، گرادیان‌های محاسبه شده در متد backward قبل از مرحله به‌روزرسانی وزن‌ها قابل تغییر هستند و می‌توان روی آن‌ها محدودسازی انجام داد تا از انفجار گرادیان جلوگیری کرد.



۳.۴ ایجاد داده مصنوعی Many-to-One و آموزش مدل با SGD

۱.۳.۴ تولید مجموعه داده مصنوعی

در این بخش یک مجموعه داده مصنوعی بسیار سبک با ساختار Many-to-One طراحی شد. در این نوع مسائل، کل دنباله ورودی تنها به یک برجسب نهایی نگاشت می شود. مسئله طراحی شده به صورت زیر است:

• ورودی: دنباله ای باینری شامل صفر و یک

• خروجی: تشخیص زوج یا فرد بودن تعداد عدد ۱ در کل دنباله

اگر تعداد ۱ها زوج باشد، برجسب کلاس ۰ (Even) و اگر فرد باشد، کلاس ۱ (Odd) اختصاص داده می شود. هر عنصر دنباله به صورت one-hot با دو ویژگی نمایش داده شده است:

$$0 \rightarrow [1, 0] \quad , \quad 1 \rightarrow [0, 1]$$

بنابراین ورودی شبکه دارای ابعاد زیر است:

(Size Batch, Length Sequence, 2)

تعداد کل نمونه ها برابر ۱۰۰۰ در نظر گرفته شد که ۸۰ درصد آن برای آموزش و ۲۰ درصد برای تست استفاده گردید. همچنین با استفاده از:

`np.random.seed(42)`

اطمینان حاصل شد که در تمامی رویکردها دقیقاً از داده های یکسان استفاده می شود. بنابراین مقایسه بین آزمایش ها کاملاً منصفانه بوده و تفاوت نتایج صرفاً ناشی از تغییر پارامترها است.

۲.۳.۴ رویکرد اول: دنباله کوتاه ($S_L = 3$) و نرخ یادگیری بالا

در رویکرد اول طول دنباله برابر با ۳ در نظر گرفته شد:

Length Sequence = 3

و مدل با نرخ یادگیری 0.9 و به مدت ۵۰۰ اپوک آموزش داده شد.

نتایج آموزش نتایج مربوط به فرآیند آموزش به صورت زیر است:

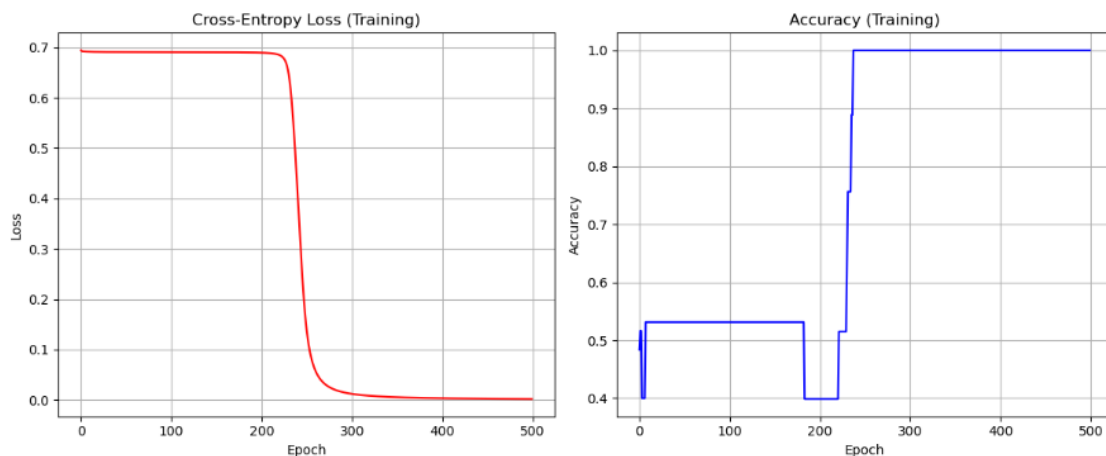
Epoch	Progress	Loss	Accuracy
50	50/500	0.6909	53.1%
100	100/500	0.6908	53.1%
150	150/500	0.6906	53.1%
200	200/500	0.6897	39.9%
250	250/500	0.1518	100.0%
⋮	⋮	⋮	⋮
500	500/500	0.0018	100.0%

نتیجه تست نتیجه مربوط به ارزیابی مدل روی داده های تست به صورت زیر است:

Test Accuracy: 100.00%

تحلیل از آنجا که طول دنباله کوتاه است، وابستگی زمانی محدود بوده و شبکه Vanilla RNN که ذاتاً حافظه کوتاه مدت دارد، می تواند به راحتی الگوی زوج/فرد بودن را یاد بگیرد.

مشاهده می شود که پس از حدود ۲۵۰ اپوک، مدل به همگرایی کامل رسیده و دقت آموزش و تست هر دو به ۱۰۰ درصد رسیده اند. این نشان می دهد که برای دنباله های کوتاه، مدل به خوبی توانایی استخراج وابستگی زمانی را دارد. نمودارهای Loss و accuracy در طول آموزش به صورت زیر است:



شکل ۱: نمودارهای دقت و خطا در طول آموزش (رویکرد اول)

همانطور که مشاهده می شود، مدل به صورت مناسبی آموزش دیده است.

۳.۳.۴ رویکرد دوم: دنباله بلند ($S_L = 10$) و نرخ یادگیری 0.9

در این رویکرد طول دنباله به ۱۰ افزایش داده شد و آموزش به مدت ۲۰۰۰ اپوک ادامه یافت.

نتایج آموزش نتیجه مربوط به فرآیند آموزش به صورت زیر است:

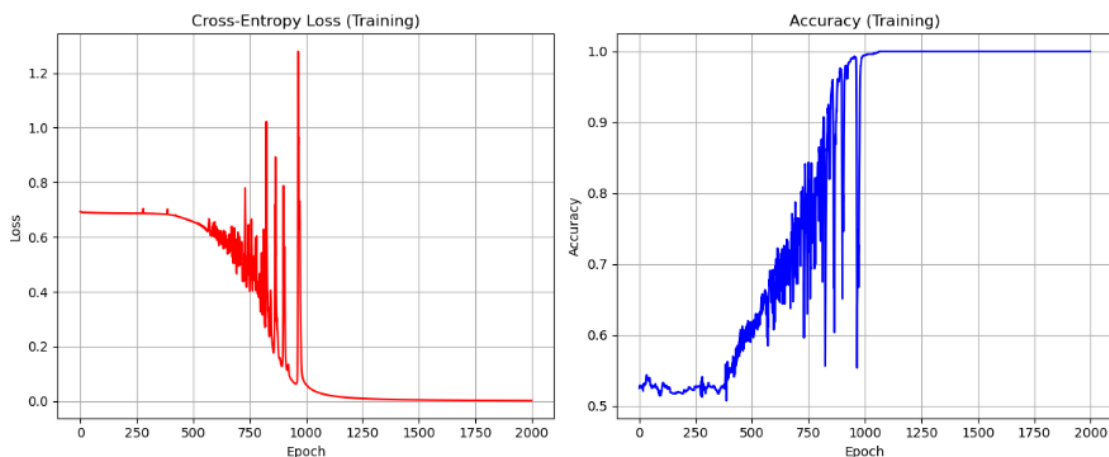
Epoch	Progress	Loss	Accuracy
500	500/2000	0.6558	59.9%
1000	1000/2000	0.0618	99.5%
1500	1500/2000	0.0047	100.0%
2000	2000/2000	0.0021	100.0%

نتیجه تست نتیجه مربوط به ارزیابی مدل روی داده های تست به صورت زیر است:

Test Accuracy: 83.00%

تحلیل با افزایش طول دنباله، وابستگی زمانی عمیق تر شده و گرادیان باید در طول گام های بیشتری منتشر شود. اگرچه در نهایت دقت آموزش به ۱۰۰ درصد رسید اما دقت تست تنها ۸۳ درصد بود. این موضوع نشان دهنده بیش برآزش (Overfitting) است.

شبکه به دلیل محدودیت حافظه کوتاه مدت خود، در دنباله های بلند قادر به یادگیری پایدار وابستگی های زمانی نیست. در نتیجه، پس از تعداد زیادی اپوک، مدل به سمت حفظ کردن نمونه های آموزش حرکت کرده و تعمیم پذیری آن کاهش یافته است. این رفتار از محدودیت های ذاتی Vanilla RNN ناشی می شود که در دنباله های بلند دچار مشکل می شود. نمودارهای Loss و accuracy در طول آموزش به صورت زیر است:



شکل ۲: نمودارهای دقت و خطا در طول آموزش (رویکرد دوم)

۴.۳.۴ تحلیل نوسانات تابع هزینه و دقت

در رویکرد دوم، نمودارهای Loss و Accuracy بر حسب تعداد اپوک رفتار یکنواخت و همگرایی مناسبی ندارند و نوسانات قابل توجهی در طول آموزش مشاهده می شود. این پدیده عمدتاً به دو عامل اصلی بازمی گردد: افزایش طول دنباله زمانی و استفاده از بهینه ساز ساده‌ی SGD.



تأثیر استفاده از SGD در این پیاده‌سازی، بهینه‌سازی پارامترها با استفاده از Stochastic Gradient Descent (SGD) انجام شده است. در SGD، به‌روزرسانی وزن‌ها تنها بر اساس گرادیان لحظه‌ای انجام می‌شود و هیچ حافظه‌ای از جهت یا شدت گرادیان‌های قبلی وجود ندارد. در نتیجه:

- مسیر بهینه‌سازی بسیار حساس به نویز گرادیان است،
- مدل ممکن است بین نواحی مختلف تابع هزینه نوسان کند،
- در حضور نرخ یادگیری بالا، پرش‌های شدید اطراف کمینه‌ها رخ می‌دهد،
- شبکه به‌راحتی در کمینه‌های محلی یا نواحی ناپایدار گرفتار می‌شود.

این رفتار به‌ویژه در شبکه‌های بازگشتی با دنباله‌های بلند تشدید می‌شود، زیرا گرادیان‌ها حاصل ترکیب اثرات چندین گام زمانی هستند و ذاتاً نویز و ناپایداری بیشتری دارند.

نقش طول دنباله در نوسانات در این رویکرد، طول دنباله برابر با $S_L = 10$ است. با افزایش طول دنباله:

- عمق مؤثر شبکه در زمان افزایش می‌یابد،
- حساسیت گرادیان نسبت به تغییرات کوچک وزن‌ها بیشتر می‌شود،
- خطای هر گام زمانی می‌تواند اثر تجمعی شدیدی بر گرادیان نهایی داشته باشد.

در چنین شرایطی، استفاده از SGD بدون مکانیزم‌های کمکی، منجر به نوسان شدید در مسیر یادگیری می‌شود که به‌وضوح در نمودارهای Loss و Accuracy مشاهده می‌گردد.

چرا Adam یا Momentum می‌توانند این مشکل را کاهش دهند؟ بهینه‌سازهایی مانند SGD with Momentum و Adam با افزودن حافظه به فرآیند به‌روزرسانی وزن‌ها، بخش زیادی از این نوسانات را کاهش می‌دهند:

- Momentum با میانگین‌گیری جهت‌دار از گرادیان‌های قبلی، نوسانات کوتاه‌مدت را فیلتر کرده و حرکت هموارتری به سمت کمینه ایجاد می‌کند.
- Adam علاوه بر مومنتوم، از نرخ یادگیری تطبیقی برای هر پارامتر استفاده می‌کند که باعث پایداری بیشتر آموزش، به‌ویژه در مسائل با گرادیان‌های ناپایدار می‌شود.

در نتیجه، اگر در این رویکرد به‌جای SGD از Adam یا SGD with Momentum استفاده می‌شد، انتظار می‌رفت:

- نوسانات نمودار Loss کاهش یابد،
- منحنی Accuracy رفتار هموارتر و صعودی‌تری داشته باشد،
- همگرایی مدل سریع‌تر و پایدارتر رخ دهد.

جمع‌بندی نوسانات مشاهده‌شده در نمودارهای آموزشی رویکرد دوم نه به ضعف پیاده‌سازی، بلکه به ترکیب دو عامل زیر بازمی‌گردد:

۱. طول دنباله زمانی نسبتاً بلند،

۲. بهره‌گیری از بهینه‌ساز ساده‌ی SGD بدون مومنتوم.

این تحلیل به خوبی نشان می‌دهد که انتخاب بهینه‌ساز نقش کلیدی در پایداری آموزش شبکه‌های بازگشتی دارد و برای مسائل ترتیبی با وابستگی زمانی عمیق، استفاده از بهینه‌سازهای پیشرفته‌تر امری ضروری است.

۵.۳.۴ رویکرد سوم: دنباله بلند ($S_L = 10$) و نرخ یادگیری پایین (0.1)

در این رویکرد همان تنظیمات رویکرد دوم حفظ شد اما نرخ یادگیری از 0.9 به 0.1 کاهش داده شد.

نتایج آموزش نتایج مربوط به فرآیند آموزش به صورت زیر است:

Epoch	Progress	Loss	Accuracy
1000	1000/2000	0.6876	52.9%
1500	1500/2000	0.6872	51.9%
2000	2000/2000	0.6868	52.6%

نتیجه تست نتیجه مربوط به ارزیابی مدل روی داده های تست به صورت زیر است:

Test Accuracy: 48.00%

تحلیل در این حالت مدل تقریباً هیچ یادگیری مؤثری انجام نداده است. مقدار تابع هزینه در حوالی 0.69 باقی مانده که معادل حدس تصادفی برای طبقه‌بندی دودویی است.

علت اصلی این رفتار، پدیده محو شدن گرادیان (Vanishing Gradient) است.

در شبکه‌های بازگشتی، گرادیان هنگام عبور از گام‌های زمانی متعدد، به صورت ضرب‌های متوالی از مشتق تابع فعال‌سازی (مانند tanh) عبور می‌کند. از آنجا که مشتق tanh مقداری کمتر از ۱ دارد:

$$|\tanh'(x)| \leq 1$$

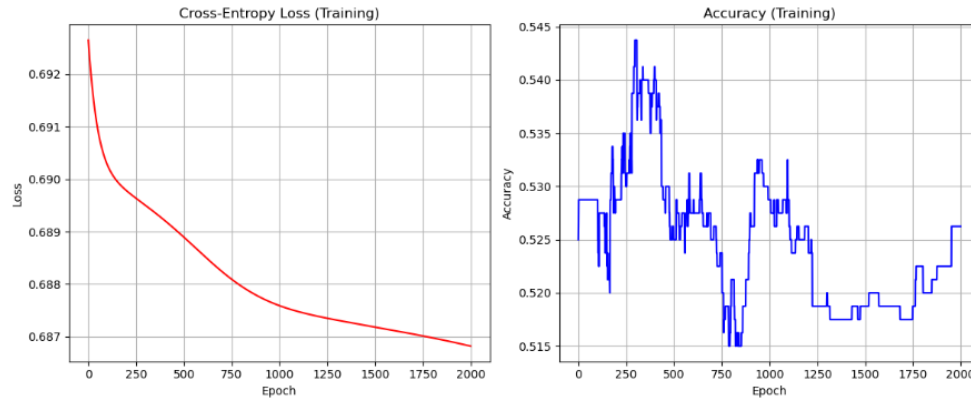
گرادیان در هر گام زمانی کوچک‌تر می‌شود و در گام‌های ابتدایی تقریباً به صفر میل می‌کند.
در نتیجه:

- گرادیان وزن‌های ابتدایی بسیار کوچک می‌شود،

- وقتی این گرادیان کوچک در نرخ یادگیری کوچک (0.1) ضرب می‌شود،

- تغییر وزن‌ها ناچیز شده و عملاً یادگیری متوقف می‌شود.

به همین دلیل مشاهده می‌شود که مدل حتی پس از ۲۰۰۰ اپوک نیز عملکردی نزدیک به حد تصادفی دارد (دقت تست ۴۸ درصد). این نتیجه به‌وضوح نشان می‌دهد که ترکیب دنباله بلند + گرادینت‌های محوشونده + نرخ یادگیری پایین می‌تواند یادگیری را کاملاً مختل کند.



شکل ۳: نمودارهای دقت و خطا در طول آموزش (رویکرد سوم)

در نمودارهای بالا مشاهده می‌شود که مدل چیزی یاد نگرفته و رفتار تصادفی دارد. در واقع مدل underfit است.

کد ۶.۳.۴

```

1  #!/usr/bin/env python
2  # coding: utf-8
3
4  # # Implementation of the VanillaRNN class
5
6  # In[62]:
7
8
9  import numpy as np
10
11
12  # =====
13  # Part A and B: Implementation of the RNN class
14  # =====
15  class BatchedVanillaRNN:
16      def __init__(self, features, hidden_size, output_size):
17          self.features = features
18          self.hidden_size = hidden_size
19          self.output_size = output_size
20
    
```




```
۲۱     # Initialize weights with a normal (Gaussian) distribution to prevent early tanh
۲۲     ↪ saturation
۲۳     self.W_combined = np.random.randn(features + hidden_size, hidden_size) * 1.0
۲۴     self.b_h = np.zeros((1, hidden_size))
۲۵
۲۶     self.W_y = np.random.randn(hidden_size, output_size) * 1.0
۲۷     self.b_y = np.zeros((1, output_size))
۲۸
۲۹     def forward(self, inputs):
۳۰         batch_size, seq_len, features = inputs.shape
۳۱
۳۲         self.cache = []
۳۳         h_prev = np.zeros((batch_size, self.hidden_size))
۳۴         outputs = np.zeros((batch_size, seq_len, self.output_size))
۳۵
۳۶         for t in range(seq_len):
۳۷             x_t = inputs[:, t, :]
۳۸             # Matrix concatenation trick
۳۹             concat_input = np.concatenate((x_t, h_prev), axis=1)
۴۰
۴۱             # Single matrix multiplication
۴۲             h_raw = np.dot(concat_input, self.W_combined) + self.b_h
۴۳             h_next = np.tanh(h_raw)
۴۴             y_t = np.dot(h_next, self.W_y) + self.b_y
۴۵
۴۶             self.cache.append((concat_input, h_next))
۴۷             outputs[:, t, :] = y_t
۴۸             h_prev = h_next
۴۹
۵۰         return outputs
۵۱
۵۲     def backward(self, d_outputs):
۵۳         dW_combined = np.zeros_like(self.W_combined)
۵۴         db_h = np.zeros_like(self.b_h)
۵۵         dW_y = np.zeros_like(self.W_y)
۵۶         db_y = np.zeros_like(self.b_y)
۵۷
۵۸         batch_size, seq_len, _ = d_outputs.shape
۵۹         dh_next = np.zeros((batch_size, self.hidden_size))
```



```
۶۰     for t in reversed(range(seq_len)):
۶۱         dy_t = d_outputs[:, t, :]
۶۲         concat_input, h_t = self.cache[t]
۶۳
۶۴         if np.any(dy_t != 0):
۶۵             dW_y += np.dot(h_t.T, dy_t)
۶۶             db_y += np.sum(dy_t, axis=0, keepdims=True)
۶۷
۶۸         dh_t = np.dot(dy_t, self.W_y.T) + dh_next
۶۹         dtanh = (1 - h_t ** 2) * dh_t
۷۰
۷۱         dW_combined += np.dot(concat_input.T, dtanh)
۷۲         db_h += np.sum(dtanh, axis=0, keepdims=True)
۷۳
۷۴         d_concat_input = np.dot(dtanh, self.W_combined.T)
۷۵         dh_next = d_concat_input[:, self.features:]
۷۶
۷۷         gradients = {
۷۸             'dW_combined': dW_combined, 'db_h': db_h,
۷۹             'dW_y': dW_y, 'db_y': db_y
۸۰         }
۸۱         return gradients
۸۲
۸۳
۸۴ # # Implementing Softmax and Cross_entropy_loss
۸۵
۸۶ # In[63]:
۸۷
۸۸
۸۹ def softmax(x):
۹۰     e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
۹۱     return e_x / np.sum(e_x, axis=-1, keepdims=True)
۹۲
۹۳ def cross_entropy_loss(probs, y_true):
۹۴     n_samples = y_true.shape[0]
۹۵     log_likelihood = -np.log(probs[range(n_samples), y_true] + 1e-9)
۹۶     loss = np.sum(log_likelihood) / n_samples
۹۷
۹۸     grad = probs.copy()
۹۹     grad[range(n_samples), y_true] -= 1
```



```
100     grad /= n_samples
101     return loss, grad
102
103
104 # # Generating Data
105
106 # In[64]:
107
108
109 def generate_data(num_samples, seq_len):
110     X = np.random.randint(0, 2, size=(num_samples, seq_len))
111     y = np.sum(X, axis=1) % 2
112
113     X_onehot = np.zeros((num_samples, seq_len, 2))
114     for i in range(num_samples):
115         for t in range(seq_len):
116             X_onehot[i, t, X[i, t]] = 1
117
118     return X_onehot, y
119
120
121 # # Training model on the train set(S_L=3,Lr=0.9,Epochs=500), Inference on the test set and
122 # → evaluate it
123
124 # In[71]:
125
126
127 if __name__ == "__main__":
128     np.random.seed(42)
129     # 1. Explicit definition of network dimensions
130     BATCH_SIZE = 800           # Training batch size
131     SEQUENCE_LENGTH = 3       # Length of time sequence
132     FEATURES = 2              # Number of features at each timestep (binary one-hot)
133
134     HIDDEN_SIZE = 32
135     OUTPUT_SIZE = 2
136     LEARNING_RATE = 9.0
137     EPOCHS = 500
138     TOTAL_SAMPLES = 1000
139     CLIP_THRESHOLD = 1
```



```
139
140     # 2. Generate and split data into Train (80%) and Test (20%)
141     X_all, y_all = generate_data(TOTAL_SAMPLES, SEQUENCE_LENGTH)
142
143     X_train, y_train = X_all[:BATCH_SIZE], y_all[:BATCH_SIZE]
144     X_test, y_test = X_all[BATCH_SIZE:], y_all[BATCH_SIZE:]
145
146     print(f"Train Shape: {X_train.shape} | Test Shape: {X_test.shape}")
147
148     rnn = BatchedVanillaRNN(FEATURES, HIDDEN_SIZE, OUTPUT_SIZE)
149
150     loss_history = []
151     acc_history = []
152
153     print("\n--- Training Start ---")
154     for epoch in range(EPOCHS):
155         outputs = rnn.forward(X_train)
156         last_outputs = outputs[:, -1, :] # Many-to-One
157
158         probs = softmax(last_outputs)
159         loss, d_loss = cross_entropy_loss(probs, y_train)
160
161         # Calculate accuracy on training data
162         predictions = np.argmax(probs, axis=1)
163         accuracy = np.mean(predictions == y_train)
164
165         loss_history.append(loss)
166         acc_history.append(accuracy)
167
168         d_outputs = np.zeros_like(outputs)
169         d_outputs[:, -1, :] = d_loss
170
171         gradients = rnn.backward(d_outputs)
172
173         # Gradient clipping
174         for key in gradients:
175             np.clip(gradients[key], -CLIP_THRESHOLD, CLIP_THRESHOLD, out=gradients[key])
176
177         # Update weights
178         rnn.W_combined -= LEARNING_RATE * gradients['dW_combined']
```



```
179     rnn.b_h -= LEARNING_RATE * gradients['db_h']
180     rnn.W_y -= LEARNING_RATE * gradients['dW_y']
181     rnn.b_y -= LEARNING_RATE * gradients['db_y']
182
183     if (epoch + 1) % 50 == 0:
184         print(f"Epoch {epoch+1:3d}/{EPOCHS} | Loss: {loss:4f.} | Acc: {accuracy*100:1f.}%")
185
186     # 3. Evaluation on test data (unseen data)
187     print("\n--- Evaluation on Test Data ---")
188     test_outputs = rnn.forward(X_test)
189     test_probs = softmax(test_outputs[:, -1, :])
190     test_preds = np.argmax(test_probs, axis=1)
191
192     test_accuracy = np.mean(test_preds == y_test)
193     print(f"Test Accuracy: {test_accuracy*100:2f.}%\n")
194
195     # Display some examples of model performance on test data
196     print("Displaying first 5 test samples:")
197     for i in range(5):
198         seq = [np.argmax(x) for x in X_test[i]] # Convert one-hot back to original numbers for
199         ↪ display
200         print(f"Sequence: {seq}")
201         print(f"Model Prediction: {'Odd (1)' if test_preds[i] == 1 else 'Even (0)'} | True Label:
202         ↪ {'Odd (1)' if y_test[i] == 1 else 'Even (0)'}")
203         print("-" * 30)
204
205     # In[72]:
206
207     import matplotlib.pyplot as plt
208     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
209
210     ax1.plot(loss_history, color='red')
211     ax1.set_title("Cross-Entropy Loss (Training)")
212     ax1.set_xlabel("Epoch")
213     ax1.set_ylabel("Loss")
214     ax1.grid(True)
215
216     ax2.plot(acc_history, color='blue')
```



```
۲۱۷ ax2.set_title("Accuracy (Training)")
۲۱۸ ax2.set_xlabel("Epoch")
۲۱۹ ax2.set_ylabel("Accuracy")
۲۲۰ ax2.grid(True)
۲۲۱
۲۲۲ plt.tight_layout()
۲۲۳ plt.show()
۲۲۴
۲۲۵
۲۲۶ # # Training model on the train set(S_L=10,Lr=0.9,Epochs=2000), Inference on the test set and
    → evaluate it
۲۲۷
۲۲۸ # In[73]:
۲۲۹
۲۳۰
۲۳۱ if __name__ == "__main__":
۲۳۲     np.random.seed(42)
۲۳۳     BATCH_SIZE = 800
۲۳۴     SEQUENCE_LENGTH = 10
۲۳۵     FEATURES = 2
۲۳۶
۲۳۷     HIDDEN_SIZE = 32
۲۳۸     OUTPUT_SIZE = 2
۲۳۹     # Increase learning rate to 0.1 (because using vanilla SGD and gradients are averaged)
۲۴۰     LEARNING_RATE = 9.0
۲۴۱     EPOCHS = 2000
۲۴۲     TOTAL_SAMPLES = 1000
۲۴۳     CLIP_THRESHOLD = 1
۲۴۴
۲۴۵     X_all, y_all = generate_data(TOTAL_SAMPLES, SEQUENCE_LENGTH)
۲۴۶
۲۴۷     X_train, y_train = X_all[:BATCH_SIZE], y_all[:BATCH_SIZE]
۲۴۸     X_test, y_test = X_all[BATCH_SIZE:], y_all[BATCH_SIZE:]
۲۴۹
۲۵۰     print(f"Train Shape: {X_train.shape} | Test Shape: {X_test.shape}")
۲۵۱
۲۵۲     rnn = BatchedVanillaRNN(FEATURES, HIDDEN_SIZE, OUTPUT_SIZE)
۲۵۳
۲۵۴     loss_history = []
۲۵۵     acc_history = []
```



```
۲۵۶
۲۵۷ print("\n--- Training Start ---")
۲۵۸ for epoch in range(EPOCHS):
۲۵۹     outputs = rnn.forward(X_train)
۲۶۰     last_outputs = outputs[:, -1, :]
۲۶۱
۲۶۲     probs = softmax(last_outputs)
۲۶۳     loss, d_loss = cross_entropy_loss(probs, y_train)
۲۶۴
۲۶۵     predictions = np.argmax(probs, axis=1)
۲۶۶     accuracy = np.mean(predictions == y_train)
۲۶۷
۲۶۸     loss_history.append(loss)
۲۶۹     acc_history.append(accuracy)
۲۷۰
۲۷۱     d_outputs = np.zeros_like(outputs)
۲۷۲     d_outputs[:, -1, :] = d_loss
۲۷۳
۲۷۴     gradients = rnn.backward(d_outputs)
۲۷۵
۲۷۶     for key in gradients:
۲۷۷         np.clip(gradients[key], -CLIP_THRESHOLD, CLIP_THRESHOLD, out=gradients[key])
۲۷۸
۲۷۹     rnn.W_combined -= LEARNING_RATE * gradients['dW_combined']
۲۸۰     rnn.b_h -= LEARNING_RATE * gradients['db_h']
۲۸۱     rnn.W_y -= LEARNING_RATE * gradients['dW_y']
۲۸۲     rnn.b_y -= LEARNING_RATE * gradients['db_y']
۲۸۳
۲۸۴     if (epoch + 1) % 100 == 0:
۲۸۵         print(f"Epoch {epoch+1:4d}/{EPOCHS} | Loss: {loss:4f.} | Acc: {accuracy*100:1f.}%")
۲۸۶
۲۸۷ print("\n--- Evaluation on Test Data ---")
۲۸۸ test_outputs = rnn.forward(X_test)
۲۸۹ test_probs = softmax(test_outputs[:, -1, :])
۲۹۰ test_preds = np.argmax(test_probs, axis=1)
۲۹۱
۲۹۲ test_accuracy = np.mean(test_preds == y_test)
۲۹۳ print(f"Test Accuracy: {test_accuracy*100:2f.}%\n")
۲۹۴
۲۹۵ print("Displaying first 5 test samples:")
```



```
۲۹۶     for i in range(5):
۲۹۷         seq = [np.argmax(x) for x in X_test[i]]
۲۹۸         print(f"Sequence: {seq}")
۲۹۹         print(f"Model Prediction: {'Odd (1)' if test_preds[i] == 1 else 'Even (0)'} | True Label:
        ↳ {'Odd (1)' if y_test[i] == 1 else 'Even (0)'}")
۳۰۰         print("-" * 30)
۳۰۱
۳۰۲
۳۰۳ # In[74]:
۳۰۴
۳۰۵
۳۰۶ fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
۳۰۷
۳۰۸ ax1.plot(loss_history, color='red')
۳۰۹ ax1.set_title("Cross-Entropy Loss (Training)")
۳۱۰ ax1.set_xlabel("Epoch")
۳۱۱ ax1.set_ylabel("Loss")
۳۱۲ ax1.grid(True)
۳۱۳
۳۱۴ ax2.plot(acc_history, color='blue')
۳۱۵ ax2.set_title("Accuracy (Training)")
۳۱۶ ax2.set_xlabel("Epoch")
۳۱۷ ax2.set_ylabel("Accuracy")
۳۱۸ ax2.grid(True)
۳۱۹
۳۲۰ plt.tight_layout()
۳۲۱ plt.show()
۳۲۲
۳۲۳
۳۲۴ # # Training model on the train set( $S_L=10, Lr=0.1, Epochs=2000$ ), Inference on the test set and
        ↳ evaluate it
۳۲۵
۳۲۶ # In[76]:
۳۲۷
۳۲۸
۳۲۹ if __name__ == "__main__":
۳۳۰     np.random.seed(42)
۳۳۱     BATCH_SIZE = 800
۳۳۲     SEQUENCE_LENGTH = 10
۳۳۳     FEATURES = 2
```




```
۳۳۴
۳۳۵     HIDDEN_SIZE = 32
۳۳۶     OUTPUT_SIZE = 2
۳۳۷     # Decrease learning rate to 1.0 (because we're using simple SGD and gradients are averaged)
۳۳۸     LEARNING_RATE = 1.0
۳۳۹     EPOCHS = 2000
۳۴۰     TOTAL_SAMPLES = 1000
۳۴۱     CLIP_THRESHOLD = 1
۳۴۲
۳۴۳     X_all, y_all = generate_data(TOTAL_SAMPLES, SEQUENCE_LENGTH)
۳۴۴
۳۴۵     X_train, y_train = X_all[:BATCH_SIZE], y_all[:BATCH_SIZE]
۳۴۶     X_test, y_test = X_all[BATCH_SIZE:], y_all[BATCH_SIZE:]
۳۴۷
۳۴۸     print(f"Train Shape: {X_train.shape} | Test Shape: {X_test.shape}")
۳۴۹
۳۵۰     rnn = BatchedVanillaRNN(FEATURES, HIDDEN_SIZE, OUTPUT_SIZE)
۳۵۱
۳۵۲     loss_history = []
۳۵۳     acc_history = []
۳۵۴
۳۵۵     print("\n--- Training Start ---")
۳۵۶     for epoch in range(EPOCHS):
۳۵۷         outputs = rnn.forward(X_train)
۳۵۸         last_outputs = outputs[:, -1, :]
۳۵۹
۳۶۰         probs = softmax(last_outputs)
۳۶۱         loss, d_loss = cross_entropy_loss(probs, y_train)
۳۶۲
۳۶۳         predictions = np.argmax(probs, axis=1)
۳۶۴         accuracy = np.mean(predictions == y_train)
۳۶۵
۳۶۶         loss_history.append(loss)
۳۶۷         acc_history.append(accuracy)
۳۶۸
۳۶۹         d_outputs = np.zeros_like(outputs)
۳۷۰         d_outputs[:, -1, :] = d_loss
۳۷۱
۳۷۲         gradients = rnn.backward(d_outputs)
۳۷۳
```



```

۳۷۴     for key in gradients:
۳۷۵         np.clip(gradients[key], -CLIP_THRESHOLD, CLIP_THRESHOLD, out=gradients[key])
۳۷۶
۳۷۷     rnn.W_combined -= LEARNING_RATE * gradients['dW_combined']
۳۷۸     rnn.b_h -= LEARNING_RATE * gradients['db_h']
۳۷۹     rnn.W_y -= LEARNING_RATE * gradients['dW_y']
۳۸۰     rnn.b_y -= LEARNING_RATE * gradients['db_y']
۳۸۱
۳۸۲     if (epoch + 1) % 100 == 0:
۳۸۳         print(f"Epoch {epoch+1:4d}/{EPOCHS} | Loss: {loss:4f.} | Acc: {accuracy*100:1f.}%")
۳۸۴
۳۸۵     print("\n--- Evaluation on Test Data ---")
۳۸۶     test_outputs = rnn.forward(X_test)
۳۸۷     test_probs = softmax(test_outputs[:, -1, :])
۳۸۸     test_preds = np.argmax(test_probs, axis=1)
۳۸۹
۳۹۰     test_accuracy = np.mean(test_preds == y_test)
۳۹۱     print(f"Test Accuracy: {test_accuracy*100:2f.}%\n")
۳۹۲
۳۹۳     print("Displaying first 5 test samples:")
۳۹۴     for i in range(5):
۳۹۵         seq = [np.argmax(x) for x in X_test[i]]
۳۹۶         print(f"Sequence: {seq}")
۳۹۷         print(f"Model Prediction: {'Odd (1)' if test_preds[i] == 1 else 'Even (0)'} | True Label:
↳ {'Odd (1)' if y_test[i] == 1 else 'Even (0)'}")
۳۹۸         print("-" * 30)
۳۹۹
۴۰۰
۴۰۱ # In[77]:
۴۰۲
۴۰۳
۴۰۴ fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
۴۰۵
۴۰۶ ax1.plot(loss_history, color='red')
۴۰۷ ax1.set_title("Cross-Entropy Loss (Training)")
۴۰۸ ax1.set_xlabel("Epoch")
۴۰۹ ax1.set_ylabel("Loss")
۴۱۰ ax1.grid(True)
۴۱۱
۴۱۲ ax2.plot(acc_history, color='blue')

```



```

۴۱۳ ax2.set_title("Accuracy (Training)")
۴۱۴ ax2.set_xlabel("Epoch")
۴۱۵ ax2.set_ylabel("Accuracy")
۴۱۶ ax2.grid(True)
۴۱۷
۴۱۸ plt.tight_layout()
۴۱۹ plt.show()
۴۲۰

```

۵ سوال ۵: پیاده سازی حرفه ای و ترفند های آموزش RNN ها در Pytorch

۱.۵ داده های ترتیبی (Sequential Data)

در بسیاری از مسائل یادگیری ماشین، داده ها به صورت مستقل از یکدیگر در نظر گرفته می شوند. برای مثال در مسائل دسته بندی تصاویر، هر تصویر یک نمونه مستقل است و ترتیب قرارگیری تصاویر در دیتاست تأثیری بر نتیجه مدل ندارد. اما در دسته ای از مسائل، ترتیب داده ها دارای معنا و اطلاعات مهمی است. این نوع داده ها را داده های ترتیبی یا Sequential Data می نامند. در داده های ترتیبی، هر عنصر از دنباله به عناصر قبلی یا حتی عناصر بعدی وابسته است. به عبارت دیگر، اگر ترتیب داده ها تغییر کند، معنای کل دنباله نیز تغییر خواهد کرد. یک دنباله را می توان به صورت زیر نمایش داد:

$$X = (x_1, x_2, x_3, \dots, x_T) \quad (۷۷)$$

در این رابطه:

- عنصر مربوط به گام زمانی t است x_t

- طول دنباله است T

در بسیاری از مسائل، هدف این است که با مشاهده کل دنباله، یک برچسب برای آن پیش بینی شود. این نوع مسئله را Sequence Classification می نامند.

تابع یادگیری در این حالت به شکل زیر تعریف می شود:

$$f(x_1, x_2, \dots, x_T) = y \quad (۷۸)$$

که در آن y برچسب نهایی دنباله است.

نمونه های رایج از داده های ترتیبی عبارت اند از:

- متن در پردازش زبان طبیعی (Natural Language Processing)



- سیگنال‌های صوتی
 - سری‌های زمانی مالی
 - داده‌های سنسورها
 - توالی‌های زیستی مانند DNA
- برای مثال در یک جمله:

The cat sat on the mat

ترتیب کلمات بسیار مهم است. اگر ترتیب کلمات تغییر کند، معنی جمله کاملاً عوض می‌شود. به همین دلیل مدل باید بتواند وابستگی بین عناصر مختلف دنباله را یاد بگیرد. این وابستگی معمولاً به صورت زیر مدل می‌شود:

$$P(x_t | x_1, x_2, \dots, x_{t-1}) \quad (۷۹)$$

یعنی احتمال وقوع عنصر x_t وابسته به عناصر قبلی دنباله است. مدل‌های معمول مانند شبکه‌های عصبی Feedforward نمی‌توانند این وابستگی ترتیبی را به خوبی مدل کنند، زیرا این مدل‌ها فرض می‌کنند ورودی‌ها مستقل هستند. به همین دلیل برای داده‌های ترتیبی از معماری‌هایی مانند RNN، LSTM و GRU استفاده می‌شود که قادرند اطلاعات گذشته را در حالت پنهان (Hidden State) ذخیره کنند.

۲.۵ چالش توالی‌های طولانی (Long Sequences)

در بسیاری از کاربردهای واقعی، طول دنباله‌ها می‌تواند بسیار بزرگ باشد. برای مثال:

- یک جمله ممکن است 20 تا 30 کلمه داشته باشد
 - یک پاراگراف ممکن است بیش از 100 کلمه داشته باشد
 - سیگنال‌های صوتی ممکن است هزاران گام زمانی داشته باشند
- در چنین شرایطی، مدل باید بتواند اطلاعات مربوط به مراحل اولیه دنباله را تا مراحل انتهایی حفظ کند. در شبکه‌های بازگشتی ساده (RNN) حالت پنهان در هر گام زمانی به صورت زیر محاسبه می‌شود:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b) \quad (۸۰)$$

در این رابطه:

- x_t ورودی در زمان t
- h_{t-1} حالت پنهان قبلی

• ماتریس وزن انتقال حالت W_{hh}

• ماتریس وزن ورودی W_{xh}

این رابطه نشان می‌دهد که هر حالت پنهان به حالت قبلی وابسته است. اگر این رابطه را برای چند گام زمانی باز کنیم، مشاهده می‌کنیم که حالت پنهان در زمان t به صورت زیر به حالت اولیه وابسته می‌شود:

$$h_t = \tanh(W_{hh}^t h_0 + \sum_{k=1}^t W_{hh}^{t-k} W_{xh} x_k) \quad (۸۱)$$

در این معادله، توان‌های متوالی ماتریس W_{hh} ظاهر می‌شوند. این موضوع در فرآیند آموزش شبکه مشکلاتی ایجاد می‌کند که یکی از مهم‌ترین آن‌ها انفجار گرادیان است.

۳.۵ پدیده انفجار گرادیان (Exploding Gradient)

شبکه‌های بازگشتی با استفاده از الگوریتم Backpropagation Through Time یا به اختصار BPTT آموزش داده می‌شوند. در این الگوریتم، گرادیان خطا نسبت به پارامترها از انتهای دنباله به ابتدای آن منتقل می‌شود. گرادیان کلی نسبت به یک پارامتر به صورت زیر نوشته می‌شود:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \left(\frac{\partial L}{\partial h_t} \prod_{k=t}^1 \frac{\partial h_k}{\partial h_{k-1}} \right) \quad (۸۲)$$

در این رابطه یک ضرب زنجیره‌ای از ژاکوبین‌ها وجود دارد:

$$\prod_{k=1}^T \frac{\partial h_k}{\partial h_{k-1}} \quad (۸۳)$$

اگر مقدار نرم این ژاکوبین‌ها بزرگ‌تر از ۱ باشد، این ضرب زنجیره‌ای به صورت نمایی رشد می‌کند.

$$\left\| \frac{\partial h_k}{\partial h_{k-1}} \right\| > 1 \quad (۸۴)$$

در نتیجه:

$$\left\| \prod_{k=1}^T \frac{\partial h_k}{\partial h_{k-1}} \right\| \rightarrow \infty \quad (۸۵)$$

برای درک بهتر این موضوع یک مثال عددی ساده در نظر بگیرید. فرض کنید مقدار متوسط این ضریب برابر ۱.۵ باشد و طول دنباله برابر ۲۰ باشد:

$$1.5^{20} \approx 3325 \quad (۸۶)$$

در این حالت مقدار گرادیان به سرعت بسیار بزرگ می‌شود. این موضوع باعث مشکلات زیر می‌شود:



- ناپایداری در فرآیند آموزش

- جهش‌های شدید در به‌روزرسانی پارامترها

- تولید مقادیر NaN

- عدم همگرایی مدل

به همین دلیل در مدل‌های بازگشتی معمولاً از تکنیک‌هایی مانند Gradient Clipping استفاده می‌شود تا اندازه گرادینت محدود شود.

۴.۵ محاسبات هدررفته ناشی از Padding

یکی دیگر از چالش‌های مهم در کار با داده‌های ترتیبی، متفاوت بودن طول دنباله‌ها است. برای مثال ممکن است دو دنباله زیر را داشته باشیم:

$$S_1 = (x_1, x_2, x_3, x_4, x_5) \quad (۸۷)$$

$$S_2 = (y_1, y_2) \quad (۸۸)$$

برای اینکه بتوانیم این داده‌ها را به صورت دسته‌ای (Batch) به شبکه عصبی بدهیم، باید طول آن‌ها را یکسان کنیم. رایج‌ترین روش برای این کار استفاده از Padding است. در این روش دنباله کوتاه‌تر با مقادیر صفر یا یک توکن خاص پر می‌شود:

$$S_1 = (x_1, x_2, x_3, x_4, x_5) \quad (۸۹)$$

$$S_2 = (y_1, y_2, 0, 0, 0) \quad (۹۰)$$

در اینجا مقدار 0 نشان‌دهنده padding token است. اگر اندازه بچ برابر 2 و طول حداکثر دنباله برابر 5 باشد، مدل مجبور است در مجموع:

$$2 \times 5 = 10 \quad (۹۱)$$

گام محاسباتی انجام دهد. اما در واقع تعداد گام‌های واقعی برابر است با:

$$5 + 2 = 7 \quad (۹۲)$$

بنابراین:

$$10 - 7 = 3 = \text{محاسبات اضافی} \quad (۹۳)$$

هرچه اختلاف طول دنباله‌ها بیشتر باشد، این اتلاف محاسباتی نیز بیشتر خواهد شد. این مشکل دو پیامد مهم دارد:

- افزایش زمان محاسبات
- مصرف بیشتر حافظه

علاوه بر این، اگر مدل از وجود padding آگاه نباشد، ممکن است این مقادیر را به عنوان داده واقعی پردازش کند که باعث ایجاد نویز در فرآیند یادگیری می‌شود.

به همین دلیل در بسیاری از پیاده‌سازی‌های مدرن از تکنیک‌هایی مانند Packed Sequences استفاده می‌شود که باعث می‌شود مدل فقط روی بخش‌های واقعی دنباله محاسبه انجام دهد و گام‌های مربوط به padding را نادیده بگیرد.

۵.۵ ساختار و نحوه عملکرد BiLSTM

شبکه‌های Bidirectional Long Short-Term Memory که به اختصار BiLSTM نامیده می‌شوند، توسعه‌ای از معماری LSTM هستند که هدف آن‌ها بهره‌برداری همزمان از اطلاعات گذشته و آینده در یک دنباله است. در بسیاری از مسائل ترتیبی، تنها استفاده از اطلاعات گذشته کافی نیست، زیرا معنای یک عنصر ممکن است به عناصر بعدی نیز وابسته باشد. به عنوان مثال در جمله زیر:

He went to the bank

کلمه bank می‌تواند به معنی بانک مالی یا کناره رودخانه باشد. برای تشخیص معنای صحیح، نیاز داریم کلمات بعدی جمله را نیز ببینیم. در چنین حالتی استفاده از یک LSTM یک‌طرفه که فقط از گذشته به آینده حرکت می‌کند کافی نیست.

۱.۵.۵ مرور ساختار LSTM

برای درک بهتر BiLSTM ابتدا ساختار LSTM را مرور می‌کنیم. در LSTM در هر گام زمانی t ، سه گیت اصلی و یک حالت حافظه وجود دارد:

گیت فراموشی (Forget Gate)

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (94)$$

این گیت تعیین می‌کند چه مقدار از حافظه قبلی حفظ شود.

گیت ورودی (Input Gate)

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (95)$$

بردار کاندید حافظه

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad (96)$$



به روزرسانی حافظه

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (97)$$

گیت خروجی

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (98)$$

حالت پنهان نهایی

$$h_t = o_t \odot \tanh(c_t) \quad (99)$$

وجود مسیر نسبتاً خطی در به روزرسانی c_t باعث می شود گرادین بتواند در دنباله های طولانی بهتر منتقل شود و مشکل محوشدگی کاهش یابد.

۲.۵.۵ ایده اصلی Bidirectional Processing

در LSTM استاندارد، پردازش فقط در یک جهت انجام می شود:

$$x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T \quad (100)$$

اما در BiLSTM دو شبکه LSTM مستقل وجود دارد:

- یک شبکه در جهت رو به جلو (Forward)
 - یک شبکه در جهت معکوس (Backward)
- شبکه رو به جلو حالت های زیر را تولید می کند:

$$\vec{h}_t = \text{LSTM}_{forward}(x_t, \vec{h}_{t-1}) \quad (101)$$

شبکه معکوس دنباله را از انتها به ابتدا پردازش می کند:

$$\overleftarrow{h}_t = \text{LSTM}_{backward}(x_t, \overleftarrow{h}_{t+1}) \quad (102)$$

در نهایت خروجی هر گام زمانی از الحاق این دو بردار به دست می آید:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (103)$$

در این رابطه نماد $[\cdot; \cdot]$ نشان دهنده الحاق برداری (Concatenation) است.



۳.۵.۵ تحلیل شهودی عملکرد

در جهت رو به جلو، مدل اطلاعات گذشته را خلاصه می‌کند. در جهت معکوس، مدل اطلاعات آینده را خلاصه می‌کند. بنابراین در هر لحظه زمانی t ، مدل به اطلاعات کامل تری از کل دنباله دسترسی دارد. اگر اندازه بردار پنهان هر جهت برابر H باشد، خروجی نهایی در هر گام زمانی دارای ابعاد زیر خواهد بود:

$$\dim(h_t) = 2H \quad (۱۰۴)$$

زیرا دو بردار با اندازه H به یکدیگر الحاق شده‌اند.

۴.۵.۵ کاربرد در دسته‌بندی دنباله (Sequence Classification)

در مسائل دسته‌بندی کل دنباله، معمولاً فقط به نمایش نهایی کل توالی نیاز داریم. در BiLSTM این نمایش معمولاً از آخرین حالت‌های پنهان هر دو جهت ساخته می‌شود:

$$h_{final} = [\vec{h}_T; \overleftarrow{h}_1] \quad (۱۰۵)$$

بردار فوق خلاصه‌ای از کل اطلاعات گذشته و آینده در دنباله است. این بردار سپس به یک لایه کاملاً متصل (Fully Connected Layer) داده می‌شود:

$$y = Wh_{final} + b \quad (۱۰۶)$$

و در نهایت با اعمال تابع فعال‌سازی مناسب (مثلاً Softmax) احتمال کلاس‌ها محاسبه می‌شود:

$$\hat{y} = \text{Softmax}(Wh_{final} + b) \quad (۱۰۷)$$

۵.۵.۵ مزایای BiLSTM

- استفاده همزمان از اطلاعات گذشته و آینده
- افزایش دقت در مسائل وابسته به زمینه (Context-Dependent)
- مناسب برای متن، گفتار و داده‌های زیستی

۶.۵.۵ هزینه محاسباتی

از آنجا که دو شبکه LSTM به طور همزمان اجرا می‌شوند، پیچیدگی محاسباتی تقریباً دو برابر یک LSTM معمولی است. اگر پیچیدگی یک LSTM برابر باشد با:



$$\mathcal{O}(TH^2) \quad (108)$$

در BiLSTM خواهیم داشت:

$$\mathcal{O}(2TH^2) \quad (109)$$

که تقریباً دو برابر است.

۷.۵.۵ جمع‌بندی مفهومی

BiLSTM را می‌توان به صورت دو مشاهده‌گر تصور کرد که یکی از ابتدا به انتها و دیگری از انتها به ابتدا دنباله را بررسی می‌کند. هر دو مشاهده‌گر برداشت خود را از دنباله ارائه می‌دهند و این دو برداشت با هم ترکیب می‌شوند تا نمایش کامل‌تری از داده ساخته شود. به همین دلیل BiLSTM در بسیاری از مسائل پیچیده‌تری نسبت به LSTM یک‌طرفه عملکرد بهتری دارد، به‌ویژه زمانی که درک هر عنصر به اطلاعات قبل و بعد از آن وابسته باشد.

۶.۵ بررسی `pad_packed_sequence` و `pack_padded_sequence`

در مسائل واقعی، دنباله‌ها معمولاً طول یکسان ندارند. برای پردازش دسته‌ای (Batch Processing) در چارچوب‌هایی مانند PyTorch، لازم است تمام دنباله‌های داخل یک بچ دارای طول یکسان باشند. به همین دلیل از تکنیک Padding استفاده می‌شود. فرض کنید سه دنباله با طول‌های متفاوت داشته باشیم:

$$S_1 = (x_1, x_2, x_3, x_4) \quad (110)$$

$$S_2 = (y_1, y_2) \quad (111)$$

$$S_3 = (z_1, z_2, z_3) \quad (112)$$

بیشترین طول برابر ۴ است، بنابراین دو دنباله دیگر را پد می‌کنیم:

$$S_2 = (y_1, y_2, 0, 0) \quad (113)$$

$$S_3 = (z_1, z_2, z_3, 0) \quad (114)$$

حال ورودی مدل یک تنسور با ابعاد زیر خواهد بود:

$$\text{Size Batch} \times \text{Length Max} \quad (115)$$

اگر اندازه بچ برابر ۳ و طول حداکثر برابر ۴ باشد:



$$3 \times 4 = 12 \quad (116)$$

اما تعداد گام‌های واقعی برابر است با:

$$4 + 2 + 3 = 9 \quad (117)$$

پس:

$$12 - 9 = 3 = \text{محاسبات هدررفته} \quad (118)$$

در دیتاست‌های بزرگ که اختلاف طول‌ها زیاد است، این اتلاف بسیار قابل توجه می‌شود.

۱.۶.۵ مشکل مهم‌تر از اتلاف محاسبات

مشکل فقط اتلاف محاسبات نیست. در حالت عادی، LSTM از روی مقادیر padding نیز عبور می‌کند و آن‌ها را مانند داده واقعی پردازش می‌کند. در نتیجه:

- گرادینان از روی گام‌های پد عبور می‌کند
- حالت پنهان تحت تأثیر صفرهای غیرواقعی قرار می‌گیرد
- مدل ممکن است الگوهای نادرست یاد بگیرد

۲.۶.۵ راه‌حل: Packed Sequence

در PyTorch می‌توان قبل از ورود داده به LSTM آن را به ساختار فشرده تبدیل کرد:

$$\text{PackedSequence} \quad (119)$$

این ساختار باعث می‌شود مدل فقط روی گام‌های واقعی محاسبه انجام دهد و گام‌های پد را نادیده بگیرد. فرض کنید طول‌ها برابر باشند با:

$$\text{lengths} = [4, 3, 2] \quad (120)$$

در حالت عادی پردازش به صورت زیر است:



گام ۱: سه عنصر
 گام ۲: سه عنصر
 گام ۳: سه عنصر
 گام ۴: سه عنصر

اما در حالت packed:

گام ۱: سه عنصر
 گام ۲: سه عنصر
 گام ۳: دو عنصر
 گام ۴: یک عنصر

در نتیجه تعداد کل محاسبات دقیقاً برابر با مجموع طول‌های واقعی خواهد بود.
 از نظر ریاضی، اگر طول‌های دنباله برابر باشند با $l_1, l_2, \dots, l_n$ ، در حالت عادی:

$$\text{Computation Total} = n \times \max(l_i) \quad (121)$$

اما در حالت packed:

$$\text{Computation Total} = \sum_{i=1}^n l_i \quad (122)$$

که بهینه‌تر است.

۳.۶.۵ چرا بعداً از `pad_packed_sequence` استفاده می‌کنیم؟

پس از عبور از LSTM، خروجی همچنان در قالب فشرده است. اگر نیاز داشته باشیم خروجی را به شکل تانسور عادی بازگردانیم (مثلاً برای Sequence Labeling) از تابع زیر استفاده می‌کنیم:

`pad_packed_sequence` (۱۲۳)

اما در مسئله دسته‌بندی کل دنباله، معمولاً فقط به حالت‌های نهایی h_n نیاز داریم و نیازی به بازسازی کامل توالی نیست.

۷.۵ لایه Embedding

۱.۷.۵ تعریف مفهومی

در مسائل متنی یا داده‌های گسسته، ورودی‌ها معمولاً به صورت شناسه‌های عددی (Integer Index) نمایش داده می‌شوند.
 برای مثال فرض کنید واژگان ما شامل ۵ کلمه باشد:



cat, dog, car, tree, house

می‌توان آن‌ها را به صورت زیر کدگذاری کرد:

→ 0 cat

→ 1 dog

→ 2 car

→ 3 tree

→ 4 house

این اعداد صرفاً شناسه هستند و هیچ معنای هندسی ندارند.
اگر این اعداد را مستقیماً وارد شبکه کنیم، مدل ممکن است تصور کند:

$$4 > 1 \quad (۱۲۴)$$

در حالی که این مقایسه بی معنا است.

۲.۷.۵ راه‌حل: نگاشت به فضای برداری

Embedding یک نگاشت قابل یادگیری است:

$$E : \{0, 1, \dots, V - 1\} \rightarrow \mathbb{R}^d \quad (۱۲۵)$$

که در آن:

• V اندازه واژگان• d بعد فضای embedding

در عمل، embedding یک ماتریس پارامتری است:

$$E \in \mathbb{R}^{V \times d} \quad (۱۲۶)$$

هر سطر این ماتریس یک بردار نمایشی برای یک توکن است.
اگر ورودی x_t برابر اندیس i باشد، خروجی embedding برابر است با:

$$x_t^{(embed)} = E_i \quad (۱۲۷)$$



۳.۷.۵ مقایسه با One-Hot

در روش One-Hot، هر کلمه به برداری با طول V تبدیل می‌شود که فقط یک مؤلفه آن برابر 1 است. برای واژگان 5 تایی:

$$\text{cat} = (1, 0, 0, 0, 0) \quad (۱۲۸)$$

مشکل این روش:

- بعد بسیار بالا
- بردارهای کاملاً پراکنده (Sparse)
- عدم نمایش شباهت معنایی
- در مقابل، embedding:
- چگال (Dense)
- کم بعد
- قابل یادگیری
- قادر به نمایش شباهت معنایی

برای مثال ممکن است پس از آموزش داشته باشیم:

$$E_{\text{cat}} = (0.2, -0.7, 1.1) \quad (۱۲۹)$$

$$E_{\text{dog}} = (0.3, -0.6, 1.0) \quad (۱۳۰)$$

فاصله این دو بردار کوچک است، بنابراین مدل متوجه شباهت معنایی آن‌ها می‌شود.

۴.۷.۵ نقش embedding در BiLSTM

LSTM با بردارهای پیوسته کار می‌کند. اگر ورودی‌ها به صورت اندیس باشند، عملیات ضرب ماتریسی معنی‌دار نخواهد بود. در نتیجه قبل از ورود به BiLSTM داریم:

$$x_t \in \mathbb{Z} \quad (۱۳۱)$$

پس از embedding:



$$x_t \in \mathbb{R}^d \quad (۱۳۲)$$

و حالا می‌توان آن را وارد رابطه زیر کرد:

$$h_t = \text{LSTM}(x_t^{(embed)}, h_{t-1}) \quad (۱۳۳)$$

۸.۵ تحلیل دقیق معماری RobustBiLSTM

در این بخش، معماری مدل RobustBiLSTM به صورت دقیق بررسی می‌شود. هدف این مدل، انجام sequence classification روی دنباله‌هایی با طول متغیر است؛ به گونه‌ای که هم اطلاعات گذشته و هم اطلاعات آینده هر دنباله در نمایش نهایی لحاظ شود. علاوه بر این، مدل طوری طراحی شده است که اثر padding را به حداقل رسانده، محاسبات غیرضروری را حذف کند و در عین حال پایداری آموزش را نیز حفظ نماید.

۱.۸.۵ هدف کلی مدل

ایده اصلی در این معماری آن است که هر دنباله ورودی ابتدا به بردارهای چگال معنایی تبدیل شود، سپس این بردارها توسط یک Bidirectional LSTM چندلایه پردازش شوند، و در نهایت یک نمایش فشرده و سطح بالا از کل دنباله برای تصمیم‌گیری دسته‌بندی تولید گردد. به صورت خلاصه، جریان داده در مدل را می‌توان به شکل زیر نوشت:

$$x \longrightarrow \text{Embedding} \longrightarrow \text{Packed BiLSTM} \longrightarrow h_{final} \longrightarrow \text{Dropout} \longrightarrow \text{Fully Connected} \longrightarrow \text{Logits} \quad (۱۳۴)$$

در این ساختار، مهم‌ترین نکته آن است که نمایش نهایی دنباله نه از تمام خروجی‌های زمانی، بلکه از حالت‌های پنهان نهایی دو جهت LSTM استخراج می‌شود.

۲.۸.۵ پارامترهای ورودی سازنده کلاس

سازنده کلاس RobustBiLSTM چند پارامتر اصلی دریافت می‌کند که هر یک نقش مشخصی در ظرفیت مدل و رفتار آن دارند.

vocab_size این پارامتر تعداد کل توکن‌های موجود در واژگان را مشخص می‌کند. اگر واژگان مدل شامل V نماد باشد، آنگاه لایه Embedding باید قادر باشد هر اندیس صحیح بین ۰ تا $V - 1$ را به یک بردار چگال نگاشت کند. در اینجا اندیس ۰ برای padding رزرو شده است.



embedding_dim این پارامتر بعد فضای embedding را تعیین می‌کند. اگر $\text{embedding_dim} = d_e$ باشد، هر توکن ورودی به یک بردار در $\mathbb{R}^{d_e}$ نگاشت می‌شود. بنابراین اگر ورودی اولیه تنها شامل اندیس توکن‌ها باشد، پس از عبور از لایه Embedding، نمایش برداری آن به صورت زیر خواهد بود:

$$x_i \in \{0, 1, \dots, V - 1\} \implies e_i \in \mathbb{R}^{d_e} \quad (۱۳۵)$$

hidden_dim این پارامتر تعداد واحدهای پنهان در هر جهت از LSTM را تعیین می‌کند. اگر $\text{hidden_dim} = d_h$ باشد، هر جهت روبه جلو و روبه عقب، در هر گام زمانی یک بردار در $\mathbb{R}^{d_h}$ تولید می‌کند. در نتیجه چون شبکه bidirectional است، نمایش ترکیبی هر گام زمانی در خروجی LSTM دارای بعد $2d_h$ خواهد بود.

num_layers این پارامتر مشخص می‌کند چند لایه LSTM روی یکدیگر قرار گرفته‌اند. اگر تعداد لایه‌ها برابر L باشد، آنگاه خروجی هر لایه به عنوان ورودی لایه بعدی استفاده می‌شود. این انباشت لایه‌ای باعث می‌شود مدل علاوه بر الگوهای ساده زمانی، روابط پیچیده‌تر و سلسله‌مراتبی را نیز یاد بگیرد.

num_classes این پارامتر تعداد کلاس‌های خروجی را تعیین می‌کند. اگر مسئله یک دسته‌بندی دودویی باشد، معمولاً $\text{num_classes} = 2$ در نظر گرفته می‌شود. در این حالت، خروجی نهایی مدل یک بردار logit با دو مؤلفه خواهد بود.

dropout این پارامتر احتمال خاموش‌سازی تصادفی بخشی از نورون‌ها را تعیین می‌کند. هدف از آن کاهش بیش‌برازش و افزایش تعمیم‌پذیری مدل است. اگر مقدار این پارامتر را p در نظر بگیریم، در مرحله آموزش هر مؤلفه با احتمال p صفر می‌شود.

۳.۸.۵ تحلیل اجزای داخلی مدل

لایه Embedding اولین مؤلفه مدل، لایه Embedding است. این لایه ماتریسی قابل یادگیری به صورت زیر دارد:

$$E \in \mathbb{R}^{V \times d_e} \quad (۱۳۶)$$

که در آن V اندازه واژگان و d_e بعد بردارهای هشده است. اگر یک توکن با اندیس i وارد مدل شود، خروجی لایه Embedding سطر i -ام این ماتریس خواهد بود:

$$e_i = E[i] \quad (۱۳۷)$$

وجود پارامتر $\text{padding_idx} = 0$ اهمیت زیادی دارد؛ زیرا باعث می‌شود بردار مربوط به توکن padding در آموزش اثرگذار نباشد و این اندیس به عنوان داده واقعی تفسیر نشود.

لایه **BiLSTM** پس از تبدیل توکن‌ها به بردارهای چگال، دنباله وارد لایه LSTM دوطرفه می‌شود. برای هر گام زمانی t ، LSTM روبه‌جلو و روبه‌عقب به ترتیب چنین عمل می‌کنند:

$$\vec{h}_t = \text{LSTM}_{\text{forward}}(e_t, \vec{h}_{t-1}, \vec{c}_{t-1}) \quad (138)$$

$$\overleftarrow{h}_t = \text{LSTM}_{\text{backward}}(e_t, \overleftarrow{h}_{t+1}, \overleftarrow{c}_{t+1}) \quad (139)$$

در نتیجه نمایش خروجی هر موقعیت زمانی برابر است با:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (140)$$

یعنی اطلاعات مربوط به گذشته و آینده هر موقعیت با یکدیگر ترکیب می‌شوند.

بلوک نهایی **Fully Connected** پس از استخراج نمایش نهایی دنباله، این بردار وارد یک بلوک fully connected چندمرحله‌ای می‌شود. ابتدا بعد ویژگی از $2d_h$ به d_h کاهش می‌یابد، سپس یک تابع فعال‌سازی ReLU اعمال می‌شود، پس از آن دوباره dropout قرار می‌گیرد، و در انتها نگاشت نهایی به فضای کلاس‌ها انجام می‌شود. این فرآیند را می‌توان به صورت زیر نوشت:

$$z_1 = W_1 h_{\text{final}} + b_1 \quad (141)$$

$$a_1 = \text{ReLU}(z_1) \quad (142)$$

$$a'_1 = \text{Dropout}(a_1) \quad (143)$$

$$\text{logits} = W_2 a'_1 + b_2 \quad (144)$$

خروجی نهایی logits هنوز احتمال نرمال‌شده نیست، بلکه ورودی خام تابع زیان CrossEntropyLoss محسوب می‌شود.

۴.۸.۵ تحلیل دقیق متد forward

متد forward سه ورودی اصلی دریافت می‌کند:

- x : دنباله‌های padding-شده در قالب یک تانسور.
- length : طول واقعی هر دنباله پیش از padding.
- hidden : حالت پنهان اولیه، که در این پیاده‌سازی اختیاری است.



شکل ورودی x اگر اندازه دسته را با B و بیشینه طول دنباله در آن دسته را با $T_{\max}$ نمایش دهیم، آنگاه ورودی x قبل از embedding دارای شکل زیر است:

$$x \in \mathbb{N}^{B \times T_{\max}} \quad (145)$$

یعنی هر سطر یک دنباله از اندیس توکن‌ها است. پس از اعمال embedding، شکل آن به صورت زیر تبدیل می‌شود:

$$x_{emb} \in \mathbb{R}^{B \times T_{\max} \times d_e} \quad (146)$$

نقش **length** بردار **length** طول واقعی هر دنباله را قبل از padding مشخص می‌کند:

$$\text{length} = [\ell_1, \ell_2, \dots, \ell_B] \quad (147)$$

که در آن ℓ_i طول نمونه i -ام است. این بردار برای حذف بخش‌های padding از محاسبات LSTM استفاده می‌شود.

۵.۸.۵ تحلیل تابع `pack_padded_sequence`

در این مدل، پس از embedding از تابع `pack_padded_sequence` استفاده می‌شود تا LSTM فقط روی بخش واقعی داده محاسبه انجام دهد و موقعیت‌های padding را نادیده بگیرد.

ورودی‌های اصلی تابع این تابع به طور مفهومی چهار ورودی مهم دارد:

- `input`: تسور دنباله‌های padding-شده.
- `lengths`: طول واقعی هر دنباله.
- `batch_first`: مشخص می‌کند ترتیب ابعاد ورودی چگونه است.
- `enforce_sorted`: مشخص می‌کند آیا دنباله‌ها باید بر اساس طول به صورت نزولی مرتب شده باشند یا خیر.

batch_first چیست؟ اگر `batch_first=True` باشد، شکل داده ورودی به صورت زیر است:

$$(B, T_{\max}, d_e) \quad (148)$$

اما اگر `batch_first=False` باشد، انتظار تابع این است که ورودی به شکل زیر باشد:

$$(T_{\max}, B, d_e) \quad (149)$$

در این پیاده‌سازی، چون از ابتدا داده‌ها در `DataLoader` به صورت `batch-first` ساخته شده‌اند، تنظیم `batch_first=True` ضروری و سازگار با بقیه کد است.



enforce_sorted چیست؟ برخی پیاده‌سازی‌ها برای افزایش کارایی، انتظار دارند دنباله‌ها پیش از ورود به `pack_padded_sequence` بر اساس طول به صورت نزولی مرتب شده باشند. اگر `enforce_sorted=True` باشد، این شرط باید رعایت شود. اما در اینجا مقدار آن `False` قرار داده شده است تا نیاز به مرتب‌سازی دستی وجود نداشته باشد. در نتیجه، حتی اگر طول‌ها به صورت نامرتب باشند، تابع به درستی عمل خواهد کرد.

مثال عددی برای **packing** فرض کنید سه دنباله در یک دسته داشته باشیم:

$$s_1 = [4, 2, 7, 5], \quad s_2 = [8, 3], \quad s_3 = [6, 1, 9] \quad (150)$$

پس از padding با صفر، ورودی به شکل زیر در می‌آید:

$$x = \begin{bmatrix} 4 & 2 & 7 & 5 \\ 8 & 3 & 0 & 0 \\ 6 & 1 & 9 & 0 \end{bmatrix} \quad (151)$$

و بردار طول‌ها برابر است با:

$$\text{lengths} = [4, 2, 3] \quad (152)$$

اگر این داده مستقیماً وارد LSTM شود، شبکه روی صفرهای انتهایی نیز محاسبه انجام می‌دهد. اما با استفاده از `pack_padded_sequence`، فقط توکن‌های واقعی پردازش می‌شوند و صفرها از گراف محاسباتی حذف می‌گردند. بنابراین هم سرعت بهتر می‌شود و هم گرادینان ناشی از نواحی padding وارد آموزش نمی‌شود.

۶.۸.۵ تحلیل خروجی LSTM: تفاوت `packed_output` و `output` و `h_n`

خروجی LSTM در این معماری به صورت زیر است:

$$\text{packed_output}, (h_n, c_n) \quad (153)$$

که هر کدام معنای متفاوتی دارند.

packed_output این متغیر شامل خروجی همه گام‌های زمانی است، اما در قالب فشرده‌شده. یعنی ساختار آن دیگر یک تنسور معمولی سه‌بعدی نیست، بلکه شیئی از نوع `PackedSequence` است. این خروجی برای مواقعی مفید است که بخواهیم از نمایش همه موقعیت‌های زمانی استفاده کنیم، مثلاً در `sequence labeling` یا `token-level prediction`.



pad_packed_sequence اگر بخواهیم packed_output را دوباره به فرم تنسور معمولی برگردانیم، از pad_packed_sequence استفاده می‌کنیم. خروجی آن دو بخش دارد:

• output: تنسور بازسازی شده با padding

• output_lengths: طول واقعی هر دنباله

اگر batch_first=True باشد، شکل output چنین خواهد بود:

$$\text{output} \in \mathbb{R}^{B \times T_{\max} \times 2d_h} \quad (154)$$

h_n متغیر h_n حالت پنهان نهایی همه لایه‌ها و همه جهت‌ها را نگه می‌دارد. شکل آن به صورت زیر است:

$$h_n \in \mathbb{R}^{(L \times D) \times B \times d_h} \quad (155)$$

که در آن L تعداد لایه‌ها و D تعداد جهت‌ها است. در BiLSTM داریم:

$$D = 2 \quad (156)$$

بنابراین شکل h_n برابر می‌شود با:

$$h_n \in \mathbb{R}^{(2L) \times B \times d_h} \quad (157)$$

۷.۸.۵ چرا در این مسئله فقط از h_n استفاده شده است؟

در مسئله دسته‌بندی دنباله، هدف نهایی برچسب‌گذاری کل دنباله است، نه تک‌تک موقعیت‌های زمانی. بنابراین ما به یک بردار خلاصه از کل دنباله نیاز داریم، نه به خروجی همه گام‌ها. دقیقاً به همین دلیل استفاده از h_n انتخابی طبیعی و مناسب است.

دلیل اول: سازگاری مستقیم با **sequence classification** در sequence classification باید از روی کل دنباله یک تصمیم نهایی گرفته شود. آخرین حالت پنهان در LSTM دقیقاً برای همین منظور مناسب است، زیرا اطلاعات تجمع‌یافته تمام دنباله را در خود نگه می‌دارد. در جهت روبه جلو، $\vec{h}_T$ خلاصه‌ای از ابتدای دنباله تا انتها را دارد و در جهت روبه عقب، $\overleftarrow{h}_1$ خلاصه‌ای از انتها تا ابتدا را حفظ می‌کند.

دلیل دوم: پرهیز از محاسبات و حافظه اضافی اگر از output یا خروجی pad_packed_sequence استفاده می‌کردیم، باید تنسور کامل همه گام‌های زمانی را نگه می‌داشتیم. این موضوع هم حافظه بیشتری مصرف می‌کند و هم نیازمند یک مرحله تصمیم‌گیری اضافی برای انتخاب یا جمع‌بندی گام‌های زمانی است. اما h_n از قبل همان خلاصه مناسب را در اختیار ما قرار می‌دهد.



دلیل سوم: عدم نیاز به بازیابی ساختار زمانی کامل در وظایفی مثل named entity recognition یا part-of-speech tagging لازم است برای هر موقعیت زمانی یک خروجی وجود داشته باشد، بنابراین output اهمیت دارد. اما در اینجا فقط یک برچسب برای کل دنباله می‌خواهیم؛ پس بازیابی خروجی کل دنباله از packed_output ضرورتی ندارد.

۸.۸.۵ چرا خروجی نهایی concatenate شده است؟

در BiLSTM، دو مسیر پردازشی وجود دارد:

- مسیر روبه جلو که دنباله را از چپ به راست می‌خواند.
- مسیر روبه عقب که دنباله را از راست به چپ می‌خواند.

در نتیجه، دو حالت پنهان نهایی مهم داریم:

$$h_{\text{forward}} = \vec{h}_T \quad (158)$$

$$h_{\text{backward}} = \overleftarrow{h}_1 \quad (159)$$

برای ساختن نمایش نهایی، این دو بردار کنار هم قرار می‌گیرند:

$$h_{\text{final}} = [h_{\text{forward}}; h_{\text{backward}}] \quad (160)$$

این الحاق باعث می‌شود مدل به صورت همزمان هم اطلاعات وابسته به گذشته و هم اطلاعات وابسته به آینده را برای تصمیم‌گیری در اختیار داشته باشد.

مثال عددی فرض کنید $\text{hidden_dim} = 4$ باشد. در این صورت:

$$h_{\text{forward}} = [0.2, -0.5, 1.1, 0.7] \quad (161)$$

$$h_{\text{backward}} = [0.9, 0.3, -0.2, 0.4] \quad (162)$$

پس از concatenate خواهیم داشت:

$$h_{\text{final}} = [0.2, -0.5, 1.1, 0.7, 0.9, 0.3, -0.2, 0.4] \quad (163)$$

در نتیجه، به جای یک بردار ۴ بعدی، یک بردار ۸ بعدی خواهیم داشت:



$$h_{final} \in \mathbb{R}^{2d_h} \quad (۱۶۴)$$

این بردار غنی‌تر از حالتی است که فقط یکی از دو جهت استفاده شود، زیرا ترکیبی از دو دیدگاه زمانی متفاوت را در خود نگه می‌دارد.

چرا جمع یا میانگین نگرفتیم؟ ممکن بود به جای concatenate از جمع یا میانگین دو بردار استفاده شود، اما در آن صورت بخشی از اطلاعات ساختاری از بین می‌رفت. الحاق باعث می‌شود هر دو بردار به صورت مستقل حفظ شوند و لایه fully connected خودش یاد بگیرد که چگونه از هر بخش استفاده کند.

۹.۸.۵ تحلیل دقیق pad_packed_sequence

اگرچه در این مدل از خروجی بازشده این تابع در تصمیم نهایی استفاده نشده است، اما بررسی آن از نظر مفهومی بسیار مهم است. این تابع یک PackedSequence را دوباره به نمایش padded tensor تبدیل می‌کند. به صورت مفهومی داریم:

$$\text{PackedSequence} \rightarrow \text{Padded Tensor} + \text{Lengths} \quad (۱۶۵)$$

پارامتر مهم این تابع نیز batch_first است. اگر این پارامتر True باشد، خروجی به صورت $(B, T_{\max}, 2d_h)$ خواهد بود. در غیر این صورت، فرم خروجی برابر $(T_{\max}, B, 2d_h)$ است. در این کد، اگرچه output محاسبه می‌شود، اما در ادامه مورد استفاده قرار نمی‌گیرد، زیرا هدف اصلی فقط استخراج نمایش فشرده نهایی دنباله است.

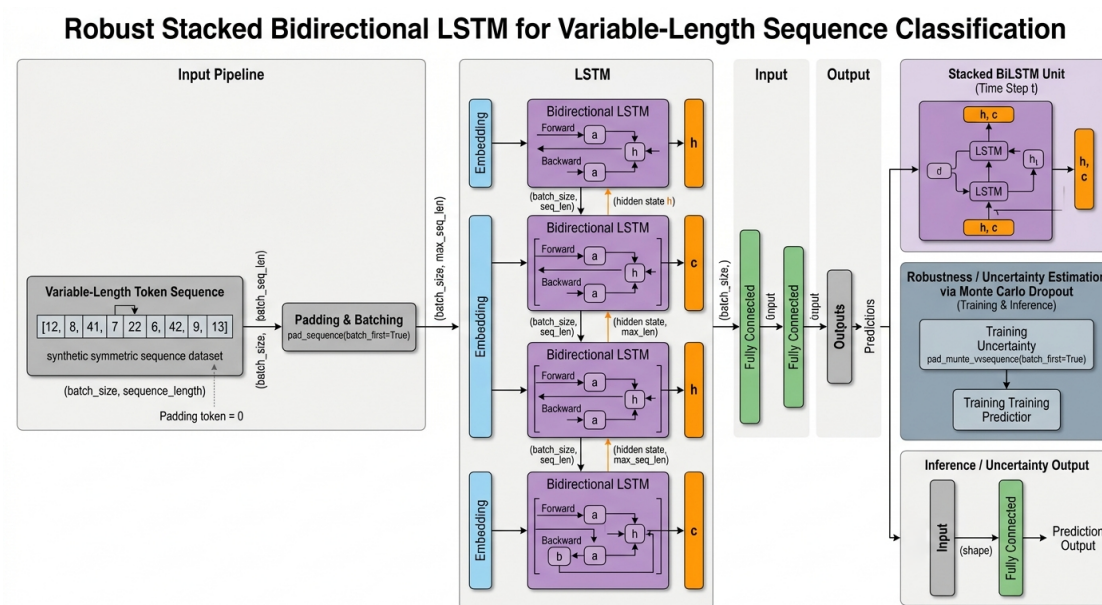
۱۰.۸.۵ تحلیل حالت‌های پنهان و سلولی

خروجی دوم LSTM شامل دو جزء h_n و c_n است:

$$(h_n, c_n) \quad (۱۶۶)$$

h_n این متغیر نمایش خروجی نهایی حافظه کوتاه‌مدت در پایان پردازش است و مستقیماً برای دسته‌بندی استفاده می‌شود.

c_n این متغیر وضعیت سلولی نهایی را نگه می‌دارد و بیشتر نقش آن حفظ و انتقال حافظه بلندمدت درون ساختار LSTM است. در این مدل، برای مرحله دسته‌بندی از c_n استفاده نشده است، زیرا خروجی مورد نیاز ما همان نمایش خلاصه‌شده در h_n است.



شکل ۴: ساختار کلی مدل RobustBiLSTM

۱۱.۸.۵ ساختار بلوک دیاگرامی مدل

همان‌طور که در شکل ۴ مشاهده می‌شود:

۱. ورودی: داده‌ها به صورت خام وارد شده و پس از padding به صورت یک تانسور منظم در می‌آیند.
۲. پردازش: لایه‌های Stacked BiLSTM به صورت متوالی ویژگی‌ها را استخراج می‌کنند.
۳. خروجی: بردار الحاق شده وارد بلوک Fully Connected شده و در نهایت کلاس نهایی پیش‌بینی می‌شود.

۹.۵ تحلیل دیتاست: تولید، ساختار و توزیع نمونه‌ها

در این بخش، به تشریح جامع دیتاست مصنوعی مورد استفاده برای آموزش و ارزیابی مدل RobustBiLSTM می‌پردازیم. این دیتاست به گونه‌ای طراحی شده است که چالش‌های کلیدی مسائل Sequence Classification با طول متغیر را شبیه‌سازی کند و دلیل مناسب بودن آن برای معماری BiLSTM را مورد بررسی قرار می‌دهیم.

۱.۹.۵ نحوه تولید دیتاست

دیتاست با استفاده از تابع `generate_balanced_symmetric_dataset` تولید شده است. هدف اصلی این تابع، ایجاد داده‌هایی برای مسئله دسته‌بندی شباهت دنباله‌ها است. هر دنباله از سه بخش اصلی تشکیل شده است:

$$\text{Sequence} = [\text{Left Segment} \mid \text{Noise Segment} \mid \text{Right Segment}] \quad (۱۶۷)$$

ساختار نمونه‌های مثبت و منفی

- نمونه‌های مثبت (برچسب ۱): در این دسته، بخش `Right Segment` تقریباً معکوس `Left Segment` است. یک اغتشاش کوچک نیز به صورت کنترل‌شده به بخش راست اضافه می‌شود تا مسئله کمی واقع‌گرایانه‌تر شود. این ساختار به مدل کمک می‌کند تا تقارن یا شباهت ترتیبی را یاد بگیرد.
- نمونه‌های منفی (برچسب ۰): در این دسته، بخش `Right Segment` به صورت کاملاً تصادفی و بدون وابستگی به `Left Segment` تولید می‌شود. این امر باعث عدم تقارن و شباهت پایین بین دو نیمه می‌شود.

پارامترهای تولید دیتاست تولید دیتاست با پارامترهای زیر انجام شده است:

- `num_samples = 10000`: تعداد کل نمونه‌های تولید شده.
- `positive_ratio = 0.6`: ۶۰٪ نمونه‌ها مثبت و ۴۰٪ منفی هستند.
- `vocab_size = 60`: اندازه واژگان.
- `max_half_len = 7`, `min_half_len = 3`: طول بخش‌های چپ و راست (بدون احتساب نویز) بین ۳ تا ۷ توکن است.
- `noise_len = 1`: یک توکن نویز بین بخش چپ و راست قرار می‌گیرد.

۲.۹.۵ نمونه‌هایی از دیتاست

برای درک بهتر ساختار داده‌ها، سه نمونه اول پس از تولید و قبل از `padding` به صورت زیر هستند:

۱. نمونه ۱ (برچسب ۱):

- Left part: [36, 16, 41, 46, 26]



- Noise token: [19]
- Right part: [25, 47, 41, 15, 36]
- Full sequence: [36, 16, 41, 46, 26, 19, 25, 47, 41, 15, 36]

۲. نمونه ۲ (برچسب ۱):

- Left part: [26, 54, 55, 15, 40, 19]
- Noise token: [28]
- Right part: [19, 41, 15, 56, 54, 25]
- Full sequence: [26, 54, 55, 15, 40, 19, 28, 19, 41, 15, 56, 54, 25]

۳. نمونه ۳ (برچسب ۰):

- Left part: [1, 16, 51, 23, 33, 44, 56]
- Noise token: [15]
- Right part: [9, 22, 45, 23, 44, 24, 8]
- Full sequence: [1, 16, 51, 23, 33, 44, 56, 15, 9, 22, 45, 23, 44, 24, 8]

این نمونه‌ها نشان می‌دهند که مدل باید بتواند الگوهای زمانی را در دو جهت تشخیص دهد و همچنین طول متغیر دنباله‌ها را مدیریت کند.

۳.۹.۵ چرا این دیتاست برای BiLSTM مناسب است؟

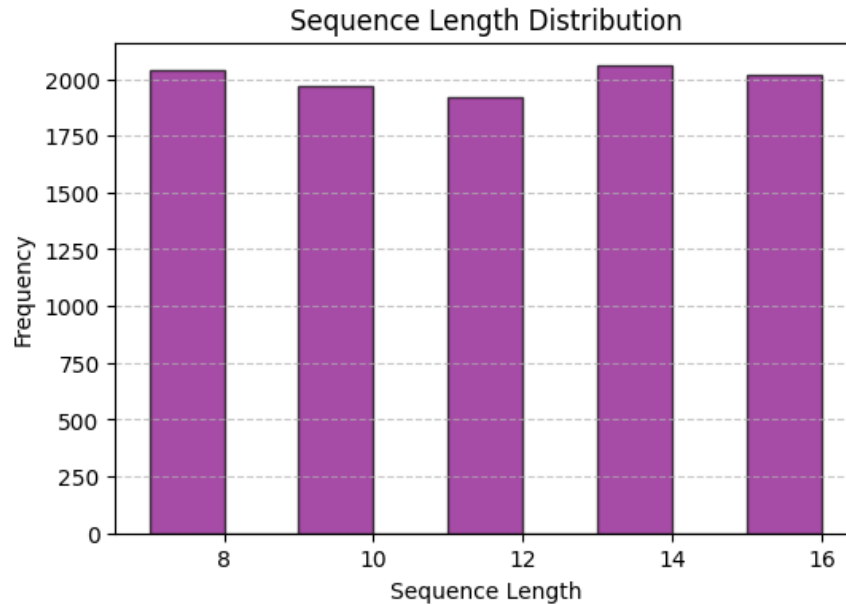
این دیتاست به دلایل زیر برای معماری BiLSTM بسیار مناسب است:

- وابستگی ترتیبی و دوطرفه: تشخیص شباهت بین نیمه چپ و راست نیازمند درک توالی است. علاوه بر این، در نمونه‌های مثبت، نیاز به تشخیص وارونگی (ترتیب معکوس) وجود دارد که BiLSTM به خوبی از پس آن برمی‌آید. LSTM یک طرفه فقط قادر به دیدن گذشته است، اما BiLSTM با داشتن دو جهت پردازش، هم اطلاعات گذشته و هم اطلاعات آینده را برای درک رابطه بین دو نیمه دنباله استفاده می‌کند.
- یادگیری نمایش معنایی: مدل باید یاد بگیرد که شباهت معنایی بین بخش چپ و راست (که به صورت دنباله توکن‌ها نمایش داده شده) را استخراج کند. این کار از طریق لایه Embedding و سپس پردازش توسط BiLSTM انجام می‌شود.
- مدیریت طول متغیر: همانطور که گفته شد، دنباله‌ها طول متغیر دارند. BiLSTM با ترکیب pack_padded_sequence این چالش را به خوبی مدیریت می‌کند.

۴.۹.۵ تحلیل توزیع دیتاست

نمودارهای ارائه شده، اطلاعات ارزشمندی درباره توزیع نمونه‌ها و طول دنباله‌ها ارائه می‌دهند.

توزیع طول دنباله‌ها (Sequence Length Distribution) در نمودار اول (۵)، توزیع طول دنباله‌ها در دیتاست مشاهده می‌شود. همانطور که در نمودار ۵ مشاهده می‌شود، طول دنباله‌ها در بازه تقریبی ۸ تا ۱۶ قرار دارند و توزیع نسبتاً یکنواختی بین این طول‌ها وجود دارد.



شکل ۵: توزیع طول دنباله‌ها در دیتاست

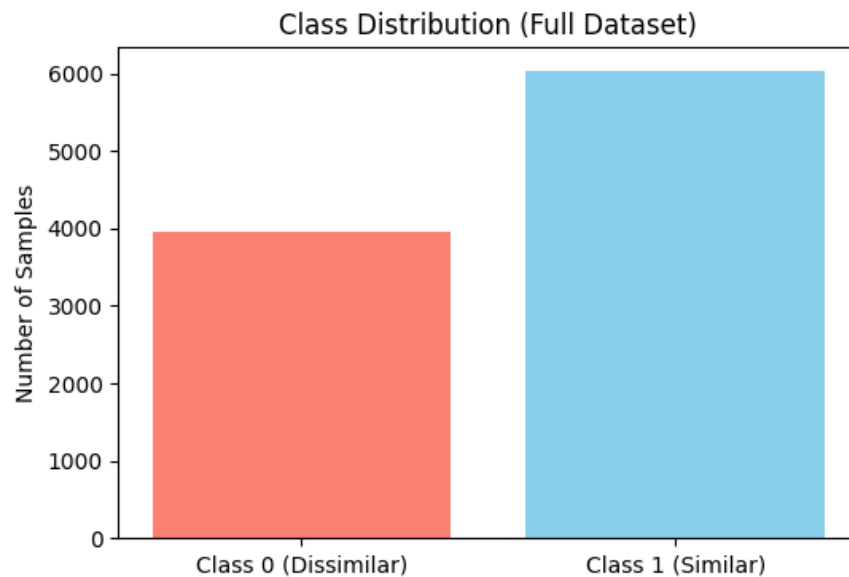
دارد. این موضوع نشان می‌دهد که مدل با دنباله‌هایی با طول‌های متفاوت سروکار دارد و این نیازمند مکانیزم‌های مدیریت طول متغیر (مانند padding و packing) است. پراکندگی طول‌ها در این بازه، تمرین خوبی برای مدل BiLSTM خواهد بود.

توزیع کلاس‌ها در کل دیتاست (Class Distribution (Full Dataset)) نمودار دوم (۶) توزیع کلی کلاس‌ها را در کل دیتاست نشان می‌دهد. نمودار ۶ نشان می‌دهد که دیتاست تولید شده نامتوازن نیست، اما کمی تمایل به سمت کلاس ۱ (نمونه‌های مشابه) دارد (۶۰۰۰ نمونه در مقابل ۴۰۰۰ نمونه). نسبت کلاس ۱ حدود ۶۰٪ است که با پارامتر positive_ratio تولید دیتاست همخوانی دارد. این نامتوازنی کم، معمولاً برای مدل‌ها مشکلی ایجاد نمی‌کند و حتی می‌تواند به یادگیری بهتر الگوهای کلاس پرتعدادتر کمک کند.

توزیع کلاس‌ها در مجموعه‌های آموزش و اعتبارسنجی (Train vs Validation Class Distribution) نمودار سوم (۷) مقایسه توزیع کلاس‌ها در بخش آموزش و اعتبارسنجی را نشان می‌دهد. نمودار ۷ مقایسه توزیع کلاس‌ها بین بخش آموزش (نارنجی) و اعتبارسنجی (سبز) را نشان می‌دهد. روش train_test_split با پارامتر stratify=labels تضمین کرده است که نسبت کلاس‌ها در هر دو مجموعه تقریباً حفظ شود. این امر برای اطمینان از ارزیابی عادلانه مدل بسیار مهم است. همانطور که دیده می‌شود، مجموعه آموزش و اعتبارسنجی هر دو دارای نسبت مشابهی از کلاس ۰ و کلاس ۱ هستند، که نشان‌دهنده حفظ ساختار دیتاست در تقسیم‌بندی است.

۵.۹.۵ جمع‌بندی تحلیل دیتاست

دیتاست تولیدی، نمونه‌ای مناسب از مسائل Sequence Classification با چالش‌های کلیدی مانند طول متغیر و نیاز به درک وابستگی‌های ترتیبی در دو جهت است. طراحی این دیتاست به خوبی با معماری BiLSTM و تکنیک‌های مدیریت داده‌های ترتیبی هماهنگ است.



شکل ۶: توزیع کلاس‌ها در کل دیتاست

توزیع تقریباً متعادل کلاس‌ها و طول دنباله‌ها، آموزش و ارزیابی مدل RobustBiLSTM را تسهیل می‌کند و اطمینان می‌دهد که مدل بتواند الگوهای پیچیده را به خوبی فراگیرد.

۱۰.۵ آماده‌سازی داده و پایداری گرادین‌ها در LSTM

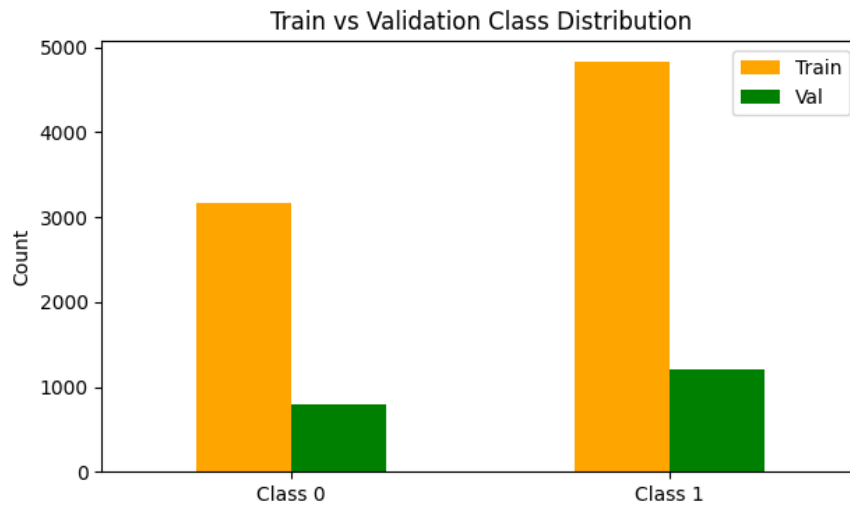
در این بخش، به تشریح دقیق فرآیند آماده‌سازی داده‌ها برای مدل RobustBiLSTM و همچنین تحلیل عمیق تابع `init_orthogonal_lstm` و اهمیت مقداردهی اولیه متعامد در بهبود جریان گرادین‌ها در طول انتشار به عقب (BPTT) می‌پردازیم.

۱.۱۰.۵ مدیریت داده‌های با طول متغیر: `SequenceDataset` و `collate_fn`

همانطور که پیشتر اشاره شد، دنباله‌های ورودی به مدل دارای طول‌های متغیر هستند. شبکه‌های عصبی، به خصوص در زمان پردازش دسته‌ای (batch processing)، نیاز به ورودی‌هایی با ابعاد ثابت دارند. کلاس `SequenceDataset` و تابع `collate_fn` با همکاری یکدیگر این مشکل را حل می‌کنند.

کلاس `SequenceDataset` این کلاس به عنوان یک واسط استاندارد عمل می‌کند که داده‌ها را به شکلی قابل فهم برای `DataLoader` در PyTorch سازماندهی می‌کند. وظایف اصلی آن عبارتند از:

- `__len__`: این متد تعداد کل نمونه‌های موجود در دیتاست را برمی‌گرداند. `DataLoader` از این مقدار برای تعیین تعداد دفعات تکرار در هر دوره (epoch) استفاده می‌کند.
- `__getitem__(self, index)`: این متد با دریافت یک اندیس، یک نمونه داده (شامل دنباله و برچسب مربوطه) را از دیتاست بازیابی کرده و برمی‌گرداند. این تابع اساس دسترسی به داده‌های منفرد را فراهم می‌کند.



شکل ۷: مقایسه توزیع کلاس‌ها بین مجموعه‌های آموزش و اعتبارسنجی

به عبارت دیگر، `SequenceDataset` امکان دسترسی به هر جفت (دنباله، برچسب) را به صورت مستقل فراهم می‌کند، اما هنوز مشکل طول متغیر دنباله‌ها در زمان ایجاد دسته‌های آموزشی (batches) پابرجاست.

تابع `collate_fn` این تابع نقش کلیدی در تجمیع نمونه‌های بازیابی شده توسط `SequenceDataset` به یک دسته (batch) منسجم و قابل پردازش توسط مدل دارد. `collate_fn` توسط `DataLoader` برای هر دسته فراخوانی می‌شود و وظایف زیر را انجام می‌دهد:

۱. جمع‌آوری نمونه‌ها: ابتدا، لیستی از نمونه‌ها (که هر کدام شامل یک دنباله و یک برچسب هستند) از `SequenceDataset` جمع‌آوری می‌شود.

۲. جداسازی دنباله‌ها و برچسب‌ها: دنباله‌ها و برچسب‌ها از یکدیگر جدا می‌شوند.

۳. محاسبه طول واقعی دنباله‌ها: برای هر دنباله در دسته، طول واقعی آن (یعنی تعداد توکن‌های غیر از توکن‌های padding) محاسبه می‌شود. این اطلاعات، که به صورت یک لیست بازگردانده می‌شوند، برای عملیات بعدی مانند `pack_padded_sequence` حیاتی هستند.

۴. **Padding** دنباله‌ها: مهم‌ترین وظیفه `collate_fn`، اطمینان از یکسان بودن طول تمام دنباله‌ها در یک دسته است. این کار با استفاده از تابع `torch.nn.utils.rnn.pad_sequence` انجام می‌شود. این تابع دنباله‌های با طول‌های متفاوت را با افزودن توکن‌های padding (معمولاً با مقدار ۰) تا رسیدن به طول بیشینه دنباله در آن دسته، تکمیل می‌کند.

- `batch_first=True`: این پارامتر تضمین می‌کند که ابعاد دسته خروجی به صورت $(B, T_{\max}, D)$ باشد، که B اندازه دسته، $T_{\max}$ بیشینه طول دنباله در دسته، و D بعد ویژگی (مانند ابعاد embedding) است. این فرمت با ساختار ورودی بسیاری از مدل‌های PyTorch مطابقت دارد.

- `padding_value=0`: این مقدار مشخص می‌کند که توکن padding چه عددی خواهد بود. معمولاً اندیس ۰ برای توکن padding در لایه Embedding در نظر گرفته می‌شود.



بدون `collate_fn`، `DataLoader` قادر به ایجاد دسته‌هایی با ابعاد ثابت از دنباله‌هایی با طول متغیر نخواهد بود و پردازش مدل با خطا مواجه می‌شود. `collate_fn` با استانداردسازی طول دنباله‌ها، امکان استفاده کارآمد از LSTM و سایر لایه‌های مبتنی بر توالی را فراهم می‌سازد.

۲.۱۰.۵ تحلیل عمیق مقداردهی اولیه متعامد در LSTM

اهمیت مقداردهی اولیه وزن‌ها مقداردهی اولیه مناسب وزن‌های یک شبکه عصبی، گامی حیاتی در موفقیت فرآیند آموزش است. این امر به ویژه در شبکه‌های عمیق و همچنین شبکه‌های بازگشتی مانند LSTM که با مشکلاتی نظیر ناپایداری گرادیان‌ها (محوشدگی یا انفجار) مواجه هستند، اهمیت دوچندان پیدا می‌کند.

تابع `init_orthogonal_lstm`: جزئیات و مثال مفهومی تابع `init_orthogonal_lstm` مسئول مقداردهی اولیه ماتریس‌های وزن در سلول LSTM به گونه‌ای است که این ماتریس‌ها خاصیت تعامد را داشته باشند. خاصیت تعامد: یک ماتریس W را متعامد گوئیم هرگاه شرط زیر برقرار باشد:

$$WW^T = W^TW = I \quad (۱۶۸)$$

که در آن W^T ترانپوز ماتریس W و I ماتریس همانی است. این خاصیت به این معناست که تبدیل خطی که توسط ماتریس W انجام می‌شود، طول هر بردار را حفظ می‌کند. به عبارت دیگر، اگر v یک بردار باشد، آنگاه $\|Wv\| = \|v\|$. مثال مفهومی: یک ماتریس چرخش در صفحه دوبعدی نمونه‌ای از ماتریس متعامد است:

$$W = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \quad (۱۶۹)$$

اگر این ماتریس را به بردار $v = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ اعمال کنیم، خواهیم داشت: $Wv = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}$. طول این بردار همچنان ۱ است ($\sqrt{\cos^2(\theta) + \sin^2(\theta)} = 1$). که برابر با طول v است. این ماتریس صرفاً بردار را به اندازه زاویه θ دوران می‌دهد.

کاربرد در LSTM: در سلول LSTM، ماتریس‌های وزن متعددی وجود دارند که تبدیلات خطی را بر روی ورودی فعلی و حالت پنهان قبلی اعمال می‌کنند. این ماتریس‌ها شامل وزن‌های مربوط به دروازه ورودی (W_{ii}, W_{hi})، دروازه فراموشی (W_{if}, W_{hf})، دروازه خروجی (W_{io}, W_{ho}) و دروازه حالت سلول (W_{ic}, W_{hc}) هستند. تابع `init_orthogonal_lstm` این ماتریس‌ها (یا زیرمجموعه‌ای از آن‌ها، معمولاً ماتریس‌های "recurrent" مانند $W_{hi}, W_{hf}, W_{ho}, W_{hc}$) را به صورت متعامد مقداردهی اولیه می‌کند.

چرا مقداردهی اولیه متعامد به پایداری گرادیان در BPTT کمک می‌کند؟ فرآیند انتشار گرادیان در طول زمان در شبکه‌های بازگشتی (BPTT) شامل ضرب مکرر ماتریس‌های وزن است. فرض کنید گرادیان نسبت به حالت پنهان در زمان t به حالت پنهان در زمان $t-1$ به صورت $\frac{\partial h_t}{\partial h_{t-1}}$ محاسبه شود. اگر این نسبت برابر با یک ماتریس W باشد، آنگاه گرادیان نسبت به حالت پنهان در زمان $t-k$ به حالت پنهان در زمان t برابر با ضرب مکرر W خواهد بود: $\frac{\partial h_t}{\partial h_{t-k}} = W^k$.

• انفجار گرادیان (Gradient Explosion): اگر مقادیر ویژه (eigenvalues) ماتریس W بزرگتر از ۱ باشند (یعنی $|\lambda| > 1$)، آنگاه با افزایش k ، $|\lambda^k|$ به صورت نمایی رشد می‌کند. این منجر به انفجار گرادیان می‌شود، زیرا گرادیان‌ها بسیار بزرگ شده و باعث تغییرات ناگهانی و نامطلوب در وزن‌ها می‌گردند که آموزش را بی‌ثبات می‌کند.



• محوشدگی گرادیان (Vanishing Gradient): اگر مقادیر ویژه ماتریس W کوچکتر از ۱ باشند (یعنی $|\lambda| < 1$)، آنگاه با افزایش k ، $|\lambda^k|$ به صورت نمایی به سمت صفر میل می‌کند. این باعث می‌شود گرادیان‌ها بسیار کوچک شده و عملاً لایه‌های اولیه شبکه (یا اطلاعات مربوط به زمان‌های دورتر) نتوانند سیگنال گرادیان کافی برای به‌روزرسانی وزن‌ها دریافت کنند، که مانع یادگیری وابستگی‌های بلندمدت می‌شود.

نقش تعامد: ماتریس‌های متعامد دارای مقادیر ویژه با اندازه دقیقاً ۱ هستند ($|\lambda| = 1$). این بدان معناست که ضرب مکرر یک ماتریس متعامد در خودش، اندازه مقادیر ویژه را تغییر نمی‌دهد. در نتیجه:

$$\|W^k v\| = \|v\| \quad (170)$$

این خاصیت باعث می‌شود که انتشار گرادیان‌ها از طریق ماتریس‌های وزن متعامد، از نظر مقیاس (norm) نسبتاً پایدار بماند. این امر از بروز شدید انفجار یا محوشدگی گرادیان جلوگیری کرده و به LSTM اجازه می‌دهد تا اطلاعات و گرادیان‌ها را برای مدت طولانی‌تری در طول دنباله حفظ کند. این امر به طور خاص برای یادگیری وابستگی‌های بلندمدت (long-term dependencies) که LSTM برای آن طراحی شده، بسیار حیاتی است.

ورودی‌ها و خروجی‌ها تابع `init_orthogonal_lstm` معمولاً پارامترهای ماتریسی یک ماژول LSTM را به عنوان ورودی دریافت می‌کند. این پارامترها شامل ماتریس‌های وزن برای تبدیل ورودی فعلی و حالت پنهان قبلی به حالت‌های جدید و گیت‌ها هستند. تابع، این ماتریس‌ها را با مقادیر متعامد جایگزین کرده و آن‌ها را به ماژول LSTM برمی‌گرداند.

پیامدهای عدم استفاده از مقداردهی اولیه متعامد همانطور که ذکر شد، عدم استفاده از این روش می‌تواند منجر به:

- آموزش ناپایدار: گرادیان‌های منفجر شونده یا محوشونده، فرآیند بهینه‌سازی را بسیار دشوار و ناپایدار می‌کنند.
- همگرایی کند: ممکن است مدل بسیار کند همگرا شود یا در یک نقطه بهینه محلی نامناسب گیر کند.
- عملکرد ضعیف در وظایف نیازمند حافظه بلندمدت: توانایی LSTM در به خاطر سپردن اطلاعات از گذشته‌های دور به شدت تحت تأثیر قرار می‌گیرد.

بنابراین، مقداردهی اولیه متعامد یک استراتژی مؤثر برای بهبود آموزش و عملکرد شبکه‌های LSTM، به خصوص در مواجهه با دنباله‌های طولانی و وظایفی است که به حافظه بلندمدت نیاز دارند.

۱۱.۵ مدیریت حافظه و جلوگیری از ناپایداری گرادیان‌ها

در این بخش به دو موضوع مهم در آموزش شبکه‌های عصبی عمیق می‌پردازیم: مدیریت حافظه با استفاده از تابع `detach_hidden` و روش برش گرادیان (Gradient Clipping) برای جلوگیری از ناپایداری در فرآیند آموزش.

۱.۱۱.۵ تابع `detach_hidden` و مدیریت حافظه

عملکرد تابع `detach_hidden` وظیفه اصلی این تابع، جدا کردن (detaching) حالت‌های پنهان (hidden states) از گراف محاسباتی (computational graph) فعلی است. در PyTorch، زمانی که عملیاتی روی تانسورها انجام می‌شود، یک گراف محاسباتی ساخته می‌شود که مسیر محاسبه گرادیان‌ها را در طول انتشار به عقب (backpropagation) تعیین می‌کند.

- **hidden**: این پارامتر معمولاً یک تاپل (tuple) شامل تانسورهای است که حالت پنهان سلول LSTM یا RNN را در انتهای یک دنباله یا یک بخش از دنباله نشان می‌دهند (معمولاً شامل حالت پنهان h_n و حالت سلول c_n برای LSTM).
- **h.detach()**: متد detach() یک تانسور جدید ایجاد می‌کند که از گراف محاسباتی فعلی جدا شده است. تانسور جدید همان مقادیر تانسور اصلی را دارد، اما هیچ تاریخچه‌ای از محاسباتی که منجر به تولید آن شده، ندارد. این بدان معناست که گرادین‌ها از طریق این تانسور جدا شده به عقب منتشر نخواهند شد.
- **tuple(...)**: چون حالت پنهان LSTM معمولاً شامل دو بخش (حالت پنهان h_n و حالت سلول c_n) است، تابع این دو را در یک تاپل دریافت کرده و هر کدام را جداگانه با detach() پردازش می‌کند تا یک تاپل جدید از تانسورهای جدا شده را برگرداند.

مدیریت حافظه و پشته (Stack) در طول فرآیند انتشار به عقب، PyTorch باید تاریخچه تمام محاسبات را برای محاسبه گرادین‌ها ذخیره کند. این تاریخچه در حافظه RAM نگهداری می‌شود و می‌تواند حجم قابل توجهی را اشغال کند، به خصوص در شبکه‌های عمیق یا هنگام پردازش دنباله‌های طولانی.

تابع detach_hidden به طور مستقیم حافظه RAM را "مدیریت" نمی‌کند، بلکه با جلوگیری از ذخیره تاریخچه محاسبات برای حالت‌های پنهان، به طور غیرمستقیم به **کاهش مصرف حافظه** کمک می‌کند. وقتی حالت‌های پنهان با detach() جدا می‌شوند، PyTorch نیازی به نگهداری اطلاعات مربوط به نحوه محاسبه آن‌ها برای انتشار گرادین ندارد. این امر به خصوص در سناریوهایی که حالت پنهان از یک مرحله به مرحله بعدی منتقل می‌شود (مانند پردازش دنباله‌های بسیار طولانی در حالت ناپیوسته یا زمانی که نیاز به پردازش دسته‌هایی داریم که از حالت پنهان مرحله قبل استفاده می‌کنند) مفید است.

از نظر "پشته" (stack) در مفهوم سیستم عامل یا ساختار داده، این تابع مستقیماً آن را مدیریت نمی‌کند. پشته (stack) در علوم کامپیوتر معمولاً به حافظه موقتی برای ذخیره فراخوانی‌های تابع، متغیرهای محلی و آدرس‌های بازگشت اشاره دارد. تابع detach_hidden محلی خود (hidden) را پردازش می‌کند و یک نتیجه (تاپل) را برمی‌گرداند. این فرآیند به طور معمول در پشته فراخوانی (call stack) رخ می‌دهد، اما خود تابع تأثیری بر اندازه کلی پشته سیستم ندارد، بلکه صرفاً از حافظه RAM برای نگهداری تانسورها و گراف محاسباتی استفاده می‌کند.

چرا این تابع در پروژه فعلی ما (RobustBiLSTM) به آن نیاز نیست؟ دلیل اصلی عدم نیاز به تابع detach_hidden در پیاده‌سازی فعلی ما این است که نحوه استفاده از LSTM در این پروژه به گونه‌ای است که نیازی به حفظ و انتقال حالت پنهان بین دسته‌های آموزشی مجزا (به صورت پیوسته) نیست.

- پردازش دسته‌ای مستقل: در پیاده‌سازی رایج PyTorch برای مسائل دسته‌بندی دنباله، هر دسته آموزشی (batch) به طور مستقل پردازش می‌شود. تابع forward مدل، دنباله‌های موجود در یک دسته را دریافت کرده، آن‌ها را پردازش می‌کند و خروجی نهایی (مثلاً logit ها) و حالت پنهان نهایی آن دسته را برمی‌گرداند.

- عدم نیاز به حالت پنهان قبلی برای دسته بعدی: برای دسته آموزشی بعدی، معمولاً حالت پنهان اولیه به صورت "صفر" مقداردهی می‌شود (یا از طریق پارامتر hidden در تابع forward که مقدارش None است، بازنشانی می‌شود). این بدان معناست که هیچ وابستگی محاسباتی بین پردازش یک دسته و دسته بعدی وجود ندارد.

- گراف محاسباتی برای هر دسته: گراف محاسباتی فقط برای محاسبات درون یک دسته ساخته می‌شود. پس از اتمام پردازش آن دسته و محاسبه گرادین‌ها (در صورت نیاز)، گراف برای آن دسته دور ریخته می‌شود. بنابراین، نیازی به "جدا کردن" حالت پنهان

برای جلوگیری از مصرف بیش از حد حافظه در طول عمر مدل نیست، زیرا این حالت‌ها برای دسته‌های بعدی استفاده نمی‌شوند یا به طور کامل بازنشانی می‌شوند.

تحلیل دقیق‌تر: چرا به آن نیاز نیست؟ در مدل RobustBiLSTM ما، همانطور که در توضیحات تابع forward مدل دیده می‌شود:

- تابع forward ورودی‌های x ، length، hidden را می‌گیرد.
- اگر hidden مقدار None باشد، حالت پنهان اولیه به صورت صفر مقداردهی می‌شود: `self.lstm.flatten_parameters()` یا مقداردهی دستی داخل forward که به طور پیش‌فرض صفر است.
- پس از پردازش دنباله‌ها با `pack_padded_sequence` و سپس `lstm`، خروجی‌ها (h_n, c_n) ، `output` به دست می‌آیند.
- این خروجی‌ها (h_n, c_n) برای مرحله بعدی آموزش (اگر مثلاً بخواهیم حالت پنهان را برای دنباله‌های بسیار طولانی حفظ کنیم) یا برای محاسبات بعدی در همان دسته استفاده می‌شوند.
- اما نکته کلیدی این است که این h_n, c_n بازگشتی به ابتدای تابع forward برای دسته بعدی ندارند. DataLoader دسته بعدی را با حالت پنهان اولیه صفر بارگذاری می‌کند.

بنابراین، هیچ مسیر گرادیانی (gradient path) وجود ندارد که از حالت پنهان یک دسته به حالت پنهان دسته بعدی منتقل شود. این امر باعث می‌شود که نیازی به قطع کردن (detaching) حالت پنهان نباشد، زیرا گراف محاسباتی به طور طبیعی پس از پردازش هر دسته از بین می‌رود و حافظه مربوطه آزاد می‌شود. تابع `detach_hidden` بیشتر در سناریوهایی مانند پردازش تکراری (unrolling) یک دنباله طولانی در چند مرحله زمانی یا در شبکه‌هایی که حالت پنهان به صورت صریح بین دسته‌های متوالی حفظ می‌شود، کاربرد دارد.

۲.۱۱.۵ برش گرادیان (Gradient Clipping)

توضیح دقیق برش گرادیان برش گرادیان یک تکنیک تنظیم (regularization technique) است که برای جلوگیری از انفجار گرادیان (exploding gradients) در طول فرآیند آموزش شبکه‌های عصبی عمیق، به ویژه شبکه‌های بازگشتی، استفاده می‌شود. این تکنیک به سادگی با محدود کردن (clipping) اندازه (norm) بردار گرادیان، به طوری که از یک حد آستانه (threshold) مشخص تجاوز نکند، عمل می‌کند.

چرا از برش گرادیان استفاده می‌کنیم؟ همانطور که در بخش مقداردهی اولیه متعامد اشاره شد، انفجار گرادیان می‌تواند فرآیند آموزش را کاملاً مختل کند. زمانی که گرادیان‌ها بسیار بزرگ می‌شوند:

- به‌روزرسانی‌های نامطلوب: وزن‌های شبکه با مقادیر بسیار بزرگ به‌روزرسانی می‌شوند که می‌تواند مدل را از نقطه بهینه دور کرده و باعث واگرایی (divergence) آموزش شود.
- عدم پایداری عددی: محاسبات ممکن است با اعداد بسیار بزرگ مواجه شده و منجر به خطاهای عددی (numerical instability) شوند.

برش گرادیان با قرار دادن یک سقف بر اندازه گرادیان‌ها، این مشکل را برطرف می‌کند.

نحوه عملکرد برش گرادیان فرض کنید ∇W بردار گرادیان نسبت به وزن‌های W باشد. اندازه (norm) این بردار معمولاً با استفاده از نرم اقلیدسی (L_2 norm) محاسبه می‌شود:

$$\|\nabla W\|_2 = \sqrt{\sum_i \left(\frac{\partial L}{\partial W_i}\right)^2} \quad (171)$$

که L تابع هزینه (loss function) است.

فرآیند برش گرادیان به صورت زیر است:

۱. محاسبه گرادیان‌ها برای تمام پارامترهای مدل.

۲. محاسبه اندازه (norm) بردار گرادیان کل (که معمولاً مجموع گرادیان‌های تمام پارامترها است).

۳. مقایسه با حد آستانه: اگر اندازه گرادیان از یک حد آستانه از پیش تعیین شده (max_norm) بیشتر باشد:

۴. مقیاس‌بندی گرادیان: بردار گرادیان را مقیاس‌بندی می‌کنیم تا اندازه آن برابر با max_norm شود. این کار با ضرب گرادیان در نسبت $max_norm / \|\nabla W\|_2$ انجام می‌شود:

$$\nabla W_{clipped} = \nabla W \times \frac{max_norm}{\|\nabla W\|_2} \quad (172)$$

۵. اعمال گرادیان برش خورده: سپس از گرادیان برش خورده ($\nabla W_{clipped}$) برای به‌روزرسانی وزن‌ها استفاده می‌شود (مثلاً با الگوریتم Adam یا SGD).

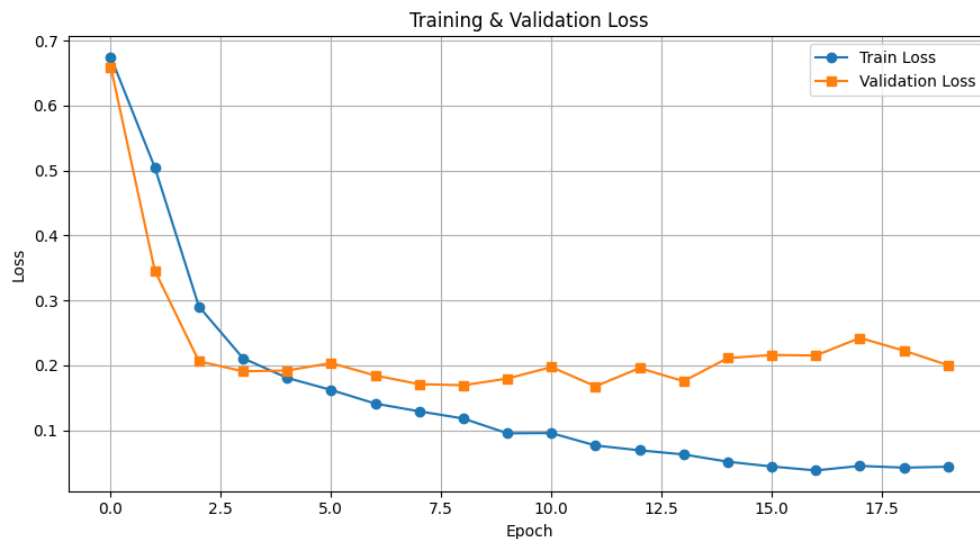
اگر اندازه گرادیان کمتر یا مساوی max_norm باشد، گرادیان بدون تغییر باقی می‌ماند.

کامک برش گرادیان به فرآیند آموزش

- افزایش پایداری آموزش: اصلی‌ترین مزیت آن، جلوگیری از واگرایی آموزش به دلیل انفجار گرادیان است.
- بهبود همگرایی: با حفظ گرادیان‌ها در یک محدوده معقول، فرآیند همگرایی به سمت نقطه بهینه پایدارتر و قابل پیش‌بینی‌تر می‌شود.
- مناسب برای RNN‌ها: شبکه‌های بازگشتی به دلیل ماهیت تکراری خود، بیشتر در معرض انفجار گرادیان هستند، بنابراین برش گرادیان یک ابزار استاندارد در آموزش آن‌ها محسوب می‌شود.

۱۲.۵ تحلیل نتایج آموزش و اعتبارسنجی

نمودار زیر روند تغییرات خطای آموزش و اعتبارسنجی را در طول دوره‌های آموزش نشان می‌دهد.



شکل ۸: نمودار خطای آموزش و اعتبارسنجی در طول دوره‌ها.

۱. همگرایی موفقیت‌آمیز: مدل در ابتدا به سرعت یاد گرفته و خطای آموزش و اعتبارسنجی را به شدت کاهش داده است.
 ۲. نشانه‌های بیش‌برازش (Overfitting): از حدود دوره ۴-۵ به بعد، شکاف بین خطای آموزش (که همچنان کاهش می‌یابد) و خطای اعتبارسنجی (که نوسان دارد و گاهی افزایش می‌یابد) شروع به ظاهر شدن می‌کند. این امر نشان می‌دهد که مدل در حال یادگیری جزئیاتی است که مختص داده‌های آموزشی است و ممکن است به تعمیم‌پذیری کلی آسیب بزند.
 ۳. عملکرد کلی: با وجود نشانه‌های بیش‌برازش، دقت نهایی اعتبارسنجی (حدود ۹۴.۷٪) نسبتاً خوب است و نشان می‌دهد که مدل توانایی تشخیص الگوهای تقارن را تا حد زیادی کسب کرده است.
- در مجموع، مدل RobustBiLSTM عملکرد قابل قبولی در تشخیص تقارن دنباله‌ها از خود نشان داده است، اما برای دستیابی به بهترین عملکرد و جلوگیری از بیش‌برازش، نیاز به تنظیم دقیق‌تر فرآیند آموزش و احتمالاً پیاده‌سازی تکنیک‌های منظم‌سازی پیشرفته‌تر دارد.

۱۳.۵ کد کامل

```

۱ import torch
۲ import torch.nn as nn
۳ from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
۴ import random
۵ from torch.utils.data import DataLoader
۶ from sklearn.model_selection import train_test_split
۷
۸
۹ class RobustBiLSTM(nn.Module):
۱۰     """
۱۱     - Capture both past and future contextual information (Bidirectional)

```



```
12 - Learn hierarchical temporal representations (Stacked LSTM)
13 - Reduce overfitting using Dropout (Regularization)
14 - Avoid unnecessary computation on padding tokens
15 - Handle variable-length sequences efficiently
16 """
17
18 def __init__(self,
19     vocab_size: int,
20     embedding_dim: int,
21     hidden_dim: int,
22     num_layers: int,
23     num_classes: int,
24     dropout: float = 5.0):
25     """
26     vocab_size : Number of tokens in vocabulary
27     embedding_dim : Dimensionality of token embeddings
28     hidden_dim : Number of hidden units in each LSTM direction
29     num_layers : Number of stacked LSTM layers
30     num_classes : Number of output classes for sequence classification
31     dropout : Dropout probability for regularization
32     """
33     super().__init__()
34
35
36
37     """
38     Calling super().init() initializes the parent nn.Module class, enabling parameter
39     registration, autograd, and proper model behavior during training.
40     """
41
42     self.hidden_dim = hidden_dim
43     self.num_layers = num_layers
44
45     # Embedding layer converts token indices into dense vectors
46     # padding_idx=0 ensures that padding tokens do not affect learning
47     self.embedding = nn.Embedding(
48         num_embeddings=vocab_size,
49         embedding_dim=embedding_dim,
50         padding_idx=0
51     )
```

```
۵۲
۵۳     self.lstm = nn.LSTM(input_size = embedding_dim,
۵۴                          hidden_size = hidden_dim,
۵۵                          num_layers = num_layers,
۵۶                          batch_first = True,
۵۷                          bidirectional = True, # bidirectional=True ensures that the model
۵۸                          dropout = dropout if num_layers > 1 else 0 # dropout is applied
۵۹                          )
۶۰
۶۱     self.dropout = nn.Dropout(dropout)
۶۲
۶۳     #self.fc = nn.Linear(hidden_dim * 2, num_classes) # Fully Connected layer for sequence
۶۴     # BiLSTM outputs hidden_dim * 2 features (forward + backward)
۶۵
۶۶     self.fc = nn.Sequential(
۶۷         nn.Linear(hidden_dim * 2, hidden_dim),
۶۸         nn.ReLU(),
۶۹         nn.Dropout(dropout),
۷۰         nn.Linear(hidden_dim, num_classes)
۷۱ )
۷۲
۷۳
۷۴     def forward (self, x: torch.Tensor, length: torch.Tensor, hidden=None) -> torch.Tensor:
۷۵         """
۷۶         Forward pass of the model
۷۷
۷۸         x : Padded input tensor of shape:
۷۹             (batch_size, max_sequence_length, input_dim)
۸۰
۸۱         lengths : True lengths of each sequence before padding
۸۲             Shape: (batch_size,)
۸۳
۸۴         Returns
۸۵         logits : Output logits for sequence classification
۸۶             Shape: (batch_size, num_classes)
۸۷         """
۸۸     x = self.embedding(x)
```



```
۸۹ packed_input = pack_padded_sequence(x,
۹۰                                 length.cpu(),
۹۱                                 batch_first = True,
۹۲                                 enforce_sorted = False
۹۳                                 )
۹۴
۹۵ """
۹۶ This operation removes padding tokens from computation.
۹۷ - No unnecessary computation on padding
۹۸ - No gradient noise from padded positions
۹۹ - Faster and more memory-efficient training
۱۰۰ """
۱۰۱
۱۰۲ packed_output , (h_n, c_n) = self.lstm(packed_input)
۱۰۳
۱۰۴ """ The hidden state h represents the final output of the LSTM and
۱۰۵ summarizes the sequence, while the cell state c stores long-term
۱۰۶ memory and enables stable gradient flow.
۱۰۷
۱۰۸ For a Bidirectional LSTM:
۱۰۹ - h_n[-2] → last layer, forward direction
۱۱۰ - h_n[-1] → last layer, backward direction
۱۱۱ """
۱۱۲
۱۱۳
۱۱۴ # Unpack the sequence (not used here). The model uses the final hidden states
۱۱۵ # (h_n) as the sequence representation instead of the full LSTM outputs.
۱۱۶ output, output_lengths = pad_packed_sequence(packed_output, batch_first=True)
۱۱۷
۱۱۸ # hidden state_n -> (num_layers * num_directions, batch_size, hidden_dim)
۱۱۹ # cell state_n -> (num_layers * num_directions, batch_size, hidden_dim)
۱۲۰
۱۲۱ h_forward = h_n[-2,:,:]
۱۲۲ h_backward = h_n[-1,:,:]
۱۲۳
۱۲۴
۱۲۵ #Concatenate forward and backward hidden states
۱۲۶ #This creates a complete sequence representation containing both past and future context.
۱۲۷ h_final = torch.cat((h_forward, h_backward), dim = 1) # (batch_size, hidden_dim * 2)
۱۲۸
```



```
۱۲۹         h_final = self.dropout(h_final)
۱۳۰         logits = self.fc(h_final)
۱۳۱
۱۳۲         return logits, (h_n, c_n)
۱۳۳
۱۳۴ def generate_balanced_symmetric_dataset(
۱۳۵     num_samples: int = 8000,
۱۳۶     positive_ratio: float = 6.0,
۱۳۷     vocab_size: int = 60,
۱۳۸     min_half_len: int = 3,
۱۳۹     max_half_len: int = 7,
۱۴۰     noise_len: int = 1,
۱۴۱     seed: int = 42
۱۴۲ ):
۱۴۳     """
۱۴۴     Generate a synthetic sequence classification dataset with controllable class balance.
۱۴۵
۱۴۶     The dataset is designed for sequence similarity classification:
۱۴۷         - Class 1 (Positive): Right half is approximately the reverse of the left half
۱۴۸         - Class 0 (Negative): Right half is random (dissimilar)
۱۴۹
۱۵۰     Each sequence structure:
۱۵۱         [ left | noise | right ]
۱۵۲
۱۵۳
۱۵۴     Parameters
۱۵۵     num_samples : int
۱۵۶         Total number of sequences to generate.
۱۵۷
۱۵۸     positive_ratio : float
۱۵۹         Proportion of positive (symmetric) samples.
۱۶۰         Example:
۱۶۱             5.0 → 50% positive / 50% negative
۱۶۲             6.0 → 60% positive / 40% negative
۱۶۳
۱۶۴     vocab_size : int
۱۶۵         Number of unique tokens in vocabulary.
۱۶۶         Token index 0 is reserved for padding (not used here).
۱۶۷
۱۶۸     min_half_len : int
```



```
169         Minimum length of the left (and right) segment.
170
171     max_half_len : int
172         Maximum length of the left (and right) segment.
173
174     noise_len : int
175         Number of random tokens inserted between left and right segments.
176
177     seed : int
178         Random seed for reproducibility.
179
180
181     Returns
182     sequences : List[torch.Tensor]
183         List of variable-length sequences.
184
185     labels : torch.Tensor
186         Tensor of shape (num_samples,) containing class labels (0 or 1).
187     """
188
189
190     random.seed(seed)
191     torch.manual_seed(seed)
192
193     sequences = []
194     labels = []
195
196     for _ in range(num_samples):
197
198         # Randomly choose length of left half
199         half_len = random.randint(min_half_len, max_half_len)
200
201         # Generate left segment (avoid token 0 since it is padding)
202         left = torch.randint(1, vocab_size, (half_len,))
203
204         # Generate small random noise segment
205         noise = torch.randint(1, vocab_size, (noise_len,))
206
207         # Decide whether sample is Positive or Negative
208         if random.random() < positive_ratio:
```



```
209
210     # Reverse the left sequence
211     right = torch.flip(left, dims=[0])
212
213     # Add small controlled perturbation
214     # This makes the task slightly harder and more realistic
215     noise_val = torch.randint(-1, 2, right.shape)
216     right = torch.clamp(right + noise_val, 1, vocab_size - 1)
217
218     label = 1
219
220     else:
221         # Negative Sample (Random)
222         # Generate completely random right half
223         right = torch.randint(1, vocab_size, (half_len,))
224         label = 0
225
226     # Final format: [left | noise | right]
227     seq = torch.cat([left, noise, right])
228
229     sequences.append(seq)
230     labels.append(label)
231
232     combined = list(zip(sequences, labels))
233     random.shuffle(combined)
234     sequences, labels = zip(*combined)
235
236     # Convert labels to tensor
237     labels = torch.tensor(labels, dtype=torch.long)
238
239     return list(sequences), labels
240
241 from torch.utils.data import Dataset, DataLoader
242
243 class SequenceDataset(Dataset):
244     """
245     Custom PyTorch Dataset for handling variable-length sequence data.
246
247     This dataset stores:
248     - sequences : list of token sequences (each sequence is a tensor)
```




```
۲۴۹         - labels      : corresponding class labels for each sequence
۲۵۰
۲۵۱     It allows PyTorch DataLoader to efficiently sample and batch data
۲۵۲     during training and evaluation.
۲۵۳     """
۲۵۴
۲۵۵     def __init__(self, sequences, labels):
۲۵۶
۲۵۷         self.sequences = sequences
۲۵۸         self.labels = labels
۲۵۹
۲۶۰     def __getitem__(self, idx):
۲۶۱         """
۲۶۲         idx : Index of the sample.
۲۶۳
۲۶۴         Returns
۲۶۵         sequence : Token sequence at index idx.
۲۶۶
۲۶۷         label : Class label corresponding to the sequence.
۲۶۸         """
۲۶۹         return self.sequences[idx], self.labels[idx]
۲۷۰
۲۷۱     def __len__(self):
۲۷۲         """
۲۷۳         Return the total number of samples in the dataset.
۲۷۴         This allows DataLoader to know how many items exist.
۲۷۵         """
۲۷۶         return len(self.labels)
۲۷۷
۲۷۸
۲۷۹     def collate_fn(batch):
۲۸۰         """
۲۸۱         Custom collate function used by DataLoader to create batches.
۲۸۲
۲۸۳         Since sequences have variable lengths, they cannot be directly stacked
۲۸۴         into a tensor. This function:
۲۸۵             1. Extracts sequences and labels from the batch
۲۸۶             2. Computes the true length of each sequence
۲۸۷             3. Pads sequences to the same length
۲۸۸             4. Returns tensors suitable for RNN/LSTM models
```



```
۲۸۹     """
۲۹۰
۲۹۱     # Separate sequences and labels from the batch
۲۹۲     sequences, labels = zip(*batch)
۲۹۳
۲۹۴     # Compute original sequence lengths before padding
۲۹۵     lengths = torch.tensor([len(seq) for seq in sequences], dtype=torch.long)
۲۹۶
۲۹۷     # Pad sequences so they all have the same length
۲۹۸     # padding_value=0 corresponds to the padding token
۲۹۹     padded = nn.utils.rnn.pad_sequence(
۳۰۰         sequences,
۳۰۱         batch_first=True,
۳۰۲         padding_value=0
۳۰۳     )
۳۰۴
۳۰۵     # Convert labels to tensor
۳۰۶     labels = torch.tensor(labels, dtype=torch.long)
۳۰۷
۳۰۸     return padded, lengths, labels
۳۰۹
۳۱۰ sequences, labels = generate_balanced_symmetric_dataset(
۳۱۱     num_samples=10000,
۳۱۲     vocab_size=60
۳۱۳ )
۳۱۴
۳۱۵ X_train, X_val, y_train, y_val = train_test_split(
۳۱۶     sequences,
۳۱۷     labels,
۳۱۸     test_size=2.0,
۳۱۹     stratify=labels,
۳۲۰     random_state=42
۳۲۱ )
۳۲۲
۳۲۳ train_ds = SequenceDataset(X_train, y_train)
۳۲۴ val_ds = SequenceDataset(X_val, y_val)
۳۲۵
۳۲۶ train_loader = DataLoader(
۳۲۷     train_ds, batch_size=32, shuffle=True, collate_fn=collate_fn
۳۲۸ )
```



```
۳۲۹ val_loader = DataLoader(
۳۳۰     val_ds, batch_size=32, shuffle=False, collate_fn=collate_fn
۳۳۱ )
۳۳۲
۳۳۳ import matplotlib.pyplot as plt
۳۳۴ import numpy as np
۳۳۵
۳۳۶ # Compute class distribution
۳۳۷ unique, counts = np.unique(labels.numpy(), return_counts=True)
۳۳۸
۳۳۹ plt.figure(figsize=(6, 4))
۳۴۰ plt.bar(['Class 0 (Dissimilar)', 'Class 1 (Similar)'], counts, color=['salmon', 'skyblue'])
۳۴۱ plt.title("Class Distribution (Full Dataset)")
۳۴۲ plt.ylabel("Number of Samples")
۳۴۳ plt.show()
۳۴۴
۳۴۵ print(f"Class 0 count: {counts[0]}")
۳۴۶ print(f"Class 1 count: {counts[1]}")
۳۴۷ print(f"Class 1 proportion: {counts[1] / len(labels):2%.}")
۳۴۸
۳۴۹ lengths = [len(seq) for seq in sequences]
۳۵۰
۳۵۱ plt.figure(figsize=(6, 4))
۳۵۲ plt.hist(lengths, bins=range(min(lengths), max(lengths) + 2), color='purple', alpha=0.7,
→   edgecolor='black')
۳۵۳ plt.title("Sequence Length Distribution")
۳۵۴ plt.xlabel("Sequence Length")
۳۵۵ plt.ylabel("Frequency")
۳۵۶ plt.grid(axis='y', linestyle='--', alpha=7.0)
۳۵۷ plt.show()
۳۵۸
۳۵۹ print(f"Average sequence length: {np.mean(lengths):2f}.")
۳۶۰ print(f"Max sequence length: {max(lengths)}")
۳۶۱
۳۶۲ import pandas as pd
۳۶۳
۳۶۴ train_labels = y_train.numpy()
۳۶۵ val_labels = y_val.numpy()
۳۶۶
۳۶۷ labels_dist = pd.DataFrame({
```



```

۳۶۸     'Train': [np.sum(train_labels == 0), np.sum(train_labels == 1)],
۳۶۹     'Val':   [np.sum(val_labels == 0),   np.sum(val_labels == 1)]
۳۷۰ }, index=['Class 0', 'Class 1'])
۳۷۱
۳۷۲ labels_dist.plot(kind='bar', figsize=(7, 4), color=['orange', 'green'])
۳۷۳ plt.title("Train vs Validation Class Distribution")
۳۷۴ plt.ylabel("Count")
۳۷۵ plt.xticks(rotation=0)
۳۷۶ plt.show()
۳۷۷
۳۷۸ print("--- Inspecting first 3 samples ---")
۳۷۹ for i in range(3):
۳۸۰     seq = sequences[i]
۳۸۱     lbl = labels[i].item()
۳۸۲
۳۸۳     mid = len(seq) // 2
۳۸۴     left  = seq[:mid]
۳۸۵     noise = seq[mid:mid+1]
۳۸۶     right = seq[mid+1:]
۳۸۷
۳۸۸     print(f"\nSample {i+1} | Label: {lbl}")
۳۸۹     print(f"Left part:      {left.tolist()}")
۳۹۰     print(f"Noise token:    {noise.tolist()}")
۳۹۱     print(f"Right part:     {right.tolist()}")
۳۹۲     print(f"Full sequence: {seq.tolist()}")
۳۹۳     print("-" * 50)
۳۹۴
۳۹۵ def init_orthogonal_lstm(model):
۳۹۶     """
۳۹۷     Apply specialized weight initialization for LSTM-based models.
۳۹۸     This function initializes LSTM parameters using
۳۹۹     strategies that help prevent vanishing/exploding gradients and
۴۰۰     improve convergence.
۴۰۱
۴۰۲     Initialization strategy:
۴۰۳     • weight_hh (Hidden-to-Hidden weights)
۴۰۴         Initialized using Orthogonal Initialization.
۴۰۵         This helps preserve gradient norms across time steps and
۴۰۶         stabilizes long sequence training.
۴۰۷

```



```
۴۰۸     • weight_ih (Input-to-Hidden weights)
۴۰۹         Initialized using Xavier (Glorot) Uniform initialization.
۴۱۰         This keeps the variance of activations stable across layers.
۴۱۱
۴۱۲     • bias terms
۴۱۳         Initialized to zero for neutral starting conditions.
۴۱۴     """
۴۱۵
۴۱۶     for name, param in model.named_parameters():
۴۱۷
۴۱۸         # Hidden-to-hidden recurrent weights of LSTM
۴۱۹         if "weight_hh" in name:
۴۲۰             nn.init.orthogonal_(param)
۴۲۱
۴۲۲         # Input-to-hidden weights
۴۲۳         elif "weight_ih" in name:
۴۲۴             nn.init.xavier_uniform_(param)
۴۲۵
۴۲۶         # Bias terms for LSTM gates
۴۲۷         elif "bias" in name:
۴۲۸             nn.init.zeros_(param)
۴۲۹
۴۳۰     vocab_size = 60
۴۳۱     embedding_dim = 64
۴۳۲     hidden_dim = 128
۴۳۳     num_layers = 3
۴۳۴     num_classes = 2
۴۳۵
۴۳۶     model = RobustBiLSTM(
۴۳۷         vocab_size=vocab_size,
۴۳۸         embedding_dim=embedding_dim,
۴۳۹         hidden_dim=hidden_dim,
۴۴۰         num_layers=num_layers,
۴۴۱         num_classes=num_classes,
۴۴۲         dropout=3.0
۴۴۳     )
۴۴۴
۴۴۵     init_orthogonal_lstm(model)
۴۴۶
۴۴۷     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
۴۴۸
۴۴۹ criterion = nn.CrossEntropyLoss()
۴۵۰
۴۵۱ optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
۴۵۲
۴۵۳ model.to(device)
۴۵۴
۴۵۵ num_epochs = 20
۴۵۶ max_grad_norm = 0.4
۴۵۷ def detach_hidden(hidden):
۴۵۸     if hidden is None:
۴۵۹         return None
۴۶۰     return tuple(h.detach() for h in hidden)
۴۶۱
۴۶۲ import time
۴۶۳ import torch.nn.utils as utils
۴۶۴
۴۶۵ train_losses = []
۴۶۶ val_losses = []
۴۶۷
۴۶۸ for epoch in range(1, num_epochs + 1):
۴۶۹
۴۷۰     start_time = time.time()
۴۷۱
۴۷۲     model.train()
۴۷۳
۴۷۴     running_train_loss = 0.0
۴۷۵     hidden = None
۴۷۶     for batch_idx, (x, lengths, y) in enumerate(train_loader, 1):
۴۷۷
۴۷۸         x = x.to(device)
۴۷۹         y = y.to(device)
۴۸۰         lengths = lengths.cpu()
۴۸۱
۴۸۲         optimizer.zero_grad()
۴۸۳         hidden = detach_hidden(hidden)
۴۸۴         logits, hidden = model(x, lengths, hidden)
۴۸۵         loss = criterion(logits, y)
۴۸۶
۴۸۷         loss.backward()
```



```
۴۸۸
۴۸۹     utils.clip_grad_norm_(model.parameters(), max_grad_norm)
۴۹۰
۴۹۱     optimizer.step()
۴۹۲
۴۹۳     running_train_loss += loss.item()
۴۹۴
۴۹۵     avg_train_loss = running_train_loss / len(train_loader)
۴۹۶     train_losses.append(avg_train_loss)
۴۹۷
۴۹۸     model.eval()
۴۹۹
۵۰۰     running_val_loss = 0.0
۵۰۱     correct = 0
۵۰۲     total = 0
۵۰۳
۵۰۴     with torch.no_grad():
۵۰۵
۵۰۶         for x, lengths, y in val_loader:
۵۰۷
۵۰۸             x = x.to(device)
۵۰۹             y = y.to(device)
۵۱۰             lengths = lengths.cpu()
۵۱۱
۵۱۲             logits,_ = model(x, lengths, hidden=None)
۵۱۳             loss = criterion(logits, y)
۵۱۴
۵۱۵             running_val_loss += loss.item()
۵۱۶
۵۱۷             preds = torch.argmax(logits, dim=1)
۵۱۸
۵۱۹             correct += (preds == y).sum().item()
۵۲۰             total += y.size(0)
۵۲۱
۵۲۲     avg_val_loss = running_val_loss / len(val_loader)
۵۲۳     val_losses.append(avg_val_loss)
۵۲۴
۵۲۵     val_acc = 0.100 * correct / total
۵۲۶     elapsed = time.time() - start_time
۵۲۷
```



```
۵۲۸     print("=" * 80)
۵۲۹     print(f"Epoch {epoch}/{num_epochs}")
۵۳۰     print(f"Train Loss      : {avg_train_loss:4f.}")
۵۳۱     print(f"Validation Loss : {avg_val_loss:4f.}")
۵۳۲     print(f"Validation Acc  : {val_acc:2f.}%")
۵۳۳     print(f"Epoch Time    : {elapsed:2f.} sec")
۵۳۴     print("=" * 80)
۵۳۵
۵۳۶ import matplotlib.pyplot as plt
۵۳۷
۵۳۸ plt.figure(figsize=(9, 5))
۵۳۹ plt.plot(train_losses, label="Train Loss", marker="o")
۵۴۰ plt.plot(val_losses, label="Validation Loss", marker="s")
۵۴۱ plt.xlabel("Epoch")
۵۴۲ plt.ylabel("Loss")
۵۴۳ plt.title("Training & Validation Loss")
۵۴۴ plt.legend()
۵۴۵ plt.grid(True)
۵۴۶ plt.tight_layout()
۵۴۷ plt.show()
```